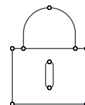
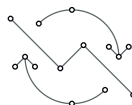
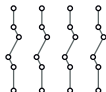
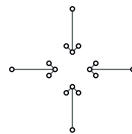
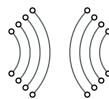
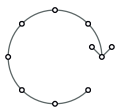
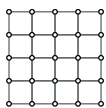


01

INTRODUCTION



The pace at which computer systems change was, is, and continues to be overwhelming. From 1945, when the modern computer era began, until about 1985, computers were large and expensive. Moreover, lacking a way to connect them, these computers operated independently of one another.

Starting in the mid-1980s, however, two advances in technology began to change that situation. The first was the development of powerful microprocessors. Initially, these were 8-bit machines, but soon 16-, 32-, and 64-bit CPUs became common. With powerful multicore CPUs, we now are again facing the challenge of adapting and developing programs to exploit parallelism. In any case, the current generation of machines have the computing power of the mainframes deployed 30 or 40 years ago, but for 1/1000th of the price or less.

The second development was the invention of high-speed computer networks. **Local-area networks** or **LANs** allow thousands of machines within a building to be connected in such a way that small amounts of information can be transferred in a few microseconds or so. Larger amounts of data can be moved between machines at rates of billions of *bits per second* (**bps**). **Wide-area network** or **WANs** allow hundreds of millions of machines all over the earth to be connected at speeds varying from tens of thousands to hundreds of millions bps and more.

Parallel to the development of increasingly powerful and networked machines, we have also been able to witness miniaturization of computer systems, with perhaps the smartphone as the most impressive outcome. Packed with sensors, lots of memory, and a powerful multicore CPU, these devices are nothing less than full-fledged computers. Of course, they also have networking capabilities. Along the same lines, so-called **nano computers** have become readily available. These small, single-board computers, often the size of a credit card, can easily offer near-desktop performance. Well-known examples include Raspberry Pi and Arduino systems.

And the story continues. As digitalization of our society continues, we become increasingly aware of how many computers are actually being used, regularly embedded into other systems such as cars, airplanes, buildings, bridges, the power grid, and so on. This awareness is, unfortunately, increased when such systems suddenly turn out to be hackable. For example, in 2021, a fuel pipeline in the United States was effectively shut down by a ransomware attack. In this case, the computer system consisted of a mix of sensors, actuators, controllers, embedded computers, servers, etc., all brought together into a single system. What many of us do not realize, is that vital infrastructures, such as fuel pipelines, are monitored and controlled by **networked computer systems**. Along the same lines, it may be time to start realizing that a modern car is actually an autonomously operating, mobile networked computer. In this case, instead of the mobile computer being carried by a person, we need to deal with the mobile computer carrying people.

The size of a networked computer system may vary from a handful of devices, to millions of computers. The interconnection network may be wired, wireless, or a combination of both. Moreover, these systems are often highly dynamic, in the sense that computers can join and leave, with the topology and performance of the underlying network almost continuously changing.

It is difficult to think of computer systems that are *not* networked. And as a matter of fact, most networked computer systems can be accessed from any place in the world because they are hooked up to the Internet. Studying to understand these systems can easily become exceedingly complex. In this chapter, we start with shedding some light on what needs to be understood to build up the bigger picture without getting lost.

1.1 From networked systems to distributed systems

Before we dive into various aspects of distributed systems, let us first consider what distribution, or decentralization, actually entails.

1.1.1 Distributed versus decentralized systems

When considering various sources, there are quite a few opinions on distributed versus decentralized systems. Often, the distinction is illustrated by three different organizations of networked computer systems, as shown in Figure 1.1, where each node represents a computer system and an edge a communication link between two nodes.

To what extent such distinctions are useful remains to be seen, especially when discussions open on the pros and cons of each organization. For example, it is often stated that centralized organizations do not scale well. Likewise, distributed organizations are said to be more robust against failures. As we shall see, none of these claims are generally true.

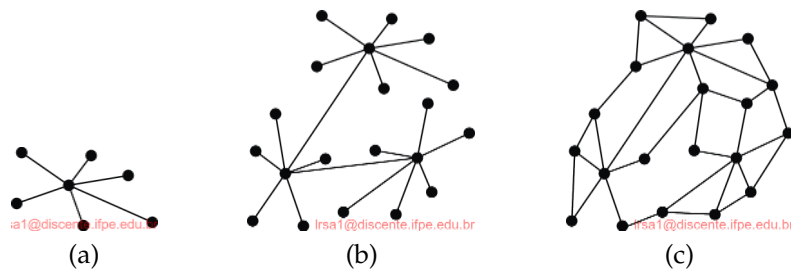


Figure 1.1: The organization of a (a) centralized, (b) decentralized, and (c) distributed system, according to various popular sources. We take a different approach, as figures such as these are really not that meaningful.

We take a different approach. If we think of a networked computer system as a collection of computers connected in a network, we can ask ourselves how these computers even became connected to each other in the first place. There are roughly two views that one can take.

The first, **integrative view**, is that there was a need to connect existing (networked) computer systems to each other. Typically, this happens when services running on a system need to be made available to users and applications that were not thought of before. This may happen, for example, when integrating financial services with project management services, as is often the case within a single organization. In the scientific-research domain, we have seen efforts to connect a myriad of often expensive resources (special-purpose computers, supercomputers, very large database systems, etc.) into what came to be known as a grid computer.

The second, **expansive view** is that an existing system required an extension through additional computers. This view is the one most often related to the field of distributed systems. It entails expanding a system with computers to hold resources close to where those resources are needed. An expansion may also be driven by the need to improve dependability: if one computer fails, then there are others who can take over. An important type of expansion is when a service needs to be made available for remote users and applications, for example, by offering a Web interface or a smartphone application. This last example also shows that the distinction between an integrative and an expansive view is not a clear-cut.

In both cases, we see that the networked system runs services, where each service is implemented as a collection of processes and resources spread across multiple computers. The two views lead to a natural distinction between two types of networked computer systems:

- A **decentralized system** is a networked computer system in which processes and resources are **necessarily** spread across multiple computers.
- A **distributed system** is a networked computer system in which processes and resources are **sufficiently** spread across multiple computers.

Before we discuss why this distinction is important, let us look at a few examples of each type of system.

Decentralized systems are mainly related to the integrative view of networked computer systems. They come to being because we want to connect systems, yet may be hindered by administrative boundaries. For example, many applications in the artificial-intelligence domain require massive amounts of data for building reliable predictive models. Normally, data is brought to the high-performance computers that literally train models before they can be used. But when data needs to stay within the perimeter of an organization (and there can be many reasons why this is necessary), we need to bring the training to the data. The result is known as **federated learning**,

and is implemented by a decentralized system, where the need for spreading processes and resources is dictated by administrative policies.

Another example of a decentralized system is that of **distributed ledger**, also known as a **blockchain**. In this case, we need to deal with the situation that participating parties do not trust each other enough to set up simple schemes for collaboration. Instead, what they do is essentially make transactions among each other fully public (and verifiable) by an extend-only ledger that keeps records of those transactions. The ledger itself is fully spread across the participants, and the participants are the ones who validate transactions (of others) before admitting them to the ledger. The result is a decentralized system in which processes and resources are, indeed, *necessarily* spread across multiple computers, in this case due to lack of trust.

As a last example of a decentralized system, consider systems that are naturally geographically dispersed. This occurs typically with systems in which an actual location needs to be monitored, for example, in the case of a power plant, a building, a specific natural environment, and so on. The system, controlling the monitors and where decisions are made, may easily be placed somewhere else than the location being monitored. One obvious example is monitoring and controlling of satellites, but also more mundane situations as monitoring and controlling traffic, trains, etc. In these examples, the necessity for spreading processes and resources comes from a spatial argument.

As we mentioned, distributed systems are mainly related to the expansive view of networked computer systems. A well-known example is making use of e-mail services, such as Google Mail. What often happens is that a user logs into the system through a Web interface to read and send mails. More often, however, is that users configure their personal computer (such as a laptop) to make use of a specific mail client. To that end, they need to configure a few settings, such as the incoming and outgoing server. In the case of Google Mail, these are `imap.gmail.com` and `smtp.gmail.com`, respectively. Logically, it seems as if these two servers will handle all your mail. However, with an estimate of close to 2 billion users as of 2022, it is unlikely that only two computers can handle all their e-mails (which was estimated to be more than 300 billion per year, that is, some 10,000 mails *per second*). Behind the scenes, of course, the entire Google Mail service has been implemented and spread across many computers, jointly forming a distributed system. That system has been set up to make sure that so many users can process their mails (i.e., ensures scalability), but also that the risk of losing mail because of failures, is minimal (i.e., the system ensures fault tolerance). To the user, however, the image of just two servers is kept up (i.e., the distribution itself is highly transparent to the user). The distributed system implementing an e-mail service, such as Google Mail, typically expands (or shrinks) as dictated by dependability requirements, in turn, dependent on the number of its users.

An entirely different type of distributed system is formed by the collection of so-called **Content Delivery Networks**, or **CDNs** for short. A well-known example is Akamai with, in 2022, over 400,000 servers worldwide. We will discuss the principle working of CDNs later in Chapter 3. What it boils down to, is that the content of an actual Website, is copied and spread across various servers of the CDN. When visiting a Website, the user is transparently redirected to a nearby server that holds all or part of the content of that Website. The choice for which server to direct a user to may depend on many things, but surely when dealing with streaming content, a server is selected for which good performance in terms of latency and bandwidth can be guaranteed. The CDN dynamically ensures that the selected server will have the required content readily available, as well as update that content when needed, or remove it from the server when there are no or very few users to service there. Meanwhile, the user knows nothing about what is going on behind the scenes (which, again, is a form of distribution transparency). We also see in this example, that content is not copied to all servers, yet only to where it makes sense, that is, *sufficiently*, and for reasons of performance. CDNs also copy content to multiple servers to provide high levels of dependability.

As a final, much smaller distributed system, consider a setup based on a **Network-Attached Storage device**, also called a **NAS**. For domestic use, a typical NAS consists of 2–4 slots for internal hard disks. The NAS operates as a file server: it is accessible through a (generally wireless) network for any authorized device, and as such can offer services like shared storage, automated backups, streaming media, and so on. The NAS itself can best be seen as a single computer optimized for storing files, and offering the ability to easily share those files. The latter is important, and together with multiple users, we essentially have a setup of a distributed system. The users will be working with a set of files that are locally (i.e., from their laptop) easily accessible (in fact, seemingly integrated into the local file system), while also directly accessible by and for other users. Again, where and how the shared files are stored is hidden (i.e., the distribution is transparent). Assuming that sharing files is the goal, then we see that indeed a NAS can provide sufficient spreading of processes and resources.

Note 1.1 (More information: Are centralized solutions bad?)

There appears to be a stubborn misconception that centralized solutions cannot scale. Moreover, they are almost always associated with introducing a single point of failure. Both reasons are often seen to be enough to dismiss centralized solutions as being a good choice when designing distributed systems.

What many people forget is that a difference should be made between logical and physical designs. A logically centralized solution can be implemented in a highly scalable distributed manner. An excellent example is the **Domain Name System (DNS)**, which we discuss extensively in Chapter 6. Logically, DNS is

organized as a huge tree, where each path from the root to a leaf node represents a fully qualified name, such as `www.distributed-systems.net`. It would be a mistake to think that the root node is implemented by just a single server. In fact, the root node is implemented by 13 different root servers, each server, in turn, implemented as a large cluster computer. The physical organization of DNS also shows that the root is *not* a single point of failure. Being highly replicated, it would take serious efforts to bring that root down and so far, all attempts to do so have failed.

Centralized solutions are not bad just because they seem to be centralized. In fact, as we shall encounter many times throughout this book, (logically, and even physically) centralized solutions are often much better than distributed counterparts for the simple reason that there *is* a single point of failure. It makes them much easier to manage, for example, and certainly in comparison where there may be multiple points of failures. Moreover, that single point of failure can be hardened against many kinds of failures as well as many kinds of security attacks. When it comes to being a performance bottleneck, we will also see that many things can be done to ensure that even that cannot be held against centralization.

In this sense, let us not forget that centralized solutions have even proven to be extremely scalable and robust. They are called cloud-based solutions. Again, their implementations can make use of very sophisticated distributed solutions, yet even then, we shall see that even those solutions may sometimes need to rely on a small set of physical machines, if only to guarantee performance.

1.1.2 Why making the distinction is relevant

Why do we make this distinction between decentralized and distributed systems? It is important to realize that centralized solutions are generally much simpler, and also simpler along different criteria. Decentralization, that is, the act of spreading the implementation of a service across multiple computers because we believe it is *necessary*, is a decision that needs to be considered carefully. Indeed, distributed and decentralized solutions are inherently difficult:

- There are many, often unexpected, dependencies that hinder understanding the behavior of these systems.
- Distributed and decentralized systems suffer almost continuously from **partial failures**: some process or resource, somewhere at one of the participating computers, is not operating according to expectations. Discovering that failure may actually take some time, while also such failures are preferably masked (i.e., they go unnoticed for users and applications), including the recovery from failures.

- Much related to partial failures is the fact that in many networked computer systems, participating nodes, processes, resources, and so on, come and go. This makes these systems highly dynamic, in turn requiring forms of automated management and maintenance, in turn increasing the complexity.
- The fact that distributed and decentralized systems are networked, used by many users and applications, and often cross multiple administrative boundaries, make them particularly vulnerable to security attacks. Therefore, understanding these systems and their behavior, requires that we understand how they can be, and are secured. Unfortunately, understanding security is not that easy.

Our distinction is one between *sufficiency* and *necessity* for spreading processes and resources across multiple computers. Throughout this book, we take the standpoint that decentralization can never be a goal in itself, and that it should focus on the *sufficiency* for spreading processes and resources across computers. In principle, the less spreading, the better. Yet at the same time, we need to realize that spreading is sometimes truly necessary, as illustrated by the examples of decentralized systems. From this point of sufficiency, the book is truly about distributed systems and where appropriate, we shall speak of decentralized systems.

Along the same lines, considering that distributed and decentralized systems are inherently complex, it is equally important to consider solutions that are as simple as possible. Therefore, we shall hardly discuss optimizations to solutions, firmly believing that the impact of their negative contribution to increased complexity outweighs the importance of their positive contribution to an increase of any type of performance.

1.1.3 Studying distributed systems

Considering that distributed systems are inherently difficult, it is important to take a systematic approach toward studying them. One of our major concerns is that there are so many explicit and implicit dependencies in distributed systems. For example, there is no such thing as a separate communication module, or a separate security module. Our approach is to take a look at distributed systems from a limited number, yet different perspectives. Each perspective is considered in a separate chapter.

- There are many ways in which distributed systems are organized. We start our discussion by taking the **architectural perspective**: what are common organizations, what are common styles? The architectural perspective will help in getting a first grip on how various components of existing systems interact and depend on each other.

- Distributed systems are all about processes. The **process perspective** is all about understanding the different forms of processes that occur in distributed systems, be they threads, their virtualization of hardware processes, clients, servers, and so on. Processes form the software backbone of distributed systems, and their understanding is essential for understanding distributed systems.
- Obviously, with multiple computers at stake, communication between processes is essential. The **communication perspective** concerns the facilities that distributed systems provide to exchange data between processes. It essentially entails mimicking procedure calls across multiple computers, high-level message passing with a wealth of semantic options, and various sorts of communication between sets of processes.
- To make distributed systems work, what happens under the hood on top of which applications are executed, is that processes coordinate things. They jointly coordinate, for example, to compensate for the lack of global clock, for realizing mutual exclusive access to shared resources, and so on. The **coordination perspective** describes a number of fundamental coordination tasks that need to be carried out as part of most distributed systems.
- To access processes and resources, we need naming. In particular, we need naming schemes that, when used, will lead to the process, resources, or whatever other type of entity that is being named. As simple as this may seem, naming not only turns out to be crucial in distributed systems, there are also many ways in which naming is supported. The **naming perspective** focuses entirely on resolving a name to the access point of the named entity.
- A critical aspect of distributed systems is that they perform well in terms of efficiency and in terms of dependability. The key instrument for both aspects is replicating resources. The only problem with replication is that updates may happen, implying that all copies of a resource need to be updated as well. It is here, that keeping up the appearance of a nondistributed system becomes challenging. The **consistency and replication perspective** essentially concentrates on the trade-offs between consistency, replication, and performance.
- We already mentioned that distributed systems are subject to partial failures. The **perspective of fault tolerance** dives into the means for masking failures and their recovery. It has proven to be one of the toughest perspectives for understanding distributed systems, mainly because there are so many trade-offs to be made, and also because completely masking failures and their recovery is provably impossible.

- As also mentioned, there is no such thing as a nonsecured distributed system. The **security perspective** focuses on how to ensure authorized access to resources. To that end, we need to discuss trust in distributed systems, along with authentication, namely verifying a claimed identity. The security perspective comes last, yet later in this chapter we shall discuss a few basic instruments that are needed to understand the role of security in the previous perspectives.

1.2 Design goals

Just because it is possible to build distributed systems does not necessarily mean that it is a good idea. In this section, we discuss four important goals that should be met to make building a distributed system worth the effort. A distributed system should make resources easily accessible; it should hide the fact that resources are distributed across a network; it should be open; and it should be scalable.

1.2.1 Resource sharing

An important goal of a distributed system is to make it easy for users (and applications) to access and share remote resources. Resources can be virtually anything, but typical examples include peripherals, storage facilities, data, files, services, and networks, to name just a few. There are many reasons for wanting to share resources. One obvious reason is that of economics. For example, it is cheaper to have a single high-end reliable storage facility be shared than having to buy and maintain storage for each user separately.

Connecting users and resources also makes it easier to collaborate and exchange information, as is illustrated by the success of the Internet with its simple protocols for exchanging files, mail, documents, audio, and video. The connectivity of the Internet has allowed geographically widely dispersed groups of people to work together by all kinds of **groupware**, that is, software for collaborative editing, teleconferencing, and so on, as is illustrated by multinational software-development companies that have outsourced much of their code production to Asia, but also the myriad of collaboration tools that became (more easily) available due to the COVID-19 pandemic.

Resource sharing in distributed systems is also illustrated by the success of file-sharing peer-to-peer networks like **BitTorrent**. These distributed systems make it simple for users to share files across the Internet. Peer-to-peer networks are often associated with distribution of media files such as audio and video. In other cases, the technology is used for distributing large amounts of data, as in the case of software updates, backup services, and data synchronization across multiple servers.

Seamless integration of resource-sharing facilities in a networked environment is also now commonplace. A group of users can simply place files into a

special shared folder that is maintained by a third party somewhere on the Internet. Using special software, the shared folder is barely distinguishable from other folders on a user's computer. In effect, these services replace the use of a shared directory on a local distributed file system, making data available to users independent of the organization they belong to, and independent of where they are. The service is offered for different operating systems. Where exactly data are stored is completely hidden from the end user.

1.2.2 Distribution transparency

An important goal of a distributed system is to hide the fact that its processes and resources are physically distributed across multiple computers, possibly separated by large distances. In other words, it tries to make the distribution of processes and resources **transparent**, that is, invisible, to end users and applications. As we shall discuss more extensively in Chapter 2, achieving distribution transparency is realized through what is known as **middleware**, sketched in Figure 1.2 (see Gazis and Katsiri [2022] for a first introduction).

In essence, what applications get to see is the same interface everywhere, whereas behind that interface, where and how processes and resources are and how they are accessed is kept transparent. There are different types of transparency, which we discuss next.

Types of distribution transparency

The concept of transparency can be applied to several aspects of a distributed system, of which the most important ones are listed in Figure 1.3. We use the term *object* to mean either a process or a resource.

Access transparency deals with hiding differences in data representation and the way that objects can be accessed. At a basic level, we want to hide differences in machine architectures, but more important is that we reach

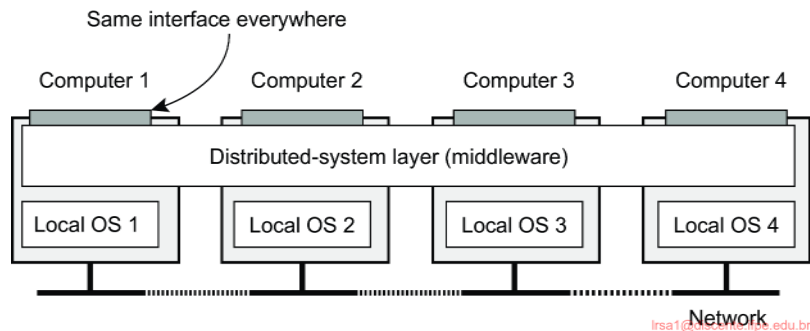


Figure 1.2: Realizing distribution transparency through a middleware layer.

Transparency	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Figure 1.3: Different forms of transparency in a distributed system (see ISO [1995]). An object can be a resource or a process.

agreement on how data is to be represented by different machines and operating systems. For example, a distributed system may have computer systems that run different operating systems, each having their own file-naming conventions. Differences in naming conventions, differences in file operations, or differences in how low-level communication with other processes is to take place, are examples of access issues that should preferably be hidden from users and applications.

An important group of transparency types concerns the location of a process or resource. **Location transparency** refers to the fact that users cannot tell where an object is physically located in the system. Naming plays an important role in achieving location transparency. In particular, location transparency can often be achieved by assigning only logical names to resources, that is, names in which the location of a resource is not secretly encoded. An example of a such a name is the **uniform resource locator (URL)** <https://www.distributed-systems.net/>, which gives no clue about the actual location of the Web server where this book is offered. The URL also gives no clue whether files at that site have always been at their current location or were recently moved there. For example, the entire site may have been moved from one data center to another, yet users should not notice. The latter is an example of **relocation transparency**, which is becoming increasingly important in the context of **cloud computing**: the phenomenon by which services are provided by huge collections of remote servers. We return to cloud computing in subsequent chapters, and, in particular, in Chapter 2.

Where relocation transparency refers to *being* moved by the distributed system, **migration transparency** is offered by a distributed system when it supports the mobility of processes and resources initiated by users, without affecting ongoing communication and operations. A typical example is communication between mobile phones: regardless whether two people

are actually moving, mobile phones will allow them to continue their conversation. Other examples that come to mind include online tracking and tracing of goods as they are being transported from one place to another, and teleconferencing (partly) using devices that are equipped with mobile Internet.

As we shall see, replication plays an important role in distributed systems. For example, resources may be replicated to increase availability or to improve performance by placing a copy close to the place where it is accessed. **Replication transparency** deals with hiding the fact that several copies of a resource exist, or that several processes are operating in some form of lockstep mode so that one can take over when another fails. To hide replication from users, it is necessary that all replicas have the same name. Consequently, a system that supports replication transparency should generally support location transparency as well, because it would otherwise be impossible to refer to replicas at different locations.

We already mentioned that an important goal of distributed systems is to allow sharing of resources. In many cases, sharing resources is done cooperatively, as in the case of communication channels. However, there are also many examples of competitive sharing of resources. For example, two independent users may each have stored their files on the same file server or may be accessing the same tables in a shared database. In such cases, it is important that each user does not notice that the other is making use of the same resource. This phenomenon is called **concurrency transparency**. An important issue is that concurrent access to a shared resource leaves that resource in a consistent state. Consistency can be achieved through locking mechanisms, by which users are, in turn, given exclusive access to the desired resource. A more refined mechanism is to make use of transactions, but these may be difficult to implement in a distributed system, notably when scalability is an issue.

Last, but certainly not least, it is important that a distributed system provides **failure transparency**. This means that a user or application does not notice that some piece of the system fails to work properly, and that the system subsequently (and automatically) recovers from that failure. Masking failures is one of the hardest issues in distributed systems and is even impossible when certain apparently realistic assumptions are made, as we will discuss in Chapter 8. The main difficulty in masking and transparently recovering from failures lies in the inability to distinguish between a dead process and a painfully slowly responding one. For example, when contacting a busy Web server, a browser will eventually time out and report that the Web page is unavailable. At that point, the user cannot tell whether the server is actually down or that the network is badly congested.

Degree of distribution transparency

Although distribution transparency is generally considered preferable for any distributed system, there are situations in which blindly attempting to hide all distribution aspects from users is not a good idea. A simple example is requesting your electronic newspaper to appear in your mailbox before 7 AM local time, as usual, while you are currently at the other end of the world living in a different time zone. Your morning paper will not be the morning paper you are used to.

Likewise, a wide-area distributed system that connects a process in San Francisco to a process in Amsterdam cannot be expected to hide the fact that Mother Nature will not allow it to send a message from one process to the other in less than approximately 35 milliseconds. Practice shows that it actually takes several hundred milliseconds using a computer network. Signal transmission is not only limited by the speed of light, but also by limited processing capacities and delays in the intermediate switches.

There is also a trade-off between a high degree of transparency and the performance of a system. For example, many Internet applications repeatedly try to contact a server before finally giving up. Consequently, attempting to mask a transient server failure before trying another one may slow down the system as a whole. In such a case, it may have been better to give up earlier, or at least let the user cancel the attempts to make contact.

Another example is where we need to guarantee that several replicas, located on different continents, must be consistent all the time. In other words, if one copy is changed, that change should be propagated to all copies before allowing any other operation. A single update operation may now even take seconds to complete, something that cannot be hidden from users.

Finally, there are situations in which it is not at all obvious that hiding distribution is a good idea. As distributed systems are expanding to devices that people carry around and where the very notion of location and context awareness is becoming increasingly important, it may be best to actually *expose* distribution rather than trying to hide it. An obvious example is making use of location-based services, which can often be found on mobile phones, such as finding a nearest shop or any nearby friends.

There are other arguments against distribution transparency. Recognizing that full distribution transparency is simply impossible, we should ask ourselves whether it is even wise to *pretend* that we can achieve it. It may be much better to make distribution explicit so that the user and application developer are never tricked into believing that there is such a thing as transparency. The result will be that users will much better understand the (sometimes unexpected) behavior of a distributed system, and are thus much better prepared to deal with this behavior.

The conclusion is that aiming for distribution transparency may be a nice goal when designing and implementing distributed systems, but that

it should be considered together with other issues such as performance and comprehensibility. The price for achieving full transparency may be surprisingly high.

Note 1.2 (Discussion: Against distribution transparency)

Several researchers have argued that hiding distribution will lead to only further complicating the development of distributed systems, exactly for the reason that full distribution transparency can never be achieved. A popular technique for achieving access transparency is to extend procedure calls to remote servers. However, Waldo et al. [1997] already pointed out that attempting to hide distribution by such remote procedure calls can lead to poorly understood semantics, for the simple reason that a procedure call *does* change when executed over a faulty communication link.

As an alternative, various researchers and practitioners are now arguing for less transparency, for example, by more explicitly using message-style communication, or more explicitly posting requests to, and getting results from remote machines, as is done on the Web when fetching pages. Such solutions will be discussed in detail in the next chapter.

A somewhat radical standpoint was taken by Wams [2012] by stating that partial failures preclude relying on the successful execution of a remote service. If such reliability cannot be guaranteed, it is then best to always perform only local executions, leading to the **copy-before-use** principle. According to this principle, data can be accessed only after they have been transferred to the machine of the process wanting that data. Moreover, modifying a data item should not be done. Instead, it can only be updated to a new version. It is not difficult to imagine that many other problems will surface. However, Wams shows that many existing applications can be retrofitted to this alternative approach without sacrificing functionality.

1.2.3 Openness

Another important goal of distributed systems is openness. An **open distributed system** is essentially a system that offers components that can easily be used by, or integrated into other systems. At the same time, an open distributed system itself will often consist of components that originate from elsewhere.

Interoperability, composability, and extensibility

To be open means that components should adhere to standard rules that describe the syntax and semantics of what those components have to offer (i.e., which service they provide). A general approach is to define services through **interfaces** using an **Interface Definition Language (IDL)**. Interface definitions written in an IDL nearly always capture only the syntax of services. In other words, they specify precisely the names of the functions that are available

together with types of the parameters, return values, possible exceptions that can be raised, and so on. The hard part is specifying precisely what those services do, that is, the semantics of interfaces. In practice, such specifications are given in an informal way by natural language.

If properly specified, an interface definition allows an arbitrary process that needs a certain interface, to talk to another process that provides that interface. It also allows two independent parties to build entirely different implementations of those interfaces, leading to two separate components that operate in exactly the same way.

Proper specifications are complete and neutral. Complete means that everything that is necessary to make an implementation has indeed been specified. However, many interface definitions are not at all complete, so that it is necessary for a developer to add implementation-specific details. Just as important is the fact that specifications do not prescribe what an implementation should look like; they should be neutral.

As pointed out in Blair and Stefani [1998], completeness and neutrality are important for interoperability and portability. **Interoperability** characterizes the extent by which two implementations of systems or components from different manufacturers can co-exist and work together by merely relying on each other's services as specified by a common standard. **Portability** characterizes to what extent an application developed for a distributed system A can be executed, without modification, on a different distributed system B that implements the same interfaces as A.

Another important goal for an open distributed system is that it should be easy to configure the system out of different components (possibly from different developers). Moreover, it should be easy to add new components or replace existing ones without affecting those components that stay in place. In other words, an open distributed system should also be **extensible**. For example, in an extensible system, it should be relatively easy to add parts that run on a different operating system, or even to replace an entire file system. Relatively simple examples of extensibility are plug-ins for Web browsers, but also those for Websites, such as the ones used for WordPress.

Note 1.3 (Discussion: Open systems in practice)

Of course, what we have just described is an ideal situation. Practice shows that many distributed systems are not as open as we would like, and that still a lot of effort is needed to put various bits and pieces together to make a distributed system. One way out of the lack of openness is to simply reveal all the gory details of a component and to provide developers with the actual source code. This approach is becoming increasingly popular, leading to so-called open-source projects, where large groups of people contribute to improving and debugging systems. Admittedly, this is as open as a system can get, but whether it is the best way is questionable.

Separating policy from mechanism

To achieve flexibility in open distributed systems, it is crucial that the system be organized as a collection of relatively small and easily replaceable or adaptable components. This implies that we should provide definitions of not only the highest-level interfaces, that is, those seen by users and applications, but also definitions for interfaces to internal parts of the system and describe how those parts interact. This approach is relatively new. Many older and even contemporary systems are constructed using a monolithic approach in which components are only logically separated but implemented as one, huge program. This approach makes it hard to replace or adapt a component without affecting the entire system. Monolithic systems thus tend to be closed instead of open.

The need for changing a distributed system is often caused by a component that does not provide the optimal policy for a specific user or application. As an example, consider caching in Web browsers. There are many different parameters that need to be considered:

Storage: Where is data to be cached? Typically, there will be an in-memory cache next to storage on disk. In the latter case, the exact position in the local file system needs to be considered.

Exemption: When the cache fills up, which data is to be removed so that newly fetched pages can be stored?

Sharing: Does each browser make use of a private cache, or is a cache to be shared among browsers of different users?

Refreshing: When does a browser check if cached data is still up-to-date? Caches are most effective when a browser can return pages without having to contact the original Website. However, this bears the risk of returning stale data. Note also that refresh rates are highly dependent on which data is actually cached: whereas timetables for trains hardly change, this is not the case for Web pages showing current highway-traffic conditions, or worse yet, stock prices.

What we need is a separation between **policy** and **mechanism**. In the case of Web caching, for example, a browser should ideally provide facilities for only storing documents (i.e., a mechanism) and at the same time allow users to decide which documents are stored and for how long (i.e., a policy). In practice, this can be implemented by offering a rich set of parameters that the user can set (dynamically). When taking this a step further, a browser may even offer facilities for plugging in policies that a user has implemented as a separate component.

Note 1.4 (Discussion: Is a strict separation really what we need?)

In theory, strictly separating policies from mechanisms seems to be the way to go. However, there is an important trade-off to consider: the stricter the separation, the more we need to make sure that we offer the appropriate collection of mechanisms. In practice, this means that a rich set of features is offered, in turn leading to many configuration parameters. As an example, the popular Firefox browser comes with a few hundred configuration parameters. Just imagine how the configuration space explodes when considering large distributed systems consisting of many components. In other words, strict separation of policies and mechanisms may lead to highly complex configuration problems.

One option to alleviate these problems is to provide reasonable defaults, and this is what often happens in practice. An alternative approach is one in which the system observes its own usage and dynamically changes parameter settings. This leads to what are known as **self-configurable systems**. Nevertheless, the fact alone that many mechanisms need to be offered to support a wide range of policies often makes coding distributed systems very complicated. Hard-coding policies into a distributed system may reduce complexity considerably, but at the price of less flexibility.

1.2.4 Dependability

As its name suggests, **dependability** refers to the degree that a computer system can be relied upon to operate as expected. In contrast to single-computer systems, dependability in distributed systems can be rather intricate due to **partial failures**: somewhere there is a component failing while the system as a whole still seems to be living up to expectations (up to a certain point or moment). Although single-computer systems can also suffer from failures that do not appear immediately, having a potentially large collection of networked computer systems complicates matters considerably. In fact, one should assume that at any time, there are always partial failures occurring. An important goal of distributed systems is to mask those failures, as well as mask the recovery from those failures. This masking is the essence of being able to tolerate faults, accordingly referred to as **fault tolerance**.

Basic concepts

Dependability is a term that covers several useful requirements for distributed systems, including the following [Kopetz and Verissimo, 1993]:

- Availability
- Reliability
- Safety
- Maintainability

Availability is defined as the property that a system is ready to be used immediately. In general, it refers to the probability that the system is operating correctly at any given moment and is available to perform its functions on behalf of its users. In other words, a highly available system is one that will most likely be working at a given instant in time.

Reliability refers to the property that a system can run continuously without failure. In contrast to availability, reliability is defined in terms of a time interval instead of an instant in time. A highly reliable system is one that will most likely continue to work without interruption during a relatively long period of time. This is a subtle but important difference when compared to availability. If a system goes down on average for one, seemingly random millisecond every hour, it has an availability of more than 99.9999 percent, but is still unreliable. Similarly, a system that never crashes but is shut down for two specific weeks every August has high reliability but only 96 percent availability. The two are not the same.

Safety refers to the situation that when a system temporarily fails to operate correctly, no catastrophic event happens. For example, many process-control systems, such as those used for controlling nuclear power plants or sending people into space, are required to provide a high degree of safety. If such control systems temporarily fail for only a very brief moment, the effects could be disastrous. Many examples from the past (and probably many more yet to come) show how hard it is to build safe systems.

Finally, **maintainability** refers to how easily a failed system can be repaired. A highly maintainable system may also show a high degree of availability, especially if failures can be detected and repaired automatically. However, as we shall see, automatically recovering from failures is easier said than done.

Traditionally, fault-tolerance has been related to the following three metrics:

- **Mean Time To Failure (MTTF)**: The average time until a component fails.
- **Mean Time To Repair (MTTR)**: The average time needed to repair a component.
- **Mean Time Between Failures (MTBF)**: Simply $MTTF + MTTR$.

Note that these metrics make sense only if we have an accurate notion of what a failure actually is. As we will encounter in Chapter 8, identifying the occurrence of a failure may actually not be so obvious.

Faults, errors, failures

A system is said to **fail** when it cannot meet its promises. In particular, if a distributed system is designed to provide its users with several services, the system has failed when one or more of those services cannot be (completely) provided. An **error** is a part of a system's state that may lead to a failure. For

example, when transmitting packets across a network, it is to be expected that some packets have been damaged when they arrive at the receiver. Damaged in this context means that the receiver may incorrectly sense a bit value (e.g., reading a 1 instead of a 0), or may even be unable to detect that something has arrived.

The cause of an error is called a **fault**. Clearly, finding out what caused an error is important. For example, a wrong or bad transmission medium may easily cause packets to be damaged. In this case, it is relatively easy to remove the fault. However, transmission errors may also be caused by bad weather conditions, such as in wireless networks. Changing the weather to reduce or prevent errors is a bit trickier.

As another example, a crashed program is clearly a failure, which may have happened because the program entered a branch of code containing a programming bug (i.e., a programming error). The cause of that bug is typically a programmer. In other words, the programmer is the cause of the error (programming bug), in turn leading to a failure (a crashed program).

Building dependable systems closely relates to controlling faults. As explained by Avizienis et al. [2004], a distinction can be made between preventing, tolerating, removing, and forecasting faults. For our purposes, the most important issue is **fault tolerance**, meaning that a system can provide its services even in the presence of faults. For example, by applying error-correcting codes for transmitting packets, it is possible to tolerate, to a certain extent, relatively poor transmission lines and reducing the probability that an error (a damaged packet) may lead to a failure.

Faults are generally classified as transient, intermittent, or permanent. **Transient faults** occur once and then disappear. If the operation is repeated, the fault goes away. A bird flying through the beam of a microwave transmitter may cause lost bits on some network (not to mention a roasted bird). If the transmission times out and is retried, it will probably work the second time.

An **intermittent fault** occurs, then vanishes on its own accord, then reappears, and so on. A loose contact on a connector will often cause an intermittent fault. Intermittent faults cause a great deal of aggravation because they are difficult to diagnose. Typically, when the fault doctor shows up, the system works fine.

A **permanent fault** is one that continues to exist until the faulty component is replaced. Burnt-out chips, software bugs, and disk-head crashes are examples of permanent faults.

Dependable systems are also required to provide security, especially in terms of confidentiality and integrity. **Confidentiality** is the property that information is disclosed only to authorized parties, while **integrity** relates to ensuring that alterations to various assets can be made only in an authorized way. Indeed, can we speak of a dependable system when confidentiality and integrity are not in place? We return to security next.

1.2.5 Security

A distributed system that is not secure, is not dependable. As mentioned, special attention is needed to ensure confidentiality and integrity, both of which are directly coupled to authorized disclosure and access of information and resources. In any computer system, **authorization** is done by checking whether an identified entity has proper access rights. In turn, this means that the system should know it is indeed dealing with the proper entity. For this reason, **authentication** is essential: verifying the correctness of a claimed identity. Equally important is the notion of **trust**. If a system can positively authenticate a person, what is that authentication worth if the person cannot be trusted? For this reason alone, proper authorization is important, as it may be used to limit any damage that a person, who could in hindsight not be trusted, can cause. For example, in financial systems, authorization may limit the amount of money a person is allowed to transfer between various accounts. We will discuss trust, authentication, and authorization at length in Chapter 9.

Key elements needed to understand security

An essential technique to making distributed systems secure is cryptography. This is not the place in this book to extensively discuss cryptography (which we also defer until Chapter 9), yet to understand how security fits into various perspectives in the following chapters, we informally introduce some of its basic elements.

Keeping matters simple, security in distributed systems is all about **encrypting** and **decrypting** data using **security keys**. The easiest way of considering a security key K is to see it as a function operating on some data $data$. We use the notation $K(data)$ to express the fact that the key K operates on data.

There are two ways of encrypting and decrypting data. In a **symmetric cryptosystem**, encryption and decryption takes place with a single key. Denoting by $E_K(data)$ the encryption of data using key E_K , and likewise $D_K(data)$ for decryption with key D_K , then in a symmetric cryptosystem, the same key is used for encryption and decryption, i.e.,

$$\text{if } data = D_K(E_K(data)) \text{ then } D_K = E_K.$$

Note that in a symmetric cryptosystem, the key will need to be kept secret by all parties that are authorized to encrypt or decrypt data. In an **asymmetric cryptosystem**, the keys used for encryption and decryption are different. In particular, there is a **public key** PK that can be used by anyone, and a **secret key** SK that is, as its name suggests, to be kept secret. Asymmetric cryptosystems are also called **public-key systems**.

Encryption and decryption in public-key systems can be used in two, fundamentally different ways. First, if Alice wants to encrypt data that can be decrypted only by Bob, she should use Bob's public key, PK_B , leading to the encrypted data $PK_B(\text{data})$. Only the holder of the associated secret key can decrypt this information, i.e., Bob, who will apply the operation $SK_B(PK_B(\text{data}))$, which returns data.

A second, and widely applied use case, is that of realizing **digital signatures**. Suppose Alice makes some data available for which it is important that any party, but let us assume it is Bob, needs to know for sure that it comes from Alice. In that case, Alice can encrypt the data with her secret key SK_A , leading to $SK_A(\text{data})$. If it can be assured that the associated public key PK_A indeed belongs to Alice, then successfully decrypting $SK_A(\text{data})$ to data, is proof that Alice knows about data: she is the only one holding the secret key SK_A . Of course, we need to make the assumption that Alice is indeed the only one who holds SK_A . We return to some of these assumptions in Chapter 9.

As it turns out, proving that an entity has seen, or knows about some data, returns frequently in secured distributed systems. Practical placement of digital signatures is generally more efficient by a **hash function**. A hash function H has the property that when operating on some data, i.e., $H(\text{data})$, it returns a fixed-length string, regardless of the length of data. Any change of data to data^* will lead to a different hash value $H(\text{data}^*)$. Moreover, given a hash value h , it is computationally impossible in practice, to discover the original data. What this all means, is that for placing a digital signature, Alice computes $\text{sig} = SK_A(H(\text{data}))$ as her signature, and tells Bob about data, H and sig. Bob, in turn, can then verify that signature by computing $PK_A(\text{sig})$ and verifying that it matches the value $H(\text{data})$.

Using cryptography in distributed systems

The application of cryptography in distributed systems comes in many forms. Besides its general use for encryption and digital signatures, cryptography forms the basis for realizing a **secure channel** between two communicating parties. Such channels basically let two parties know for sure that they are indeed communicating to the entities that they expected to communicate with. In other words, a communication channel that supports mutual authentication. A practical example of a secure channel is using https when accessing Websites. Now, many browsers demand that Websites support this protocol, and at the very least will warn the user when this is not the case. In general, using cryptography is necessary to realize authentication (and authorization) in distributed systems.

Cryptography is also used to realize secure distributed data structures. A well-known example is that of a **blockchain**, which is, literally, a chain of blocks. The basic idea is simple: hash the data in a block B_i , and place that hash value as part of the data in its succeeding block B_{i+1} . Any change in

B_i (for example, as the result of an attack), will require that the attacker also changes the stored hash value in B_{i+1} . However, because the successor of B_{i+1} contains the hash computed over the data in B_{i+1} , and thus including the *original* hash value of B_i , the attacker will also have to change the new hash value of B_i as stored in B_{i+1} . Yet changing that value, also means changing the hash value of B_{i+1} , and thus the value stored in B_{i+2} , in turn, requiring that a new hash value is to be computed for B_{i+2} , and so on. In other words, by securely linking blocks into a chain, any successful change to a block requires that *all* successive blocks be modified as well. These modifications should go unnoticed, which is virtually impossible.

Cryptography is also used for another important mechanism in distributed systems: delegating access rights. The basic idea is that Alice may want to delegate some rights to Bob, who, in turn, may want to pass some of those rights on to Chuck. Using appropriate means (which we discuss in Chapter 9, a service can securely check that Chuck has indeed been authorized to perform certain operations, without the need for that service to check with Alice whether the delegation is in place. Note that delegation is something we are now used to: many of us delegate access rights that we have as a user to specific applications, such as an e-mail client.

An upcoming distributed application of cryptography is so-called **multi-party computation**: the means for two or three parties to compute a value for which the data of those parties is needed, but without having to actually share that data. An often-used example is computing the number of votes without having to know who voted for whom.

We will see many more examples of security in distributed systems in the following chapters. With the brief explanations of the cryptographic basis, it should suffice to see how security is applied. We shall consistently use the notations as shown in Figure 1.4. Alternatively, security examples can be skipped until having studied Chapter 9.

Notation	Description
$K_{A,B}$	Secret key shared by A and B
PK_A	Public key of A
SK_A	Private (secret) key of A
$E_K(\text{data})$	Encryption of data using key E_K (or key K)
$D_K(\text{data})$	Decryption of (encrypted) data using key D_K (or key K)
$H(\text{data})$	The hash of data computed using function H

Figure 1.4: Notations for cryptosystems used in this book.

1.2.6 Scalability

For many of us, worldwide connectivity through the Internet is as common as being able to send a package to anyone anywhere around the world. Moreover, where until recently, we were used to having relatively powerful desktop computers for office applications and storage, we are now witnessing that such applications and services are being placed in what has been coined “the cloud,” in turn leading to an increase of much smaller networked devices such as tablet computers or even cloud-only laptops such as Google’s Chromebook. With this in mind, scalability has become one of the most important design goals for developers of distributed systems.

Scalability dimensions

Scalability of a system can be measured along at least three different dimensions (see [Neuman, 1994]):

Size scalability: A system can be scalable regarding its size, meaning that we can easily add more users and resources to the system without any noticeable loss of performance.

Geographical scalability: A geographically scalable system is one in which the users and resources may lie far apart, but the fact that communication delays may be significant is hardly noticed.

Administrative scalability: An administratively scalable system is one that can still be easily managed even if it spans many independent administrative organizations.

Let us take a closer look at each of these three scalability dimensions.

Size scalability When a system needs to scale, very different types of problems need to be solved. Let us first consider scaling regarding size. If more users or resources need to be supported, we are often confronted with the limitations of centralized services, although often for very different reasons. For example, many services are centralized in the sense that they are implemented by a single **server** running on a specific machine in the distributed system. In a more modern setting, we may have a group of collaborating servers co-located on a cluster of tightly coupled machines physically placed at the same location. The problem with this scheme is obvious: the server, or group of servers, can simply become a bottleneck when it needs to process an increasing number of requests. To illustrate how this can happen, let us assume that a service is implemented on a single machine. In that case, there are essentially three root causes for becoming a bottleneck:

- The computational capacity, limited by the CPUs
- The storage capacity, including the I/O transfer rate

- The network between the user and the centralized service

Let us first consider the computational capacity. Just imagine a service for computing optimal routes taking real-time traffic information into account. It is not difficult to imagine that this may be primarily a compute-bound service, requiring several (tens of) seconds to complete a request. If there is only a single machine available, then even a modern high-end system will eventually run into problems if the number of requests increases beyond a certain point.

Likewise, but for different reasons, we will run into problems when having a service that is mainly I/O bound. A typical example is a poorly designed centralized search engine. The problem with content-based search queries is that we essentially need to match a query against an entire data set. Even with advanced indexing techniques, we may still face the problem of having to process a huge amount of data exceeding the main-memory capacity of the machine running the service. As a consequence, much of the processing time will be determined by the relatively slow disk accesses and transfer of data between disk and main memory. Simply adding more or higher-speed disks will prove not to be a sustainable solution as the number of requests continues to increase.

Finally, the network between the user and the service may also be the cause of poor scalability. Just imagine a video-on-demand service that needs to stream high-quality video to multiple users. A video stream can easily require a bandwidth of 8 to 10 Mbps, meaning that if a service sets up point-to-point connections with its customers, it may soon hit the limits of the network capacity of its own outgoing transmission lines.

There are several solutions to attack size scalability, which we discuss below after having looked into geographical and administrative scalability.

Note 1.5 (Advanced: Analyzing size scalability)

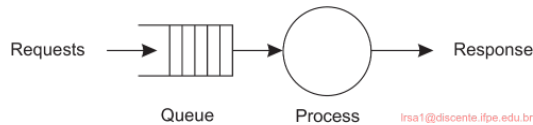


Figure 1.5: A simple model of a service as a queuing system.

Size scalability problems for centralized services can be formally analyzed using queuing theory and making a few simplifying assumptions. At a conceptual level, a centralized service can be modeled as the simple queuing system shown in Figure 1.5: requests are submitted to the service, where they are queued until further notice. As soon as the process can handle a next request, it fetches it from the queue, does its work, and produces a response. We largely follow Menasce and Almeida [2002] in explaining the performance of a centralized service.

Often, we may assume that the queue has an infinite capacity, meaning that there is no restriction on the number of requests that can be accepted for further processing. Strictly speaking, this means that the arrival rate of requests is not influenced by what is currently in the queue or being processed. Assuming that the arrival rate of requests is λ requests per second, and that the processing capacity of the service is μ requests per second, one can compute that the fraction of time p_k that there are k requests in the system is equal to:

$$p_k = \left(1 - \frac{\lambda}{\mu}\right) \left(\frac{\lambda}{\mu}\right)^k$$

If we define the **utilization** U of a service as the fraction of time that it is busy, then clearly,

$$U = \sum_{k>0} p_k = 1 - p_0 = \frac{\lambda}{\mu} \Rightarrow p_k = (1 - U)U^k$$

We can then compute the average number $\bar{N}$ of requests in the system as

$$\bar{N} = \sum_{k \geq 0} k \cdot p_k = \sum_{k \geq 0} k \cdot (1 - U)U^k = (1 - U) \sum_{k \geq 0} k \cdot U^k = \frac{(1 - U)U}{(1 - U)^2} = \frac{U}{1 - U}.$$

What we are truly interested in, is the response time R : how long does it take before the service to process a request, including the time spent in the queue. To that end, we need the average throughput X . Considering that the service is “busy” when at least one request is being processed, and that this then happens with a throughput of μ requests per second, and during a fraction U of the total time, we have:

$$X = \underbrace{U \cdot \mu}_{\text{server at work}} + \underbrace{(1 - U) \cdot 0}_{\text{server idle}} = \frac{\lambda}{\mu} \cdot \mu = \lambda$$

Using Little’s formula [Trivedi, 2002], we can then derive the response time as

$$R = \frac{\bar{N}}{X} = \frac{S}{1 - U} \Rightarrow \frac{R}{S} = \frac{1}{1 - U}$$

where $S = \frac{1}{\mu}$, the actual service time. Note that if U is small, the response-to-service time ratio is close to 1, meaning that a request is virtually instantly processed, and at the maximum speed possible. However, as soon as the utilization comes closer to 1, we see that the response-to-server time ratio quickly increases to very high values, effectively meaning that the system is coming close to a grinding halt. This is where we see scalability problems emerge. From this simple model, we can see that the only solution is bringing down the service time S . We leave it as an exercise to the reader to explore how S may be decreased.

Geographical scalability Geographical scalability has its own problems. One of the main reasons why it is still difficult to scale existing distributed systems

that were designed for local-area networks is that many of them are based on **synchronous communication**. In this form of communication, a party requesting a service, generally referred to as a **client**, blocks until a reply is sent back from the **server** implementing the service. More specifically, we often see a communication pattern consisting of many client-server interactions, as may be the case with database transactions. This approach generally works fine in LANs, where communication between two machines is often at worst a few hundred microseconds. However, in a wide-area system, we need to consider that interprocess communication may be hundreds of milliseconds, three orders of magnitude slower. Building applications using synchronous communication in wide-area systems requires a great deal of care (and not just a little patience), notably with a rich interaction pattern between client and server.

Another problem that hinders geographical scalability is that communication in wide-area networks is inherently much less reliable than in local-area networks. In addition, we generally also need to deal with limited bandwidth. The effect is that solutions developed for local-area networks cannot always be easily ported to a wide-area system. A typical example is streaming video. In a home network, even when having only wireless links, ensuring a stable, fast stream of high-quality video frames from a media server to a display is quite simple. Simply placing that same server far away and using a standard TCP connection to the display will surely fail: bandwidth limitations will instantly surface, but also maintaining the same level of reliability can easily cause headaches.

Yet another issue that pops up when components lie far apart is the fact that wide-area systems generally have only very limited facilities for multipoint communication. In contrast, local-area networks often support efficient broadcasting mechanisms. Such mechanisms have proven to be extremely useful for discovering components and services, which is essential from a management perspective. In wide-area systems, we need to develop separate services, such as naming and directory services, to which queries can be sent. These support services, in turn, need to be scalable as well and often no obvious solutions exist as we will encounter in later chapters.

Administrative scalability Finally, a difficult, and often open, question is how to scale a distributed system across multiple, independent administrative domains. A major problem that needs to be solved is that of conflicting policies regarding resource usage (and payment), management, and security. To illustrate, for many years, scientists have been looking for solutions to share their (often expensive) equipment in what is known as a **computational grid**. In these grids, a global decentralized system is constructed as a federation of local distributed systems, allowing a program running on a computer at an organization A to directly access resources at the organization B.

Many components of a distributed system that reside within a single domain can often be trusted by users that operate within that same domain. In such cases, system administration may have tested and certified applications, and may have taken special measures to ensure that such components cannot be tampered with. In essence, the users trust their system administrators. However, this trust does not expand naturally across domain boundaries.

If a distributed system expands to another domain, two types of security measures need to be taken. First, the distributed system has to protect itself against malicious attacks from the new domain. For example, users from the new domain may have only read access to the file system in its original domain. Likewise, facilities such as expensive image setters or high-performance computers may not be made available to unauthorized users. Second, the new domain has to protect itself against malicious attacks from the distributed system. A typical example is that of downloading programs, such as in the case of federated learning. Basically, the new domain does not know what to expect from such foreign code. The problem, as we shall see in Chapter 9, is how to enforce those limitations.

As a counterexample of distributed systems spanning multiple administrative domains that apparently *do not* suffer from administrative scalability problems, consider modern file-sharing peer-to-peer networks. In these cases, end users simply install a program implementing distributed search and download functions and within minutes can start downloading files. Other examples include peer-to-peer applications for telephony over the Internet such as older Skype systems [Baset and Schulzrinne, 2006], and (again older) peer-assisted audio-streaming applications such as Spotify [Kreitz and Niemelä, 2010]. A more modern application (that has yet to prove itself in terms of scalability) are blockchains. What these decentralized systems have in common is that *end users*, and not administrative entities, collaborate to keep the system up and running. At best, underlying administrative organizations such as **Internet Service Providers (ISPs)** can police the network traffic that these peer-to-peer systems cause.

Scaling techniques

Having discussed some scalability problems brings us to the question of how those problems can generally be solved. In most cases, scalability problems in distributed systems appear as performance problems caused by limited capacity of servers and network. Simply improving their capacity (e.g., by increasing memory, upgrading CPUs, or replacing network modules) is often a solution, referred to as **scaling up**. When it comes to **scaling out**, that is, expanding the distributed system by essentially deploying more machines, there are basically only three techniques we can apply: hiding communication latencies, distribution of work, and replication (see also Neuman [1994]).

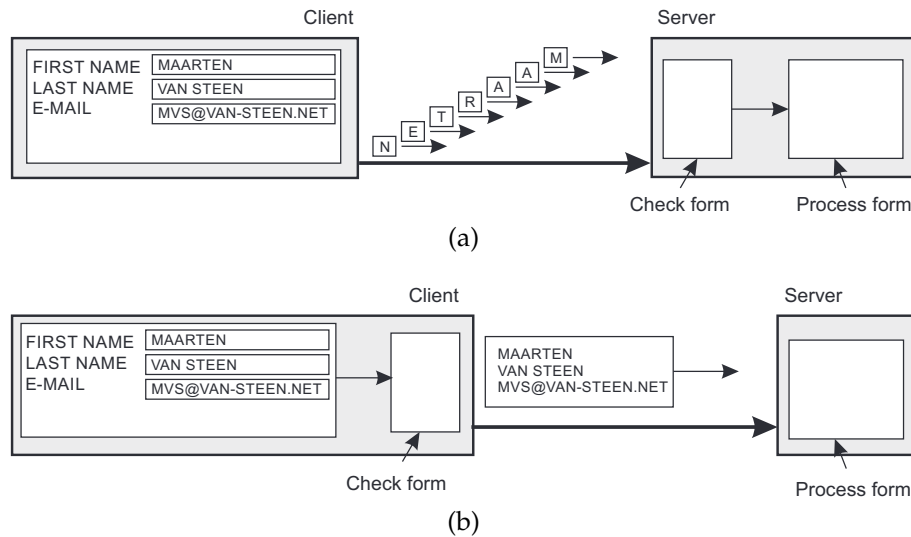


Figure 1.6: The difference between letting (a) a server or (b) a client check forms as they are being filled.

Hiding communication latencies Hiding communication latencies is applicable in the case of geographical scalability. The basic idea is simple: try to avoid waiting for responses to remote-service requests as much as possible. For example, when a service has been requested at a remote machine, an alternative to waiting for a reply from the server is to do other useful work at the requester's side. Essentially, this means constructing the requesting application in such a way that it uses only **asynchronous communication**. When a reply comes in, the application is interrupted and a special handler is called to complete the previously issued request. Asynchronous communication can often be used in batch-processing systems and parallel applications, in which independent tasks can be scheduled for execution while another task is waiting for communication to complete. Alternatively, a new thread of control can be started to perform the request. Although it blocks waiting for the reply, other threads in the process can continue.

However, there are many applications that cannot make effective use of asynchronous communication. For example, in interactive applications when a user sends a request, she will generally have nothing better to do than to wait for the answer. In such cases, a much better solution is to reduce the overall communication, for example, by moving part of the computation that is normally done at the server to the client process requesting the service. A typical case where this approach works is accessing databases using forms. Filling in forms can be done by sending a separate message for each field and waiting for an acknowledgment from the server, as shown in Figure 1.6(a). For example, the server may check for syntactic errors before accepting an entry.

A much better solution is to ship the code for filling in the form, and possibly checking the entries, to the client, and have the client return a completed form, as shown in Figure 1.6(b). This approach of shipping code is widely supported by the Web through JavaScript.

Partitioning and distribution Another important scaling technique is partitioning and distribution, which involves taking a component or other resource, splitting it into smaller parts, and subsequently spreading those parts across the system. A good example of partitioning and distribution is the Internet Domain Name System (DNS). The DNS name space is hierarchically organized into a tree of **domains**, which are divided into nonoverlapping **zones**, as shown for the original DNS in Figure 1.7. The names in each zone are handled by a single name server. Without going into too many details now (we return to DNS extensively in Chapter 6), one can think of each path name being the name of a host on the Internet, and is thus associated with a network address of that host. Basically, resolving a name means returning the network address of the associated host. Consider, for example, the name `flits.cs.vu.nl`. To resolve this name, it is first passed to the server of zone Z1 (see Figure 1.7) which returns the address of the server for zone Z2, to which the rest of the name, `flits.cs.vu`, can be handed. The server for Z2 will return the address of the server for zone Z3, which is capable of handling the last part of the name, and will return the address of the associated host.

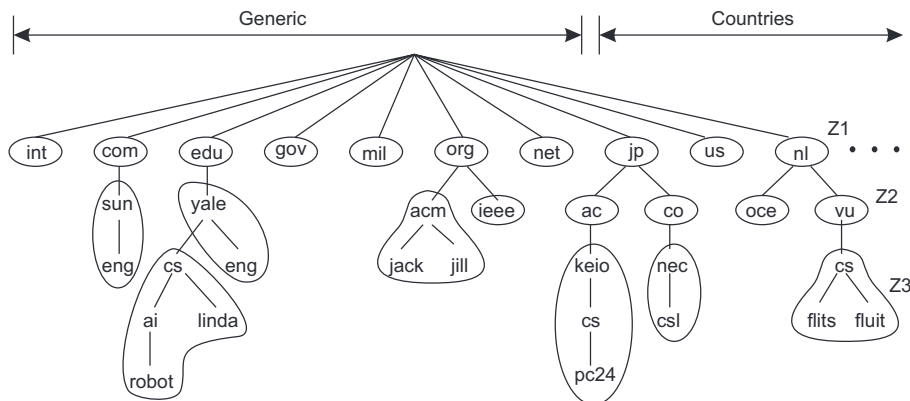


Figure 1.7: An example of dividing the (original) DNS name space into zones.

These examples illustrate how the **naming service** as provided by DNS, is distributed across several machines, thus avoiding that a single server has to deal with all requests for name resolution.

As another example, consider the World Wide Web. To most users, the Web appears to be an enormous document-based information system, in which each document has its own unique name in the form of a URL. Conceptually,

it may even appear as if there is only a single server. However, the Web is physically partitioned and distributed across *a few hundreds of millions* of servers, each handling often a number of Websites, or parts of Websites. The name of the server handling a document is encoded into that document's URL. It is only because of this distribution of documents that the Web has been capable of scaling to its current size. Yet, note that finding out how many servers provide Web-based services is virtually impossible: A Website today is so much more than a few static Web documents.

Replication Considering that scalability problems often appear in the form of performance degradation, it is generally a good idea to actually **replicate** components or resources, etc., across a distributed system. Replication not only increases availability, but also helps to balance the load between components, leading to better performance. Moreover, in geographically widely dispersed systems, having a copy nearby can hide much of the communication latency problems mentioned before.

Caching is a special form of replication, although the distinction between the two is often hard to make or even artificial. As in the case of replication, caching results in making a copy of a resource, generally in the proximity of the client accessing that resource. However, in contrast to replication, caching is a decision made by the client of a resource and not by the owner of a resource.

There is one serious drawback to caching and replication that may adversely affect scalability. Because we now have multiple copies of a resource, modifying one copy makes that copy different from the others. Consequently, caching and replication leads to **consistency** problems.

To what extent inconsistencies can be tolerated depends on the usage of a resource. For example, many Web users find it acceptable that their browser returns a cached document of which the validity has not been checked for the last few minutes. However, there are also many cases in which strong consistency guarantees need to be met, such as in the case of electronic stock exchanges and auctions. The problem with strong consistency is that an update must be immediately propagated to all other copies. Moreover, if two updates happen concurrently, it is often also required that updates are processed in the same order everywhere, introducing a global ordering problem. To make things worse, combining consistency with desirable properties such as availability may simply be impossible, as we discuss in Chapter 8.

Replication therefore often requires some global synchronization mechanism. Unfortunately, such mechanisms are extremely hard or even impossible to implement in a scalable way, if alone because network latencies have a natural lower bound. Consequently, scaling by replication may introduce other, inherently nonscalable solutions. We return to replication and consistency extensively in Chapter 7.

Discussion When considering these scaling techniques, one could argue that size scalability is the least problematic from a technical perspective. Often, increasing the capacity of a machine will save the day, although perhaps there is a high monetary cost to pay. Geographical scalability is a much tougher problem, as network latencies are naturally bound from below. As a consequence, we may be forced to copy data to locations close to where clients are, leading to problems of maintaining copies consistent. Practice shows that combining distribution, replication, and caching techniques with different forms of consistency generally leads to acceptable solutions. Finally, administrative scalability seems to be the most difficult problem to solve, partly because we need to deal with nontechnical issues, such as politics of organizations and human collaboration. The introduction and now widespread use of peer-to-peer technology has successfully demonstrated what can be achieved if end users are put in control [Lua et al., 2005; Oram, 2001]. However, peer-to-peer networks are obviously not the universal solution to all administrative scalability problems.

1.3 A simple classification of distributed systems

We have discussed distributed versus decentralized systems, yet it is also useful to classify distributed systems according to what they are being developed and used for. We make a distinction between systems that are developed for (high performance) computing, for general information processing, and those that are developed for pervasive computing, i.e., for the “Internet of Things.” As with many classifications, the boundaries between these three types are not strict and combinations can easily be thought of.

1.3.1 High-performance distributed computing

An important class of distributed systems is the one used for high-performance computing tasks. Roughly speaking, one can make a distinction between two subgroups. In **cluster computing** the underlying hardware consists of a collection of similar compute nodes, interconnected by a high-speed network, often alongside a more common local-area network for controlling the nodes. In addition, each node generally runs the same operating system.

The situation becomes very different in the case of **grid computing**. This subgroup consists as decentralized systems that are often constructed as a federation of computer systems, where each system may fall under a different administrative domain, and may be very different when it comes to hardware, software, and deployed network technology.

Note 1.6 (More information: Parallel processing)

High-performance computing more or less started with the introduction of **multi-processor machines**. In this case, multiple CPUs are organized in such a way that they all have access to the same physical memory, as shown in Figure 1.8(a). In contrast, in a **multicomputer system** several computers are connected through a network and there is no sharing of main memory, as shown in Figure 1.8(b). The shared-memory model turned out to be highly convenient for improving the performance of programs, and it was relatively easy to program.

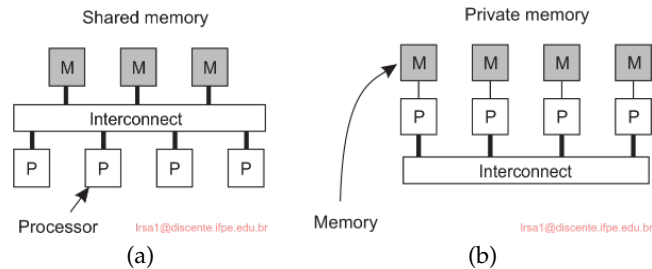


Figure 1.8: A comparison between (a) multiprocessor and (b) multicomputer architectures.

Its essence is that multiple threads of control are executing at the same time, while all threads have access to shared data. Access to that data is controlled through well-understood synchronization mechanisms like semaphores (see Ben-Ari [2006] or Herlihy et al. [2021] for more information on developing parallel programs). Unfortunately, the model does not easily scale: so far, machines have been developed in which only a few tens (and sometimes hundreds) of CPUs have efficient access to shared memory. To a certain extent, we are seeing the same limitations for multicore processors.

To overcome the limitations of shared-memory systems, high-performance computing moved to distributed-memory systems. This shift also meant that many programs had to make use of message passing instead of modifying shared data as a means of communication and synchronization between threads. Unfortunately, message-passing models have proven to be much more difficult and error-prone compared to the shared-memory programming models. For this reason, there has been significant research in attempting to build so-called **distributed shared-memory multicomputers**, or simply **DSM systems** [Amza et al., 1996].

In essence, a DSM system allows a processor to address a memory location at another computer as if it were local memory. This can be achieved using existing techniques available to the operating system, for example, by mapping all main-memory pages of the various processors into a single virtual address space. Whenever a processor A addresses a page located at another processor B, a page fault occurs at A allowing the operating system at A to fetch the content of the referenced page at B in the same way that it would normally fetch it locally from disk. At the same time, the processor B would be informed that the page is currently not accessible.

Mimicking shared-memory systems using multicomputers eventually had to be abandoned because performance could never meet the expectations of programmers, who would rather resort to far more intricate, yet better (predictably) performing message-passing models. An important side effect of exploring the hardware-software boundaries of parallel processing is a thorough understanding of consistency models, to which we return extensively in Chapter 7.

Cluster computing

Cluster computing systems became popular when the price/performance ratio of personal computers and workstations improved. At a certain point, it became financially and technically attractive to build a supercomputer using off-the-shelf technology by simply hooking up a collection of relatively simple computers in a high-speed network. In virtually all cases, cluster computing is used for parallel programming, in which a single (compute intensive) program is run in parallel on multiple machines. The principle of this organization is shown in Figure 1.9.

This type of high-performance computing has evolved considerably. As discussed extensively by Gerofi et al. [2019], the developments of supercomputers organized as clusters have reached a point where we see clusters with more than 100,000 CPUs, with each CPU having 8 or 16 cores. There are multiple networks. Most important is a network formed by dedicated high-speed interconnects between the various nodes (in other words, there is often no such thing as a shared high-speed network for computations). A separate management network, as well as nodes, are used to monitor and control

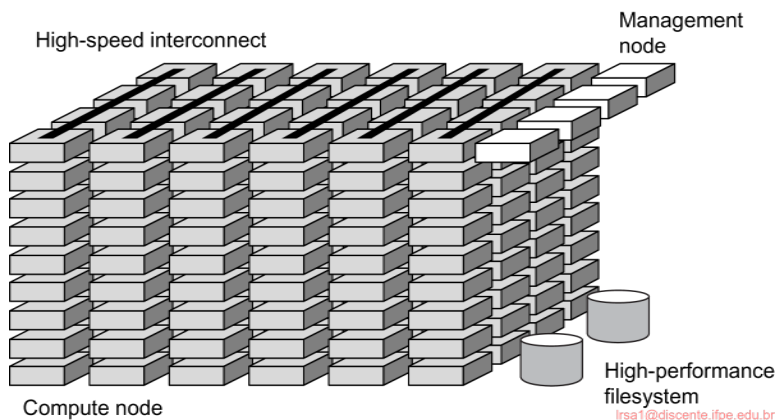


Figure 1.9: An example of a cluster computing system (adapted from [Gerofi et al., 2019].)

the organization and performance of the system as a whole. In addition, a special high-performance file system or database is used, again with its own local, dedicated network. Figure 1.9 does not show additional equipment, notably high-speed I/O as well as networking facilities for remote access and communication.

A management node is generally responsible for collecting jobs from users, to subsequently distribute the associated tasks among the various compute nodes. In practice, several management nodes are used when dealing with very large clusters. As such, a management node actually runs the software needed for the execution of programs and management of the cluster, while the compute nodes are equipped with a standard operating system extended with typical functions for communication, storage, fault tolerance, and so on.

An interesting development, as explained in Gerofi et al. [2019], is the role of the operating system. There has been a clear trend to minimize the operating system to lightweight kernels, essentially ensuring the least possible overhead. A drawback is that such operating systems become highly specialized and fine-tuned toward the underlying hardware. This specialization affects compatibility, or openness. To compensate, we are now gradually seeing so-called multikernel approaches, in which a full-fledged operating system operates next to a lightweight kernel, thus achieving the best of two worlds. This combination is also necessary given increasingly more often, a high-performance compute node is required to run multiple, independent jobs simultaneously. At present, 95% of all high-performance computers run Linux-based systems; multikernel approaches are developed for multicore CPUs, with most cores running a lightweight kernel and the other running a regular Linux system. In this way, new developments such as containers (which we discuss in Chapter 3) can also be supported. The effects for computing performance still needs to be seen.

Grid computing

A characteristic feature of traditional cluster computing is its homogeneity. In most cases, the computers in a cluster are largely the same, have the same operating system, and are all connected through the same network. However, as we just discussed, there is a continuous trend toward more hybrid architectures in which nodes are specifically configured for certain tasks. This diversity is even more prevalent in **grid-computing systems**: no assumptions are made concerning similarity of hardware, operating systems, networks, administrative domains, security policies, etc. [Rajaraman, 2016].

A key issue in a grid-computing system is that resources from different organizations are brought together to allow the collaboration of a group of people from different institutions, indeed forming a federation of systems. Such a collaboration is realized in the form of a **virtual organization**. The processes belonging to the same virtual organization have access rights to the

resources that are provided to that organization. Typically, resources consist of compute servers (including supercomputers, possibly implemented as cluster computers), storage facilities, and databases. In addition, special networked devices such as telescopes, sensors, etc., can be provided as well.

Given its nature, much of the software for realizing grid computing evolves around providing access to resources from different administrative domains, and to only those users and applications that belong to a specific virtual organization. For this reason, focus is often on architectural issues. An architecture initially proposed by Foster et al. [2001] is shown in Figure 1.10, which still forms the basis for many grid computing systems.

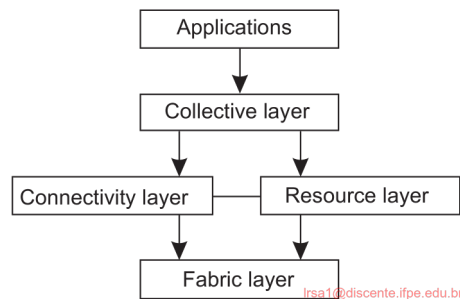


Figure 1.10: A layered architecture for grid computing systems.

The architecture consists of four layers. The lowest **fabric layer** provides interfaces to local resources at a specific site. Note that these interfaces are tailored to allow sharing of resources within a virtual organization. Typically, they will provide functions for querying the state and capabilities of a resource, along with functions for actual resource management (e.g., locking resources).

The **connectivity layer** consists of communication protocols for supporting grid transactions that span the usage of multiple resources. For example, protocols are needed to transfer data between resources, or to simply access a resource from a remote location. In addition, the connectivity layer will contain security protocols to authenticate users and resources. Note that in many cases, human users are not authenticated; instead, programs acting on behalf of the users are authenticated. In this sense, delegating rights from a user to programs is an important function that needs to be supported in the connectivity layer. We return to delegation when discussing security in distributed systems in Chapter 9.

The **resource layer** is responsible for managing a single resource. It uses the functions provided by the connectivity layer and calls directly the interfaces made available by the fabric layer. For example, this layer will offer functions for obtaining configuration information on a specific resource, or, in general, to perform specific operations such as creating a process or reading data. The

resource layer is thus seen to be responsible for access control, and hence will rely on the authentication performed as part of the connectivity layer.

The next layer in the hierarchy is the **collective layer**. It deals with handling access to multiple resources and typically consists of services for resource discovery, allocation and scheduling of tasks onto multiple resources, data replication, and so on. Unlike the connectivity and resource layer, each consisting of a relatively small, standard collection of protocols, the collective layer may consist of many protocols reflecting the broad spectrum of services it may offer to a virtual organization.

Finally, the **application layer** consists of the applications that operate within a virtual organization and which make use of the grid computing environment.

Typically, the collective, connectivity, and resource layer form the heart of what could be called a grid middleware layer. These layers jointly provide access to and management of resources that are potentially dispersed across multiple sites.

An important observation from a middleware perspective is that in grid computing, the notion of a site (or administrative unit) is common. This prevalence is emphasized by the gradual shift toward a **service-oriented architecture** in which sites offer access to the various layers through a collection of Web services [Joseph et al., 2004]. This, by now, has led to the definition of an alternative architecture known as the **Open Grid Services Architecture (OGSA)** [Foster et al., 2006]. OGSA is based upon the original ideas as formulated by Foster et al. [2001], yet having gone through a standardization process makes it complex, to say the least. OGSA implementations generally follow Web service standards.

1.3.2 Distributed information systems

Another important class of distributed systems is found in organizations that were confronted with a wealth of networked applications, but for which interoperability turned out to be a painful experience. Many of the existing middleware solutions are the result of working with an infrastructure in which it was easier to integrate applications into an enterprise-wide information system [Alonso et al., 2004; Bernstein, 1996; Hohpe and Woolf, 2004].

We can distinguish several levels at which integration can take place. Often, a networked application simply consists of a server running that application (often including a database) and making it available to remote programs, called **clients**. Such clients send a request to the server for executing a specific operation, after which a response is sent back. Integration at the lowest level allows clients to wrap several requests, possibly for different servers, into a single larger request and have it executed as a **distributed transaction**. The key idea is that all, or none of the requests are executed.

As applications became more sophisticated and were gradually separated into independent components (notably distinguishing database components from processing components), it became clear that integration should also take place by letting applications communicate directly with each other. This has now led to an industry on **enterprise application integration (EAI)**.

Distributed transaction processing

To clarify our discussion, we concentrate on database applications. In practice, operations on a database are carried out in the form of **transactions**. Programming using transactions requires special primitives that must either be supplied by the underlying distributed system or by the language runtime system. Typical examples of transaction primitives are shown in Figure 1.11. The exact list of primitives depends on what kinds of objects are being used in the transaction [Gray and Reuter, 1993; Bernstein and Newcomer, 2009]. In a mail system, there might be primitives to send, receive, and forward mail. In an accounting system, they might be quite different. READ and WRITE are typical examples, however. Ordinary statements, procedure calls, and so on, are also allowed inside a transaction. In particular, **remote procedure calls (RPC)**, that is, procedure calls to remote servers, are often also encapsulated in a transaction, leading to what is known as a **transactional RPC**. We discuss RPCs extensively in Section 4.2.

Primitive	Description
BEGIN_TRANSACTION	Mark the start of a transaction
END_TRANSACTION	Terminate the transaction and try to commit
ABORT_TRANSACTION	Kill the transaction and restore the old values
READ	Read data from a file, a table, or otherwise
WRITE	Write data to a file, a table, or otherwise

Figure 1.11: Example primitives for transactions.

BEGIN_TRANSACTION and END_TRANSACTION are used to delimit the scope of a transaction. The operations between them form the body of the transaction. The characteristic feature of a transaction is either all of these operations are executed or none are executed. These may be system calls, library procedures, or bracketing statements in a language, depending on the implementation.

This all-or-nothing property of transactions is one of the four characteristic properties that transactions have. More specifically, transactions adhere to the so-called **ACID** properties:

- **Atomic:** To the outside world, the transaction happens indivisibly
- **Consistent:** The transaction does not violate system invariants

- **Isolated:** Concurrent transactions do not interfere with each other
- **Durable:** Once a transaction commits, the changes are permanent

In distributed systems, transactions are often constructed as a number of subtransactions, jointly forming a **nested transaction** as shown in Figure 1.12. The top-level transaction may fork off children that run in parallel with one another, on different machines, to gain performance or simplify programming. Each of these children may also execute one or more subtransactions, or fork off its own children.

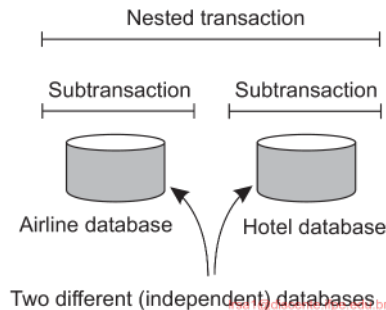


Figure 1.12: A nested transaction.

Subtransactions give rise to a subtle, but important, problem. Imagine that a transaction starts several subtransactions in parallel, and one of these commits, making its results visible to the parent transaction. After further computation, the parent aborts, restoring the entire system to the state it had before the top-level transaction started. Consequently, the results of the subtransaction that committed must nevertheless be undone. Thus, the permanence referred to above applies only to top-level transactions.

Since transactions can be nested arbitrarily deep, considerable administration is needed to get everything right. The semantics are clear, however. When any transaction or subtransaction starts, it is conceptually given a private copy of all data in the entire system for it to manipulate as it wishes. If it aborts, its private universe just vanishes, as if it had never existed. If it commits, its private universe replaces the parent's universe. Thus, if a subtransaction commits and then later a new subtransaction is started, the second one sees the results produced by the first one. Likewise, if an enclosing (higher level) transaction aborts, all its underlying subtransactions have to be aborted as well. And if several transactions are started concurrently, the result is as if they ran sequentially in some unspecified order.

Nested transactions are important in distributed systems, for they provide a natural way of distributing a transaction across multiple machines. They follow a *logical* division of the work of the original transaction. For example, a transaction for planning a trip by which three different flights need to be

reserved can be logically split up into three subtransactions. Each of these subtransactions can be managed separately and independently.

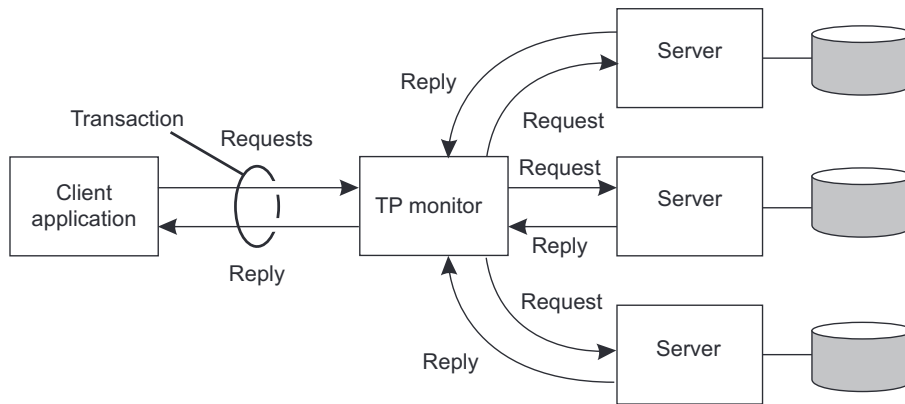


Figure 1.13: The role of a TP monitor in distributed systems.

Ever since the early days of enterprise middleware systems, the component that handles distributed (or nested) transactions belongs to the core for integrating applications at the server or database level. This component is called a **transaction-processing monitor** or **TP monitor** for short. Its main task is to allow an application to access multiple server/databases by offering it a transactional programming model, as shown in Figure 1.13. Essentially, the TP monitor coordinates the commitment of subtransactions following a standard protocol known as **distributed commit**, which we discuss in detail in Section 8.5.

An important observation is that applications wanting to coordinate several subtransactions into a single transaction do not have to implement this coordination themselves. By simply making use of a TP monitor, this coordination is done for them. This is precisely where middleware comes into play: it implements services that are useful for many applications, avoiding that such services have to be reimplemented over and over again by application developers.

Enterprise application integration

As mentioned, the more applications became decoupled from the databases they were built upon, the more evident it became that facilities were needed to integrate applications independently of their databases. In particular, application components should be able to communicate directly with each other and not merely by means of the request/reply behavior that was supported by transaction processing systems.

This need for interapplication communication led to many communication models. The main idea was that existing applications could directly exchange information, as shown in Figure 1.14.

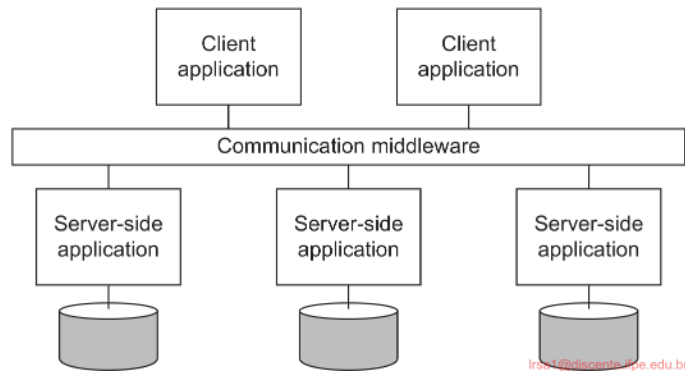


Figure 1.14: Middleware as a communication facilitator in enterprise application integration.

Several types of communication middleware exist. With **remote procedure calls (RPC)**, an application component can effectively send a request to another application component by doing a local procedure call, which results in the request being packaged as a message and sent to the callee. Likewise, the result will be sent back and returned to the application as the result of the procedure call.

As the popularity of object technology increased, techniques were developed to allow calls to remote objects, leading to what is known as **remote method invocations (RMI)**. An RMI is essentially the same as an RPC, except that it operates on objects instead of functions.

RPC and RMI have the disadvantage that the caller and callee both need to be up and running at the time of communication. In addition, they need to know exactly how to refer to each other. This tight coupling is often experienced as a serious drawback, and has led to what is known as **message-oriented middleware**, or simply **MOM**. In this case, applications send messages to logical contact points, often described by a subject. Likewise, applications can indicate their interest for a specific type of message, after which the communication middleware will take care that those messages are delivered to those applications. These so-called **publish-subscribe** systems form an important and expanding class of distributed systems.

Note 1.7 (More information: On integrating applications)

Supporting enterprise application integration is an important goal for many middleware products. In general, there are four ways to integrate applications [Hohpe and Woolf, 2004]:

File transfer: The essence of integration through file transfer, is that an application produces a file containing shared data that is subsequently read by

other applications. The approach is technically simple, making it appealing. The drawback, however, is that there are numerous things that need to be agreed upon:

- File format and layout: text, binary, its structure, and so on. Nowadays, the **extended markup language (XML)** has become popular as its files are, in principle, self-describing.
- File management: where are they stored, how are they named, who is responsible for deleting files?
- Update propagation: When an application produces a file, there may be several applications that need to read that file to provide the view of a single coherent system. As a consequence, sometimes separate programs need to be implemented that notify applications of file updates.

Shared database: Many of the problems associated with integration through files are alleviated when using a shared database. All applications will have access to the same data, and often through a high-level database language such as SQL. Furthermore, it is easy to notify applications when changes occur, as triggers are often part of modern databases. There are, however, two major drawbacks. First, there is still a need to design a common data schema, which may be far from trivial if the set of applications that need to be integrated is not completely known in advance. Second, when there are many reads and updates, a shared database can easily become a performance bottleneck.

Remote procedure call: Integration through files or a database implicitly assumes that changes by one application can easily trigger other applications to act. However, practice shows that sometimes small changes should actually trigger many applications to take actions. In such cases, it is not really the change of data that is important, but the execution of a series of actions.

Series of actions are best captured through the execution of a procedure (which may, in turn, lead to all kinds of changes in shared data). To prevent that every application needs to know all the internals of those actions (as implemented by another application), standard encapsulation techniques should be used, as deployed with traditional procedure calls or object invocations. For such situations, an application can best offer a procedure to other applications in the form of a remote procedure call, or RPC. In essence, an RPC allows an application A to make use of the information available only to the application B, without giving A direct access to that information. There are many advantages and disadvantages to remote procedure calls, which are discussed in depth in Chapter 4.

Messaging: A main drawback of RPCs is that caller and callee need to be up and running at the same time in order for the call to succeed. However, in many scenarios, this simultaneous activity is often difficult or impossible

to guarantee. In such cases, offering a messaging system carrying requests from the application A to perform an action at the application B, is what is needed. The messaging system ensures that eventually the request is delivered, and if needed, that a response is eventually returned as well. Obviously, messaging is not the panacea for application integration: it also introduces problems concerning data formatting and layout, it requires an application to know where to send a message to, there need to be scenarios for dealing with lost messages, and so on. Like RPCs, we will be discussing these issues extensively in Chapter 4.

What these four approaches tell us, is that application integration will generally not be simple. Middleware (in the form of a distributed system), however, can significantly help in integration by providing the right facilities such as support for RPCs or messaging. As said, enterprise application integration is an important target field for many middleware products.

1.3.3 Pervasive systems

The distributed systems discussed so far are largely characterized by their stability: nodes are fixed and have a more or less permanent and high-quality connection to a network. To a certain extent, this stability is realized through the various techniques for achieving distribution transparency. For example, there are many ways how we can create the illusion that only occasionally components may fail. Likewise, there are all kinds of means to hide the actual network location of a node, effectively allowing users and applications to believe that nodes stay put.

However, matters have changed since the introduction of mobile and embedded computing devices, leading to what are generally referred to as **pervasive systems**. As its name suggests, pervasive systems are intended to blend into our environment naturally. Many of their components are *necessarily* spread across multiple computers, making them arguably a type of decentralized system in our view. At the same time, most pervasive systems have many components that are *sufficiently* spread throughout the system, for example, to handle failures and such. In this sense, they are also arguably distributed systems. The seemingly strict separation between decentralized and distributed systems is thus seen to be less strict than one could initially imagine.

What makes them unique, in comparison to the computing and information systems described so far, is that the separation between users and system components is much more blurred. There is often no single dedicated interface, such as a screen/keyboard combination. Instead, a pervasive system is often equipped with many **sensors** that pick up various aspects of a user's behavior. Likewise, it may have a myriad of **actuators** to provide information and feedback, often even purposefully aiming to *steer* behavior.

Many devices in pervasive systems are characterized by being small, battery-powered, mobile, and having only a wireless connection, although not all these characteristics apply to all devices. These are not necessarily restrictive characteristics, as is illustrated by smartphones [Roussos et al., 2005] and their role in what is now coined as the **Internet of Things** [Mattern and Floerkemeier, 2010; Stankovic, 2014]. Nevertheless, notably, the fact that we often need to deal with the intricacies of wireless and mobile communication, will require special solutions to make a pervasive system as transparent or unobtrusive as possible.

In the following, we make a distinction between three different types of pervasive systems, although there is considerable overlap between the three types: ubiquitous computing systems, mobile systems, and sensor networks. This distinction allows us to focus on different aspects of pervasive systems.

Ubiquitous computing systems

So far, we have been talking about pervasive systems to emphasize that its elements have spread through in many parts of our environment. In a ubiquitous computing system, we go one step further: the system is pervasive and continuously present. The latter means that a user will be continuously interacting with the system, often not even being aware that interaction is taking place. Poslad [2009] describes the core requirements for a **ubiquitous computing system** roughly as follows:

1. **(Distribution)** Devices are networked, distributed, and accessible transparently
2. **(Interaction)** Interaction between users and devices is highly unobtrusive
3. **(Context awareness)** The system is aware of a user's context to optimize interaction
4. **(Autonomy)** Devices operate autonomously without human intervention, and are thus highly self-managed
5. **(Intelligence)** The system as a whole can handle a wide range of dynamic actions and interactions

Let us consider these requirements from a distributed-systems perspective.

Ad. 1: Distribution As mentioned, a ubiquitous computing system is an example of a distributed system: the devices and other computers forming the nodes of a system are simply networked and work together to form the illusion of a single, coherent system. Distribution also comes naturally: there will be devices close to users (such as sensors and actuators), connected to computers hidden from view and perhaps even operating remotely in a

cloud. Most, if not all, of the requirements regarding distribution transparency mentioned in Section 1.2.2, should therefore hold.

Ad. 2: Interaction When it comes to interaction with users, ubiquitous computing systems differ a lot in comparison to the systems we have been discussing so far. End users play a prominent role in the design of ubiquitous systems, meaning that special attention needs to be paid to how the interaction between users and core system takes place. For ubiquitous computing systems, much of the interaction by humans will be implicit, with an **implicit action** being defined as one “that is not primarily aimed to interact with a computerized system but which such a system understands as input” [Schmidt, 2000]. In other words, a user could be mostly unaware of the fact that input is being provided to a computer system. From a certain perspective, ubiquitous computing can be said to seemingly *hide* interfaces.

A simple example is where the settings of a car’s driver’s seat, steering wheel, and mirrors are fully personalized. If Bob takes a seat, the system will recognize that it is dealing with Bob and subsequently makes the appropriate adjustments. The same happens when Alice uses the car, while an unknown user will be steered toward making his or her own adjustments (to be remembered for later). This example already illustrates an important role of sensors in ubiquitous computing, namely as input devices that are used to identify a situation (a specific person apparently wanting to drive), whose input analysis leads to actions (making adjustments). In turn, the actions may lead to natural reactions, for example that Bob slightly changes the seat settings. The system will have to take all (implicit and explicit) actions by the user into account and react accordingly.

Ad. 3: Context awareness Reacting to the sensory input, but also the explicit input from users, is more easily said than done. What a ubiquitous computing system needs to do, is to take the **context** in which interactions take place into account. Context awareness also differentiates ubiquitous computing systems from the more traditional systems we have been discussing before, and is described by Dey and Abowd [2000] as “any information that can be used to characterize the situation of entities (i.e., whether a person, place, or object) that are considered relevant to the interaction between a user and an application, including the user and the application themselves.” In practice, context is often characterized by location, identity, time, and activity: the *where*, *who*, *when*, and *what*. A system will need to have the necessary (sensory) input to determine one or several of these context types. As discussed by Alegre et al. [2016], developing context-aware systems is difficult, if only for the reason that the notion of context is difficult to grasp.

What is important, from a distributed-systems perspective, is that raw data as collected by various sensors is lifted to a level of abstraction that can be used by applications. A concrete example is detecting where a person is,

for example in terms of GPS coordinates, and subsequently mapping that information to an actual location, such as the corner of a street, or a specific shop or other known facility. The question is where this processing of sensory input takes place: is all data collected at a central server connected to a database with detailed information on a city, or is it the user's smartphone where the mapping is done? Clearly, there are trade-offs to be considered.

Dey [2010] discusses more general approaches toward building context-aware applications. When it comes to combining flexibility and potential distribution, so-called **shared data spaces** in which processes are decoupled in time and space are attractive, yet as we shall see in later chapters, suffer from scalability problems. A survey on context-awareness and its relation to middleware and distributed systems is provided by Baldauf et al. [2007].

Ad. 4: Autonomy An important aspect of most ubiquitous computing systems is that explicit systems management has been reduced to a minimum. In a ubiquitous computing environment, there is simply no room for a systems administrator to keep everything up and running. As a consequence, the system as a whole should be able to act autonomously, and automatically react to changes. This requires a myriad of techniques, of which several will be discussed throughout this book. To give a few simple examples, think of the following:

Address allocation: In order for networked devices to communicate, they need an IP address. Addresses can be allocated automatically using protocols like the **Dynamic Host Configuration Protocol (DHCP)** [Droms, 1997] (which requires a server) or **Zeroconf** [Guttman, 2001].

Adding devices: It should be easy to add devices to an existing system. A step towards automatic configuration is realized by the **Universal Plug and Play protocol (UPnP)** [UPnP Forum, 2008]. Using UPnP, devices can discover each other and make sure that they can set up communication channels between them.

Automatic updates: Many devices in a ubiquitous computing system should be able to regularly check through the Internet if their software should be updated. If so, they can download new versions of their components and ideally continue where they left off.

Admittedly, these are simple examples, but the picture should be clear that manual intervention is to be kept to a minimum. We will be discussing many techniques related to self-management in detail throughout the book.

Ad. 5: Intelligence Finally, Poslad [2009] mentions that ubiquitous computing systems often use methods and techniques from the field of artificial

intelligence. What this means, is that often a wide range of advanced algorithms and models need to be deployed to handle incomplete input, quickly react to a changing environment, handle unexpected events, and so on. The extent to which this can or should be done in a distributed fashion is crucial from the perspective of distributed systems. Unfortunately, distributed solutions for many problems in the field of artificial intelligence are yet to be found, meaning that there may be a natural tension between the first requirement of networked and distributed devices, and advanced distributed information processing.

Mobile computing systems

As mentioned, mobility often forms an important component of pervasive systems, and many, if not all aspects that we have just discussed also apply to **mobile computing**. There are several issues that set mobile computing aside to pervasive systems in general (see also Adelstein et al. [2005] and Tarkoma and Kangasharju [2009]).

First, the devices that form part of a (distributed) mobile system may vary widely. Typically, mobile computing is done with devices such as smartphones and tablet computers. However, entirely different types of devices are now using the Internet Protocol (IP) to communicate, placing mobile computing in a different perspective. Such devices include remote controls, pagers, active badges, car equipment, various GPS-enabled devices, and so on. A characteristic feature of all these devices is that they use wireless communication. Mobile implies wireless, so it seems (although there are exceptions to the rules).

Second, in mobile computing, the location of a device is assumed to change over time. A changing location has its effects on many issues. For example, if the location of a device changes regularly, so will perhaps the services that are locally available. As a consequence, we may need to pay special attention to dynamically discovering services, but also letting services announce their presence. In a similar vein, we often also want to know where a device actually is. This may mean that we need to know the actual geographical coordinates of a device such as in tracking and tracing applications, but it may also require that we can simply detect its network position (as in mobile IP [Perkins, 2010; Perkins et al., 2011]).

Changing locations may also have a profound effect on communication. For some time, researchers in mobile computing have been concentrating on what are known as **mobile ad hoc networks**, also known as **MANETs**. The basic idea was that a group of local mobile computers would jointly set up a local, wireless network and to subsequently share resources and services. The idea never really became popular. Along the same lines, there are researchers who believe that end users are willing to share their local resources for another user's compute, storage, or communication requirements (see, e.g., Ferrer

et al. [2019]). Practice has shown over and over again that voluntarily making resources available is not something users are willing to do, even if they have resources in abundance. The effect is that in the case of mobile computing, we generally see single mobile devices setting up connections to stationary servers. Changing locations then simply means that those connections need to be handed over by routers on the path from the mobile device to the server. Mobile computing is then brought back to its essence: a mobile device connected to a server (and nothing else). In practice, this means that mobile computing is all about mobile devices making use of cloud-based services, as sketched in Figure 1.15(a).

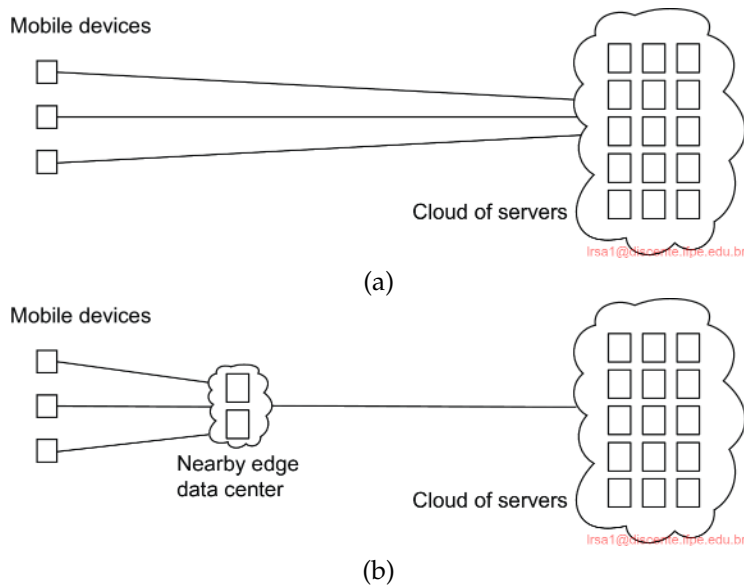


Figure 1.15: (a) Mobile Cloud Computing versus (b) Mobile Edge Computing.

Despite the conceptual simplicity of this model of mobile computing, the mere fact that so many devices make use of remote services has led to what is known as **Mobile Edge Computing**, or simply **MEC** [Abbas et al., 2018], in contrast to **Mobile Cloud Computing (MCC)**. As we shall discuss further in Chapter 2, (mobile) edge computing, as sketched in Figure 1.15(b), is becoming increasingly important in those cases where latency, but also computational issues play a role for the mobile device. Typical example applications that require short latencies *and* computational resources include augmented reality, interactive gaming, real-time sports monitoring, and various health applications [Dimou et al., 2022]. In these examples, the combination of monitoring, analyses, and immediate feedback in general make it difficult to rely on servers that may be placed thousands of miles from the mobile devices.

Sensor networks

Our last example of pervasive systems is **sensor networks**. These networks in many cases form part of the enabling technology for pervasiveness, and we see that many solutions for sensor networks return in pervasive applications. What makes sensor networks interesting from a distributed system's perspective is that they are more than just a collection of input devices. Instead, as we shall see, sensor nodes often collaborate to process the sensed data efficiently in an application-specific manner, making them very different from, for example, traditional computer networks. Akyildiz et al. [2002] and Akyildiz et al. [2005] provide an overview from a networking perspective. A more systems-oriented introduction to sensor networks is given by Zhao and Guibas [2004], but also [Hahmy, 2021] will show to be useful.

A sensor network generally consists of tens to hundreds or thousands of relatively small nodes, each equipped with one or more sensing devices. In addition, nodes can often act as actuators [Akyildiz and Kasimoglu, 2004], a typical example being the automatic activation of sprinklers when a fire has been detected. Many sensor networks use wireless communication, and the nodes are often battery powered. Their limited resources, restricted communication capabilities, and constrained power consumption demand that efficiency is high on the list of design criteria.

When zooming into an individual node, we see that, conceptually, they do not differ a lot from “normal” computers: above the hardware there is a software layer akin to what traditional operating systems offer, including low-level network access, access to sensors and actuators, memory management, and so on. Normally, support for specific services is included, such as localization, local storage (think of additional flash devices), and convenient communication facilities such as messaging and routing. However, similar to other networked computer systems, additional support is needed to effectively deploy sensor network *applications*. In distributed systems, this takes the form of middleware. For sensor networks, we can, in principle, follow a similar approach in those cases that sensor nodes are sufficiently powerful and that energy consumption is not a hindrance for running a more elaborate software stack. Various approaches are possible (see also [Zhang et al., 2021b]).

From a programming perspective, and extensively surveyed by Mottola and Picco [2011], it is important to take the scope of communication primitives into account. This scope can vary between addressing the physical neighborhood of a node, and providing primitives for systemwide communication. In addition, it may also be possible to address a specific group of nodes. Likewise, computations may be restricted to an individual node, a group of nodes, or affect all nodes. To illustrate, Welsh and Mainland [2004] use so-called **abstract regions** allowing a node to identify a neighborhood from where it can, for example, gather information:

```

1  region = k_nearest_region.create(8);
2  reading = get_sensor_reading();
3  region.putvar(reading_key, reading);
4  max_id = region.reduce(OP_MAXID, reading_key);

```

In line 1, a node first creates a region of its eight nearest neighbors, after which it fetches a value from its sensor(s). This reading is subsequently written to the previously defined region to be defined using the key `reading_key`. In line 4, the node checks whose sensor reading in the defined region was the largest, which is returned in the variable `max_id`.

When considering that sensor networks produce data, one can also focus on the data-access model. This can be done directly by sending messages to and between nodes, or either moving code between nodes to locally access data. More advanced is to make remote data directly accessible, as if variables and such were available in a shared data space. Finally, and also quite popular, is to let the sensor network provide a view of a single database. Such a view is easy to understand when realizing that many sensor networks are deployed for measurement and surveillance applications [Bonnet et al., 2002]. In these cases, an operator would like to extract information from (a part of) the network by simply issuing queries such as “What is the northbound traffic load on highway 1 at Santa Cruz?” Such queries resemble those of traditional databases. In this case, the answer will probably need to be provided through collaboration of many sensors along highway 1, while leaving other sensors untouched.

To organize a sensor network as a distributed database, there are essentially two extremes, as shown in Figure 1.16. First, sensors do not cooperate but simply send their data to a centralized database located at the operator’s site. The other extreme is to forward queries to relevant sensors and to let each compute an answer, requiring the operator to aggregate the responses.

Neither of these solutions is very attractive. The first one requires that sensors send all their measured data through the network, which may waste network resources and energy. The second solution may also be wasteful, as it discards the aggregation capabilities of sensors, which would allow much fewer data to be returned to the operator. What is needed are facilities for **in-network data processing**, similar to the previous example of abstract regions.

In-network processing can be done in numerous ways. One obvious way is to forward a query to all sensor nodes along a tree encompassing all nodes and to subsequently aggregate the results as they are propagated back to the root, where the initiator is located. Aggregation will take place where two or more branches of the tree come together. As simple as this scheme may sound, it introduces difficult questions:

- How do we (dynamically) set up an efficient tree in a sensor network?
- How does aggregation of results take place? Can it be controlled?

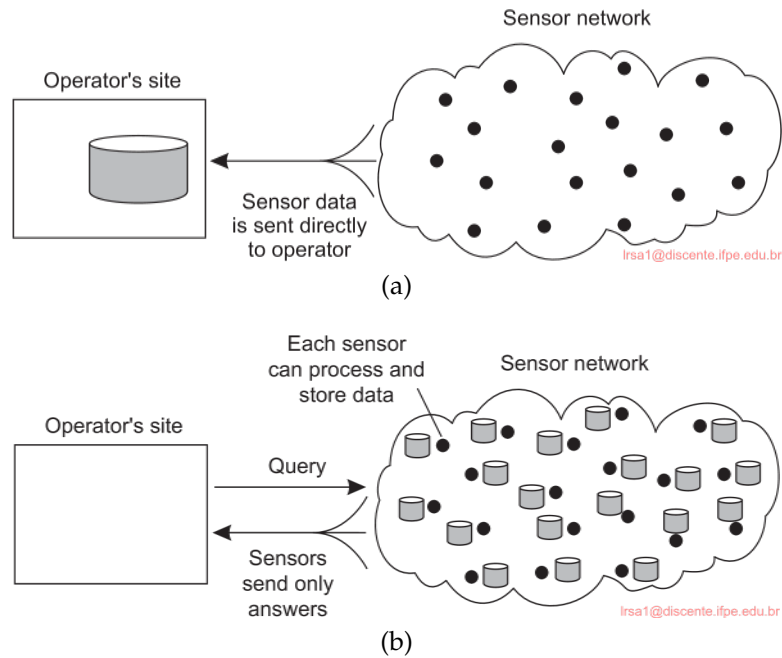


Figure 1.16: Organizing a sensor network database, while storing and processing data (a) only at the operator's site or (b) only at the sensors.

- What happens when network links fail?

These questions have been partly addressed in TinyDB, which implements a declarative (database) interface to wireless sensor networks [Madden et al., 2005]. In essence, TinyDB can use any tree-based routing algorithm. An intermediate node will collect and aggregate the results from its children, along with its own findings, and send that toward the root. To make matters efficient, queries span a period of time, allowing for careful scheduling of operations so that network resources and energy are optimally consumed.

However, when queries can be initiated from different points in the network, using single-rooted trees such as in TinyDB may not be efficient enough. As an alternative, sensor networks may be equipped with special nodes where results are forwarded to, as well as the queries related to those results. To give a simple example, queries and results related to temperature readings may be collected at a different location than those related to humidity measurements. This approach corresponds directly to the notion of publish-subscribe systems.

The interesting aspect of sensor networks, as discussed along these lines, is that we really need to concentrate on the organization of sensor *nodes*, and not the sensors themselves. Likewise, many sensor nodes will be equipped with **actuators**, i.e., devices that directly influence an environment. A typical

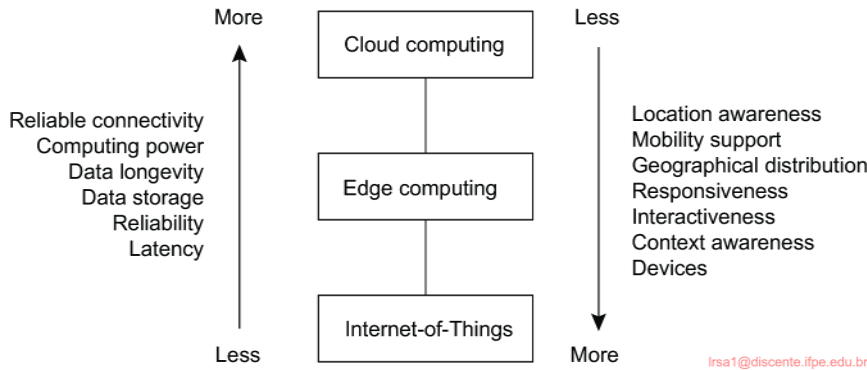


Figure 1.17: A hierarchical view from clouds to devices (adapted from Yousefpour et al. [2019]).

actuator is one that controls the temperature in a room, or switches devices on or off. By viewing and organizing the network as a distributed system, an operator is provided with a higher level of abstraction to monitor and control a situation.

Cloud, edge, things

As may have become clear by now, distributed systems span a huge range of different networked computer systems. Many of such systems operate in a setting in which the various computers are connected through a local-area network. Yet with the growth of the Internet-of-Things and the connectivity with remote services offered through cloud-based systems, new organizations across wide-area networks are emerging. Figure 1.17 presents this more hierarchical approach.

Typically, higher up the hierarchy we see that typical qualities of distributed systems improve: they become more reliable, have more capacity, and, in general, perform better. Lower in the hierarchy, we see that location-related aspects are easier facilitated, as well as performance qualities related to latencies. At the same time, the lower parts show in an increase in the number of devices and computers, whereas higher up, the number of computers becomes less.

1.4 Pitfalls

It should be clear by now that developing a distributed system is a formidable task. As we will see many times throughout this book, there are so many issues to consider, while it seems that only complexity can be the result.

Nevertheless, by following several design principles, distributed systems can be developed that strongly adhere to the goals we set out in this chapter.

Distributed systems differ from traditional software because components are dispersed across a network. Not taking this dispersion into account during design time is what makes so many systems needlessly complex and results in flaws that need to be patched later on. Peter Deutsch, at the time working at Sun Microsystems, formulated these flaws as the following false assumptions that many make when developing a distributed application for the first time:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Note how these assumptions relate to properties that are unique to distributed systems: reliability, security, heterogeneity, and topology of the network; latency and bandwidth; transport costs; and finally administrative domains. When developing nondistributed applications, most of these issues will most likely not show up.

Most of the principles we discuss in this book relate immediately to these assumptions. In all cases, we will be discussing solutions to problems that are caused by the fact that one or more assumptions are false. For example, reliable networks simply do not exist and lead to the impossibility of achieving failure transparency. We devote an entire chapter to deal with the fact that networked communication is inherently insecure. We have already argued that distributed systems need to be open and take heterogeneity into account. Likewise, when discussing replication for solving scalability problems, we are essentially tackling latency and bandwidth problems. We will also touch upon management issues at various points throughout this book.

1.5 Summary

A distributed system is a collection of networked computer systems in which processes and resources are spread across different computers. We make a distinction between *sufficiently* and *necessarily* spread, where the latter relates to decentralized systems. This distinction is important to make, as spreading processes and resources cannot be considered to be a goal by itself. Instead,

most choices for coming to a distributed system come from the need to improve the performance of a single computer system in terms of, for example, reliability, scalability, and efficiency. However, considering that most centralized systems are still much easier to manage and maintain, one should think twice before deciding to spread processes and resources. There are also cases when there is simply no choice, for example when connecting systems belonging to different organizations, or when computers simply operate from different locations (as in mobile computing).

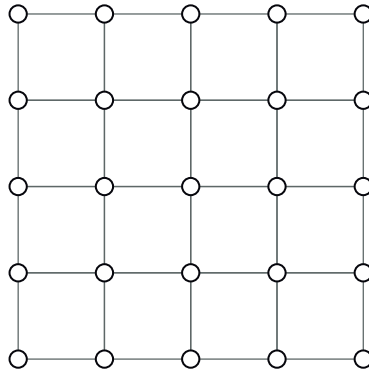
Design goals for distributed systems include sharing resources and ensuring openness. Increasingly important is designing secure distributed systems. In addition, designers aim at hiding many of the intricacies related to the distribution of processes, data, and control. However, this distribution transparency not only comes at a performance price, in practical situations it can never be fully achieved. The fact that trade-offs need to be made between achieving various forms of distribution transparency is inherent to the design of distributed systems, and can easily complicate their understanding. One specific difficult design goal that does not always blend well with achieving distribution transparency is scalability. This is particularly true for geographical scalability, in which case hiding latencies and bandwidth restrictions can turn out to be difficult. Likewise, administrative scalability, by which a system is designed to span multiple administrative domains, may easily conflict with goals for achieving distribution transparency.

Different types of distributed systems exist which can be classified as being oriented toward supporting computations, information processing, and pervasiveness. Distributed computing systems are typically deployed for high-performance applications, often originating from the field of parallel computing. A field that emerged from parallel processing was initially grid computing with a strong focus on worldwide sharing of resources, in turn leading to what is now known as cloud computing. Cloud computing goes beyond high-performance computing and also supports distributed systems found in traditional office environments, where we see databases playing an important role. Typically, transaction processing systems are deployed in these environments. Finally, an emerging class of distributed systems is where components are small, the system is composed in an ad hoc fashion, but most of all is no longer managed through a system administrator. This last class is typically represented by pervasive computing environments, including mobile-computing systems as well as sensor-rich environments.

Matters are further complicated by the fact that many developers initially make assumptions about the underlying network that are fundamentally wrong. Later, when assumptions are dropped, it may turn out to be difficult to mask unwanted behavior. A typical example is assuming that network latency is not significant. Other pitfalls include assuming that the network is reliable, static, secure, and homogeneous.

02

ARCHITECTURES



Distributed systems are often complex pieces of software, of which the components are by definition dispersed across multiple machines. To master their complexity, it is crucial that these systems be properly organized. There are different ways on how to view the organization of a distributed system, but an obvious one is to make a distinction between, on the one hand, the logical organization of the collection of software components, and on the other hand the actual physical realization.

The organization of distributed systems is mostly about the software components that constitute the system. These **software architectures** tell us how the various software components are to be organized and how they should interact. In this chapter, we will first pay attention to some commonly applied architectural styles toward organizing (distributed) computer systems.

An important goal of distributed systems is to separate applications from underlying platforms by providing a so-called middleware layer. Adopting such a layer is an important architectural decision, and its main purpose is to provide distribution transparency. However, trade-offs need to be made to achieve transparency, which has led to various techniques to adjust the middleware to the needs of the applications that make use of it. We discuss some of the more commonly applied techniques, as they affect the organization of the middleware itself.

The actual realization of a distributed system requires that we instantiate and place software components on real machines. There are many choices that can be made in doing so. The final instantiation of a software architecture is also referred to as a **system architecture**. In this chapter, we will look into traditional centralized architectures in which a single server implements most of the software components (and thus functionality), while remote clients can access that server using simple communication means. In addition, we consider decentralized peer-to-peer architectures in which all nodes more or less play equal roles. Many real-world distributed systems are often organized in a hybrid fashion, combining elements from centralized and decentralized architectures. We discuss several examples that illustrate the complexity of many real-world distributed systems.

2.1 Architectural styles

We start our discussion on architectures by first considering the logical organization of a distributed system into software components, also referred to as its **software architecture** [Bass et al., 2021; Richards and Ford, 2020]. Research on software architectures has matured considerably, and it is now commonly accepted that designing or adopting an architecture is crucial for the successful development of large software systems.

For our discussion, the notion of an **architectural style** is important. Such a style is formulated in terms of components, the way that components are

connected to each other, the data exchanged between components, and finally, how these elements are jointly configured into a system. A **component** is a modular unit with well-defined required and provided **interfaces** that is *replaceable* within its environment [OMG, 2004]. That a component can be replaced, and, in particular, while a system continues to operate, is important. This is because often it is not an option to shut down a system for maintenance. At best, only parts of it may be put temporarily out of order. Replacing a component can be done only if its interfaces remain untouched. In practice, we see that replacing or updating a component means that a part of a system (such as a server), runs a regular update and switches to the refreshed components once their installation has finished. Special measures may need to be taken when a part of the distributed system does need to be restarted to let the updates take effect. Such measures may include having replicated standbys that take over while the partial restart is taking place.

A somewhat more difficult concept to grasp is that of a **connector**, which is generally described as a mechanism that mediates communication, coordination, or cooperation among components [Bass et al., 2021]. For example, a connector can be formed by the facilities for (remote) procedure calls, message passing, or streaming data. In other words, a connector allows for the flow of control and data between components.

Using components and connectors, we can come to various configurations, which, in turn, have been classified into architectural styles. Several styles have by now been identified, of which the most important ones for distributed systems are:

- Layered architectures
- Service-oriented architectures
- Publish-subscribe architectures

In the following, we discuss each of these styles separately. We note in advance that in most real-world distributed systems, many styles are combined. Notably, following an approach by which a system is subdivided into several (logical) layers is such a universal principle that it is generally combined with most other architectural styles.

2.1.1 Layered architectures

The basic idea for the layered style is simple: components are organized in a **layered fashion** where a component at layer L_j can make a **downcall** to a component at a lower-level layer L_i (with $i < j$) and generally expects a response. Only in exceptional cases will an **upcall** be made to a higher-level component. The three common cases are shown in Figure 2.1.

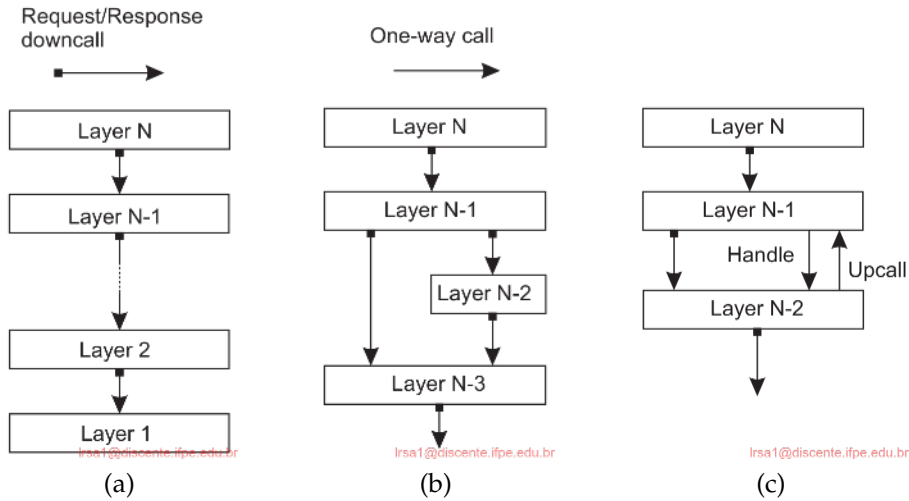


Figure 2.1: (a) Pure layered organization. (b) Mixed layered organization. (c) Layered organization with upcalls (adopted from [Krakowiak, 2009]).

Figure 2.1(a) shows a standard organization in which only downcalls to the next lower layer are made. This organization is commonly deployed in the case of network communication.

In many situations, we also encounter the organization shown in Figure 2.1(b). Consider, for example, an application A that makes use of a library L_{OS} to interface to an operating system. At the same time, the application uses a specialized mathematical library L_{math} that has been implemented by also making use of L_{OS} . In this case, referring to Figure 2.1(b), A is implemented at layer $N - 1$, L_{math} at layer $N - 2$, and L_{OS} which is common to both of them, at layer $N - 3$.

Finally, a special situation is shown in Figure 2.1(c). In some cases, it is convenient to have a lower layer do an upcall to its next higher layer. A typical example is when an operating system signals the occurrence of an event, to which end it calls a user-defined operation for which an application had previously passed a reference (typically referred to as a **handle**).

Layered communication protocols

A well-known and ubiquitously applied layered architecture is that of **communication-protocol stacks**. We will concentrate here on the global picture only and defer a detailed discussion to Section 4.1.1.

In communication-protocol stacks, each layer implements one or several, **communication services** allowing data to be sent from a destination to one or several targets. To this end, each layer offers an **interface** specifying the

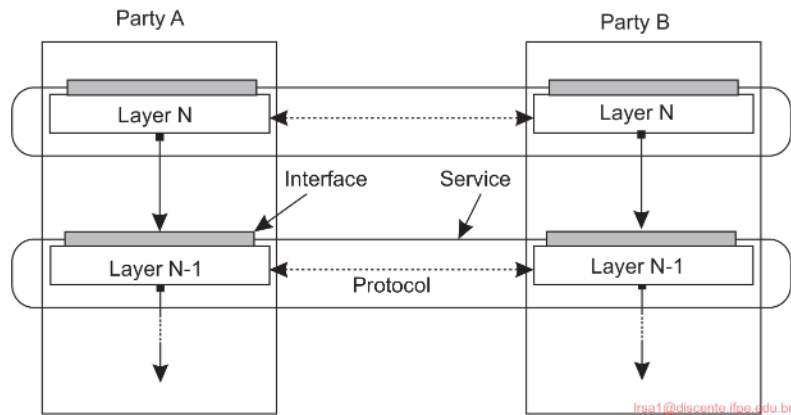


Figure 2.2: A layered communication-protocol stack, showing the difference between a service, its interface, and the protocol it deploys.

functions that can be called. In principle, the interface should completely hide the actual implementation of a service. Another important concept in the case of communication is that of a **(communication) protocol**, which describes the rules that parties will follow to exchange information. It is important to understand the difference between a service offered by a layer, the interface by which that service is made available, and the protocol that a layer implements to establish communication. This distinction is shown in Figure 2.2.

To make this distinction clear, consider a reliable, connection-oriented service, which is provided by many communication systems. In this case, a communicating party first needs to set up a connection to another party before the two can send and receive messages. Being reliable means that strong guarantees will be given that sent messages will indeed be delivered to the other side, even when there is a high risk that messages may be lost (as, for example, may be the case when using a wireless medium). In addition, such services generally also ensure that messages are delivered in the same order as that they were sent.

This kind of service is realized in the Internet by the **Transmission Control Protocol (TCP)**. The protocol specifies which messages are to be exchanged for setting up or tearing down a connection, what needs to be done to preserve the ordering of transferred data, and what both parties need to do to detect and correct data that was lost during transmission. The service is made available in the form of a relatively simple programming interface, containing calls to set up a connection, send and receive messages, and to tear down the connection again. In fact, there are different interfaces available, often dependent on the operating system or programming language used. Likewise, there are many implementations of the protocol and its interfaces. (All the gory details can be found in [Stevens, 1994; Wright and Stevens, 1995].)

Note 2.1 (Example: Two communicating parties)

To make this distinction between service, interface, and protocol more concrete, consider the following two communicating parties, also known as a **client** and a **server**, respectively, expressed in Python (note that some code has been removed for clarity).

```

1 from socket import *
2
3 s = socket(AF_INET, SOCK_STREAM)
4 (conn, addr) = s.accept() # returns new socket and addr. client
5 while True:               # forever
6     data = conn.recv(1024) # receive data from client
7     if not data: break     # stop if client stopped
8     msg = data.decode()+"*" # process the incoming data into a response
9     conn.send(msg.encode()) # return the response
10 conn.close()              # close the connection

```

(a) A simple server

```

1 from socket import *
2
3 s = socket(AF_INET, SOCK_STREAM)
4 s.connect((HOST, PORT)) # connect to server (block until accepted)
5 msg = "Hello World"     # compose a message
6 s.send(msg.encode())    # send the message
7 data = s.recv(1024)    # receive the response
8 print(data.decode())    # print the result
9 s.close()               # close the connection

```

(b) A client

Figure 2.3: Two communicating parties.

In this example, a server is created that makes use of a **connection-oriented service** as offered by the socket library available in Python. This service allows two communicating parties to reliably send and receive data over a connection. The main functions available in its interface are:

- `socket()`: to create an object representing the connection
- `accept()`: a blocking call to wait for incoming connection requests; if successful, the call returns a new socket for a separate connection
- `connect()`: to set up a connection to a specified party
- `close()`: to tear down a connection
- `send()`, `recv()`: to send and receive data over a connection, respectively

The combination of constants `AF_INET` and `SOCK_STREAM` is used to specify that the TCP protocol should be used in the communication between the two parties. These two constants can be seen as part of the interface, whereas making use of TCP is part of the offered service. *How* TCP is implemented, or for that matter, any part of the communication service, is hidden completely from the applications.

Finally, also note that these two programs implicitly adhere to an application-level protocol: apparently, if the client sends some data, the server will return it. Indeed, it operates as an echo server, where the server adds an asterisk to the data sent by the client.

Application layering

Let us now turn our attention to the logical layering of applications. Considering that a large class of distributed applications is targeted toward supporting users or applications access to databases, many people have advocated a distinction between three logical levels, essentially following a layered architectural style:

- The application-interface level
- The processing level
- The data level

In line with this layering, we see that applications can often be constructed from roughly three different pieces: a part that handles interaction with a user or some external application, a part that operates on a database or file system, and a middle part that generally contains the core functionality of the application. This middle part is logically placed at the processing level. In contrast to user interfaces and databases, there are not many aspects common to the processing level. Therefore, we shall give several examples to make this level clearer.

As a first example, consider a simple Internet search engine, for example one dedicated to buying houses. Such search engines appear as seemingly simple Websites through which someone can provide descriptors such as a city or region, a price range, the type of house, etc. The back end consists of a huge database of houses currently for sale. The processing layer does nothing else but transform the provided descriptors into a collection of database queries, retrieves the answers and post-processes these answers by, for example, ranking the output by relevance and subsequently generating an HTML page. Figure 2.4 shows this organization.

As another example, consider the organization of this book's Website, in particular, the interface that allows someone to get a personalized digital copy of the book in PDF. In this case, the interface consists of a WordPress-based Web server that merely collects the user's e-mail address (and some information on exactly which version of the book is being requested). This information is internally appended to a file `requests.txt`. The data layer is simple: it merely consists of a collection of $\text{\LaTeX}$ files and figures that jointly constitute the entire book.

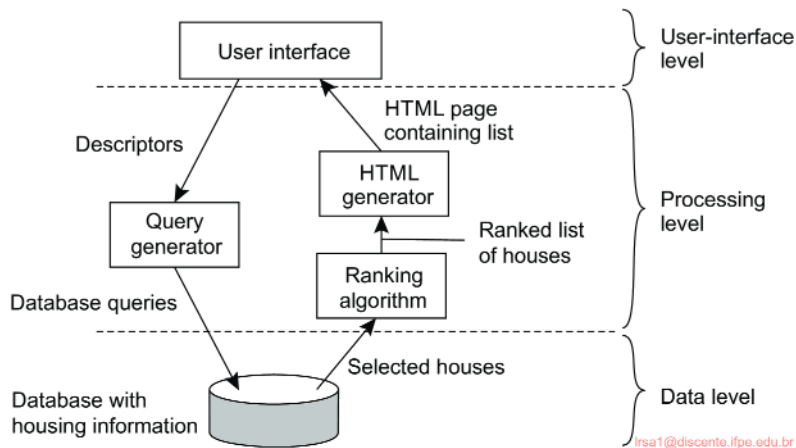


Figure 2.4: The simplified organization of an Internet search engine for housing, into three different layers.

Making a personalized copy consists of embedding the user’s e-mail address into each of the figures. To this end, once every five minutes a separate process is started that takes the list of requests and one-by-one adds the requester’s e-mail address into each bitmapped figure, generates a fresh copy of the book, stores the generated PDF in a special location (accessible through a unique URL), and sends the requester an e-mail that the copy is available for download. This process continues until all requests have been handled. In this example, we thus see that the processing layer outweighs the data layer or the application-interface layer in terms of computational efforts and actions to take.

2.1.2 Service-oriented architectures

Although the layered architectural style is popular, one of its major drawbacks is the often strong dependency between different layers. Good examples where these potential dependencies have been carefully considered are found in designing communication protocol stacks. Bad examples include applications that have essentially been designed and developed as compositions of existing components without much concern for the stability of interfaces or the components themselves, let alone the overlap of functionality between different components. (A compelling example is given by Kucharski [2020], who describes the dependency on a simple component that pads a given string with zeroes or spaces. The author withdrew the component from the NPM library, leaving thousands of programs affected.)

Such direct dependencies to specific components have led to an architectural style reflecting a more loose organization into a collection of separate,

independent entities. Each entity encapsulates a service. Whether they are called **services**, **objects**, or **microservices**, they have in common that the service is executed as a separate process (or thread). Of course, running separate entities does not necessarily lower dependencies in comparison to a layered architectural style.

Object-based architectural style

Taking the object-based approach as an example, we have a logical organization as shown in Figure 2.5. In essence, each object corresponds to what we have defined as a component, and these components are connected through a procedure-call mechanism. In the case of distributed systems, a procedure call can also take place over a network, that is, the calling object need not be executed on the same machine as the called object. In fact, *where* exactly the called object is located can be transparent to the caller: the called object may equally well run as a separate process on the same machine.

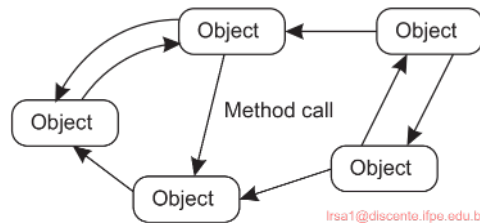


Figure 2.5: An object-based architectural style.

Object-based architectures are attractive because they provide a natural way of **encapsulating** data (called an object's **state**) and the operations that can be performed on that data (which are referred to as an object's **methods**) into a single entity. The **interface** offered by an object conceals implementation details, essentially meaning that we, in principle, can consider an object completely independent of its environment. As with components, this also means that if the interface is clearly defined and left otherwise untouched, an object should be replaceable with one having the same interface.

This separation between interfaces and the objects implementing these interfaces allows us to place an interface at one machine, while the object itself resides on another machine. This organization, which is shown in Figure 2.6 is commonly referred to as a **distributed object**, or every so often a **remote object**.

When a client **binds** to a distributed object, an implementation of the object's interface, called a **proxy**, is then loaded into the client's address space. A proxy is analogous to a so-called **client stub** in RPC systems. The only thing it does is pack method invocations into messages and unpack reply messages to return the result of the method invocation to the client. The actual object

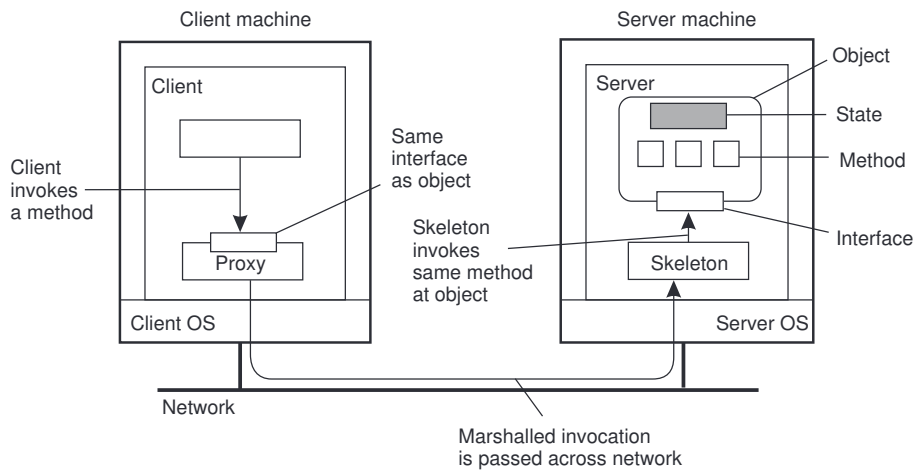


Figure 2.6: Common organization of a remote object with client-side proxy.

resides at a server machine, where it offers the same interface as it does on the client machine. Incoming invocation requests are first passed to a server stub, which unpacks them to make method invocations at the object's interface at the server. The server stub is also responsible for packing return values into a message, and forwarding these reply messages to the client-side proxy.

The server-side stub is often referred to as a **skeleton** as it provides the bare means for letting the server middleware access the user-defined objects. In practice, it often contains incomplete code in the form of a language-specific class that needs to be further specialized by the developer.

A characteristic, but somewhat counterintuitive, feature of most distributed objects is that their state is *not* distributed: it resides at a single machine. Only the interfaces implemented by the object are made available on other machines. Such objects are also referred to as **remote objects**. In a general distributed object, the state itself may be physically distributed across multiple machines, but this distribution is also hidden from clients behind the object's interfaces.

Microservice architectural style

One could argue that object-based architectures form the foundation of encapsulating services into independent units. **Encapsulation** is the keyword here: the service as a whole is realized as a self-contained entity, although it can possibly make use of other services. By clearly separating various services such that they can operate independently, we are paving the road toward **service-oriented architectures**, generally abbreviated as **SOAs**.

Stimulated by object-oriented designs and inspired by the Unix approach in which many, many small and mutually independent programs can be easily

composed to form larger programs, software architects have been working on what are called **microservices**. Essential is that each microservice runs as a separate (network) process. The implementation of a microservice could be in the form of a remote object, but this is not a requirement. Furthermore, despite that people speak of *microservices*, there is no common agreement on what the size of such a service should be. Most important, however, is that a microservice truly represents a separate, independent service. In other words, modularization is key to designing microservices [Wolff, 2017].

Nevertheless, size does matter. By already stating that microservices run as separate networked processes, we are also given a choice *where* to place a microservice. As we shall see later in this chapter, in the advent of edge and fog infrastructures, discussions have started on the **orchestration** of deploying distributed applications across different layers. In other words, where do we place what.

Coarse-grained service composition

In a service-oriented architecture, a distributed application or system is essentially constructed as a composition of many services. A difference (although not strict) with microservices is that not all of these services may belong to the same administrative organization. We already came across this phenomenon when discussing cloud computing: it may very well be that an organization running its business application makes use of storage services offered by a cloud provider. These storage services are logically completely encapsulated into a single unit, of which an interface is made available to customers.

Of course, storage is a rather basic service, but more sophisticated situations easily come to mind. Consider, for example, a Web shop selling goods such as e-books. A simple implementation, following the application layering we discussed previously, may consist of an application for processing orders, which, in turn, operates on a local database containing the e-books. Order processing typically involves selecting items, registering and checking the delivery channel (perhaps by making use of e-mail), but also making sure that a payment takes place. The latter can be handled by a separate service, run by a different organization, to which a purchasing customer is redirected for the payment, after which the e-book organization is notified so that it can complete the transaction. This example also illustrates that where microservices are considered to be relatively small, a general service may be expected to be relatively large. In fact, it is not uncommon to implement a service as a collection of microservices.

In this way, we see that the problem of developing a distributed system is partly one of *service composition*, and making sure that those services operate in harmony. Indeed, this problem is completely analogous to the enterprise application integration issues discussed in Section 1.3.2. Crucial is, and remains, that each service offers a well-defined (programming) interface. In

practice, this also means that each service offers *its own* interface, in turn, possibly making the composition of services far from trivial.

Resource-based architectures

As an increasing number of services became available over the Web and the development of distributed systems through service composition became more important, researchers started to rethink the architecture of mostly Web-based distributed systems. One of the problems with service composition is that connecting various components can easily turn into an integration nightmare.

As an alternative, one can also view a distributed system as a huge collection of resources that are individually managed by components. Resources may be added or removed by (remote) applications, and likewise can be retrieved or modified. This approach has now been widely adopted for the Web and is known as **Representational State Transfer (REST)** [Fielding, 2000]. There are four key characteristics of what are known as **RESTful architectures** [Pautasso et al., 2008]:

1. Resources are identified through a single naming scheme
2. All services offer the same interface, consisting of at most four operations, as shown in Figure 2.7
3. Messages sent to or from a service are fully self-described
4. After executing an operation at a service, that component forgets everything about the caller

The last property is also referred to as a **stateless execution**, a concept to which we return in Chapter 3.

Operation	Description
PUT	Modify a resource by transferring a new state
POST	Create a new resource
GET	Retrieve the state of a resource in some representation
DELETE	Delete a resource

Figure 2.7: The four operations available in RESTful architectures.

To illustrate how RESTful can work in practice, consider a cloud storage service, such as Amazon’s **Simple Storage Service (Amazon S3)**. Amazon S3, described in [Murty, 2008] and more recently in [Culkin and Zazon, 2022], supports two resources: *objects*, which are essentially the equivalent of files, and *buckets*, the equivalent of directories. There is no concept of placing buckets into buckets. An object named *ObjectName* contained in a bucket *BucketName* is referred to by the following **Uniform Resource Identifier (URI)**:

`https://s3.amazonaws.com/BucketName/ObjectName`

To create a bucket, or an object for that matter, an application would essentially send a PUT request with the URI of the bucket/object. In principle, the protocol that is used with the service is HTTP. In other words, it is just another HTTP request, which will subsequently be correctly interpreted by S3. If the bucket or object already exists, an HTTP error message is returned.

Similarly, to know which objects are contained in a bucket, an application would send a GET request with the URI of that bucket. S3 will return a list of object names, again as an ordinary HTTP response.

The RESTful architecture has become popular because of its simplicity. Pautasso et al. [2008] have compared RESTful services to service-specific interfaces, and, as to be expected, they both have their advantages and disadvantages. In particular, the simplicity of RESTful architectures can easily prohibit easy solutions to intricate communication schemes. One example is where distributed transactions are needed, which generally requires that services keep track of the state of execution. On the other hand, there are many examples in which RESTful architectures match perfectly a simple integration scheme of services, yet where the myriad of service interfaces will complicate matters.

Note 2.2 (Advanced: On interfaces)

Clearly, a service cannot be made easier or more difficult just because of the particular interface it offers. A service offers functionality, and at best the way that the service is accessed is determined by the interface. Indeed, one could argue that the discussion on RESTful versus service-specific interfaces is much about access transparency. To better appreciate why so many people are paying attention to this issue, let us zoom in on the **Amazon S3** service, which offers a REST interface as well as a more traditional interface (referred to as the SOAP interface). The fact that the latter has been deprecated out says a lot.

Bucket operations	Object operations
ListAllMyBuckets	PutObjectInline
CreateBucket	PutObject
DeleteBucket	CopyObject
ListBucket	GetObject
GetBucketAccessControlPolicy	GetObjectExtended
SetBucketAccessControlPolicy	DeleteObject
GetBucketLoggingStatus	GetObjectAccessControlPolicy
SetBucketLoggingStatus	SetObjectAccessControlPolicy

Figure 2.8: The operations in Amazon's S3 SOAP interface, by now deprecated.

The SOAP interface consists of approximately 16 operations, listed in Figure 2.8. However, if we were to access Amazon S3 using the Python boto3 library, we would have more than 100 operations available. In contrast, the REST interface

offers only very few operations, essentially those listed in Figure 2.7. Where do these differences come from? The answer is, of course, in the parameter space. In the case of RESTful architectures, an application will need to provide all that it wants through the parameters it passes by one of the operations. In Amazon's SOAP interface, the number of parameters per operation is generally limited, and this is certainly the case if we were to use the Python boto3 library.

Sticking to principles (so that we can avoid the intricacies of real code), suppose that we have an interface bucket that offers an operation create, requiring an input string such as mybucket, for creating a bucket with name "mybucket." Normally, the operation would be called roughly as follows:

```
import bucket
bucket.create("mybucket")
```

However, in a RESTful architecture, the call would need to be essentially encoded as a single string, such as

```
PUT "https://mybucket.s3.amazonaws.com/"
```

The difference is striking. For example, in the first case, many syntactical errors can often already be caught during compile time, whereas in the second case, checking needs to be deferred until runtime. Secondly, one can argue that specifying the semantics of an operation is much easier with specific interfaces than with ones that offer only generic operations. On the other hand, with generic operations, changes are much easier to accommodate, as they would generally involve changing the layout of strings that encode what is actually required.

2.1.3 Publish-subscribe architectures

As systems continue to grow and processes can more easily join or leave, it becomes important to have an architecture in which dependencies between processes become as loose as possible. A large class of distributed systems have adopted an architecture in which there is a strong separation between *processing* and *coordination*. The idea is to view a system as a collection of autonomously operating processes. In this model, **coordination** encompasses the communication and cooperation between processes. It forms the glue that binds the activities performed by processes into a whole [Gelernter and Carriero, 1992].

Cabri et al. [2000] provide a taxonomy of coordination models that can be applied equally to many types of distributed systems. Slightly adapting their terminology, we make a distinction between models along two different dimensions, temporal and referential, as shown in Figure 2.9.

When processes are temporally and referentially coupled, coordination takes place directly, referred to as **direct coordination**. The referential coupling generally appears in the form of explicit referencing in communication. For example, a process can communicate only if it knows the name or identifier of

	Temporally coupled	Temporally decoupled
Referentially coupled	Direct	Mailbox
Referentially decoupled	Event-based	Shared data space

Figure 2.9: Examples of different forms of coordination.

the other processes it wants to exchange information with. Temporal coupling means that processes that are communicating will both have to be up and running. In real life, talking over cell phones (and assuming that a cell phone has only one owner), is an example of direct communication.

A different type of coordination occurs when processes are temporally decoupled, but referentially coupled, which we refer to as **mailbox coordination**. In this case, there is no need for two communicating processes to be active at the same time to let communication take place. Instead, communication takes place by putting messages in a (possibly shared) mailbox. Because it is necessary to explicitly address the mailbox that will hold the messages that are to be exchanged, there is a referential coupling.

The combination of referentially decoupled and temporally coupled systems form the group of models for **event-based coordination**. In referentially decoupled systems, processes do not know each other explicitly. The only thing a process can do is **publish a notification** describing the occurrence of an event (e.g., that it wants to coordinate activities, or that it just produced some interesting results). Assuming that notifications come in all sorts and kinds, processes may **subscribe** to a specific kind of notification (see also [Mühl et al., 2006]). In an ideal event-based coordination model, a published notification will be delivered exactly to those processes that have subscribed to it. However, it is generally required that the subscriber is up-and-running at the time the notification was published.

A well-known coordination model is the combination of referentially and temporally decoupled processes, leading to what is known as a **shared data space**. The key idea is that processes communicate entirely through **tuples**, which are structured data records consisting of several fields, very similar to a row in a database table. Processes can put any type of tuple into the shared data space. To retrieve a tuple, a process provides a search pattern that is matched against the tuples. Any tuple that matches is returned.

Shared data spaces are thus seen to implement an associative search mechanism for tuples. When a process wants to extract a tuple from the data space, it specifies (some of) the values of the fields it is interested in. Any tuple that matches that specification is then removed from the data space and passed to the process.

Shared data spaces are often combined with event-based coordination: a process subscribes to certain tuples by providing a search pattern; when a process inserts a tuple into the data space, matching subscribers are notified. In both cases, we are dealing with a **publish-subscribe** architecture, and indeed, the key characteristic feature is that processes have no explicit reference to each other. The difference between a pure event-based architectural style, and that of a shared data space, is shown in Figure 2.10. We have also shown an abstraction of the *mechanism* by which publishers and subscribers are matched, known as an **event bus**.

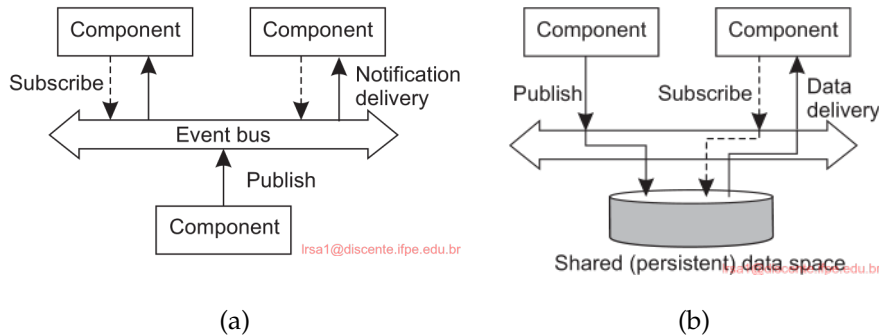


Figure 2.10: The (a) event-based and (b) shared data-space architectural style.

Note 2.3 (Example: Linda tuple spaces)

To make matters a bit more concrete, we take a closer look at **Linda**, a programming model developed in the 1980s [Carriero and Gelernter, 1989]. The shared data space in Linda is known as a **tuple space**, which supports three operations:

- $\text{in}(t)$: remove a tuple that matches the template t
- $\text{rd}(t)$: obtain a copy of a tuple that matches the template t
- $\text{out}(t)$: add the tuple t to the tuple space

Note that if a process would call $\text{out}(t)$ twice in a row, we would find that two copies of tuple t would have been stored. Formally, a tuple space is therefore always modeled as a *multiset*. Both in and rd are *blocking* operations: the caller will be blocked until a matching tuple is found, or has become available.

Consider a simple microblog application in which messages are tagged with the name of its poster and a topic, followed by a short string. Each message is modeled as a tuple $\langle \text{string}, \text{string}, \text{string} \rangle$ where the first string names the poster, the second string represents the topic, and the third one is the actual content. Assuming that we have created a shared data space called MicroBlog, Figure 2.11 shows how Alice and Bob can post messages to that space, and how

Chuck can pick a (randomly selected) message. We have omitted some code for clarity. Note that neither Alice nor Bob knows who will read their postings.

```

1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("bob","distsys","I am studying chap 2"))
4 blog._out(("bob","distsys","The linda example's pretty simple"))
5 blog._out(("bob","gtcn","Cool book!"))

```

(a) Bob's code for creating a microblog and posting three messages.

```

1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("alice","gtcn","This graph theory stuff is not easy"))
4 blog._out(("alice","distsys","I like systems more than graphs"))

```

(b) Alice's code for creating a microblog and posting two messages.

```

1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 t1 = blog._rd(("bob","distsys",str))
4 t2 = blog._rd(("alice","gtcn",str))
5 t3 = blog._rd(("bob","gtcn",str))

```

(c) Chuck reading a message from Bob's and Alice's microblog.

Figure 2.11: A simple example of using a shared data space.

In the first line of each code fragment, a process looks up the tuple space named "MicroBlog." Bob posts three messages: two on topic *distsys*, and one on *gtcn*. Alice posts two messages, one on each topic. Chuck, finally, reads three messages: one from Bob on *distsys* and one on *gtcn*, and one from Alice on *gtcn*.

Obviously, there is much room for improvement. For example, we should ensure that Alice cannot post messages under Bob's name. However, the important issue to note now, is that by providing only tags, a reader such as Chuck will be able to pick up messages without needing to directly reference the poster. In particular, Chuck could also read a randomly selected message on topic *distsys* through the statement

```
t = blog._rd((str,"distsys",str))
```

We leave it as an exercise to the reader to extend the code fragments such that a *next* message will be selected instead of a random one.

An important aspect of publish-subscribe systems is that communication takes place by describing the events that a subscriber is interested in. As a consequence, naming plays a crucial role. We return to naming later, but for now, the important issue is that often, data items are not explicitly identified by senders and receivers.

Let us first assume that events are described by a series of **attributes**. A notification describing an event is said to be **published** when it is made

available for other processes to read. To that end, a **subscription** needs to be passed to the middleware, containing a description of the event that the subscriber is interested in. Such a description typically consists of some (*attribute, value*) pairs, which is common for so-called **topic-based publish-subscribe systems**.

As an alternative, in **content-based publish-subscribe systems**, a subscription may also consist of (*attribute, range*) pairs. In this case, the specified attribute is expected to take on values within a specified range. Descriptions can sometimes be given using all kinds of predicates formulated over the attributes, very similar in nature to SQL-like queries in the case of relational databases. Obviously, the more expressive a description is allowed to be, the more difficult it will be to test whether an event matches a description.

We are now confronted with a situation in which subscriptions need to be **matched** against notifications, as shown in Figure 2.12. Often, an event actually corresponds to data becoming available. In that case, when matching succeeds, there are two possible scenarios. In the first case, the middleware may decide to forward the published notification, along with the associated data, to its current set of subscribers, that is, processes with a matching subscription. As an alternative, the middleware can also forward only a notification, at which point subscribers can execute a read operation to retrieve the data item.

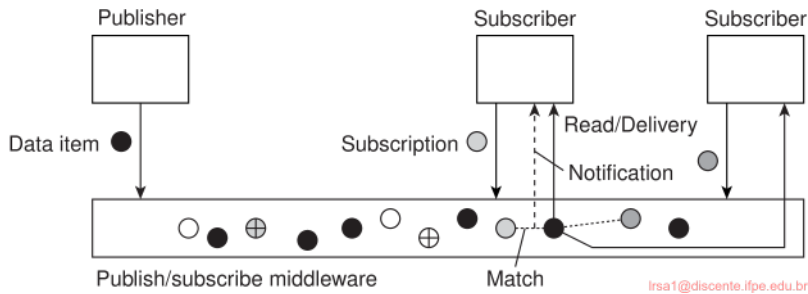


Figure 2.12: The principle of exchanging data items between publishers and subscribers.

In those cases, in which data associated with an event are immediately forwarded to subscribers, the middleware will generally not offer storage of data. Storage is either explicitly handled by a separate service, or is the responsibility of subscribers. In other words, we have a referentially decoupled, but temporally coupled system.

This situation is different when notifications are sent so that subscribers need to explicitly read the associated data. Necessarily, the middleware will have to store data items. In these situations, there are additional operations for data management. It is also possible to attach a **lease** to a data item such that when the lease expires that the data item is automatically deleted.

Events can easily complicate the processing of subscriptions. To illustrate, consider a subscription such as “notify when room ZI.1060 is unoccupied and the door is unlocked.” Typically, a distributed system supporting such subscriptions can be implemented by placing independent sensors for monitoring room occupancy (e.g., motion sensors) and those for registering the status of a door lock. Following the approach sketched so far, we would need to *compose* such primitive events into a publishable data item, to which processes can then subscribe. Event composition turns out to be a difficult task, notably when the primitive events are generated from sources dispersed across the distributed system.

Clearly, in publish-subscribe systems such as these, the crucial issue is the efficient and scalable implementation of matching subscriptions to notifications. From the outside, the publish-subscribe architecture provides lots of potential for building very large-scale distributed systems due to the strong decoupling of processes. On the other hand, devising scalable implementations without losing this independence is not a trivial exercise, notably in the case of content-based publish-subscribe systems. In this sense, although many claim that the publish-subscribe style offers the path toward scalable architectures, the fact is that implementations may easily form a bottleneck, certainly when security and privacy is at stake, as we will discuss in Chapter 9 and later in Chapter 5.

2.2 Middleware and distributed systems

To assist the development of distributed applications, distributed systems are often organized to have a separate layer of software that is logically placed on top of the respective operating systems of the computers that are part of the

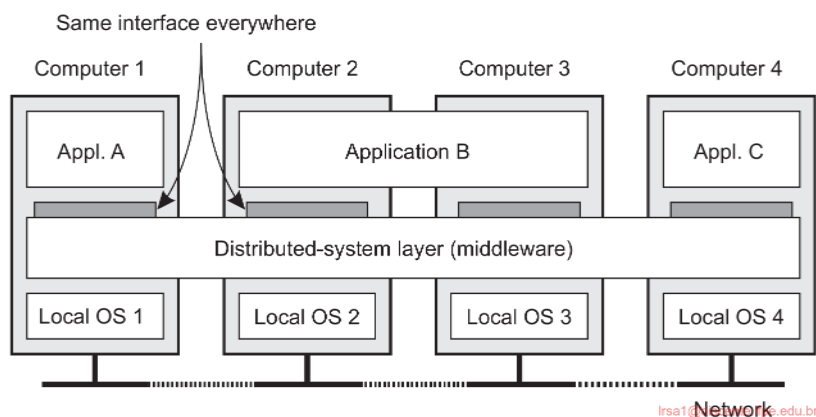


Figure 2.13: A distributed system organized in a middleware layer, which extends over multiple machines, offering each application the same interface.

system. This organization is shown in Figure 2.13, leading to what is known as **middleware** [Bernstein, 1996].

Figure 2.13 shows four networked computers and three applications, of which application B is distributed across computers 2 and 3. Each application is offered the same interface. The distributed system provides the means for components of a single distributed application to communicate with each other, but also to let different applications communicate. At the same time, it hides, as best and reasonably as possible, the differences in hardware and operating systems from each application.

In a sense, middleware is the same to a distributed system as what an operating system is to a computer: a manager of resources offering its applications to efficiently share and deploy those resources across a network. Next to resource management, it offers services that can also be found in most operating systems, including:

- Facilities for interapplication communication.
- Security services.
- Accounting services.
- Masking of and recovery from failures.

The main difference with their operating-system equivalents, is that middleware services are offered in a networked environment. Note also that most services are useful to many applications. In this sense, middleware can also be viewed as a container of commonly used components and functions that now no longer have to be implemented by applications separately.

Note 2.4 (Historical note: The term middleware)

Although the term middleware became popular in the mid 1990s, it was most likely mentioned for the first time in a report on a NATO software engineering conference, edited by Peter Naur and Brian Randell in October 1968 [Naur and Randell, 1968]. Indeed, middleware was placed precisely between applications and service routines (the equivalent of operating systems).

2.2.1 Middleware organization

Let us now zoom into the actual organization of middleware. There are two important types of *design patterns* that are often applied to the organization of middleware: wrappers and interceptors. Each targets different problems, yet addresses the same goal for middleware: achieving openness (as we discussed in Section 1.2.3).

Wrappers

When building a distributed system out of existing components, we immediately bump into a fundamental problem: the interfaces offered by the legacy component are most likely not suitable for all applications. In Section 1.3.2 we discussed how enterprise application integration could be established through middleware as a communication facilitator, but there we still implicitly assumed that, in the end, components could be accessed through their native interfaces.

A **wrapper** or **adapter** is a special component that offers an interface acceptable to a client application, of which the functions are transformed into those available at the component. In essence, it solves the problem of incompatible interfaces (see also Gamma et al. [1994]).

Although originally narrowly defined in the context of object-oriented programming, in the context of distributed systems wrappers are much more than simple interface transformers. For example, an **object adapter** is a component that allows applications to invoke remote objects, although those objects may have been implemented as a combination of library functions operating on the tables of a relational database.

As another example, reconsider Amazon's S3 storage service. As mentioned, there are two types of interfaces available, one adhering to a RESTful architecture, another following a more traditional approach. For the RESTful interface, clients will be using the HTTP protocol, essentially communicating with a traditional Web server which now acts as an adapter to the actual storage service, by partly dissecting incoming requests and subsequently handing them off to specialized servers internal to S3.

Wrappers have always played an important role in extending systems with existing components. Extensibility, which is crucial for achieving openness, used to be addressed by adding wrappers as needed. In other words, if an application A managed data that was needed by an application B, one approach would be to develop a wrapper specific for B so that it could have access to A's data. Clearly, this approach does not scale well: with N applications we would, in theory, need to develop $N \times (N - 1) = \mathcal{O}(N^2)$ wrappers.

Again, facilitating a reduction of the number of wrappers is typically done through middleware. One way of doing this is implementing a so-called **broker**, which is logically a centralized component that handles all the accesses between different applications. An often-used type is a **message broker** of which we discuss the technicalities in Section 4.3.3. In the case of a message broker, applications simply send requests to the broker containing information on what they need. The broker, having knowledge of all relevant applications, contacts the appropriate applications, possibly combines and transforms the responses and returns the result to the initial application. In principle, because a broker offers a single interface to each application, we

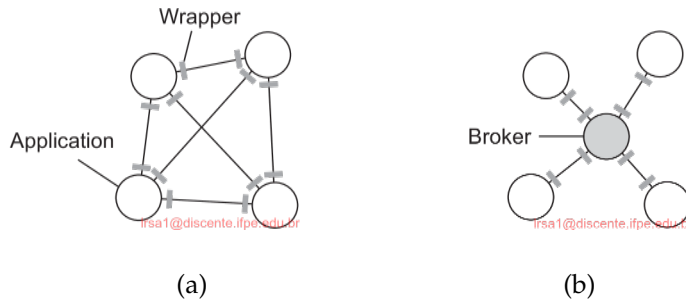


Figure 2.14: (a) Requiring each application to have a wrapper for each other application. (b) Reducing the number of wrappers by making use of a broker.

now need at most $2N = \mathcal{O}(N)$ wrappers instead of $\mathcal{O}(N^2)$. This situation is sketched in Figure 2.14.

Interceptors

Conceptually, an **interceptor** is nothing but a software construct that will break the usual flow of control and allow other (application specific) code to be executed. Interceptors are a primary means for adapting middleware to the specific needs of an application. As such, they play an important role in making middleware open. To make interceptors generic may require a substantial implementation effort, as illustrated by Schmidt et al. [2000], and it is unclear whether in such cases generality should be preferred over restricted applicability and simplicity. Furthermore, often having only limited interception facilities will improve management of the software and the distributed system as a whole.

To make matters concrete, consider interception as supported in many object-based distributed systems. The basic idea is simple: an object A can call a method that belongs to an object B, while the latter resides on a different machine than A. As we explain in detail later in the book, such a remote-object invocation is carried out in three steps:

1. Object A is offered a local interface that is the same as the interface offered by object B. A calls the method available in that interface.
2. The call by A is transformed into a generic object invocation, made possible through a general object-invocation interface offered by the middleware at the machine where A resides.
3. Finally, the generic object invocation is transformed into a message that is sent through the transport-level network interface as offered by A's local operating system.

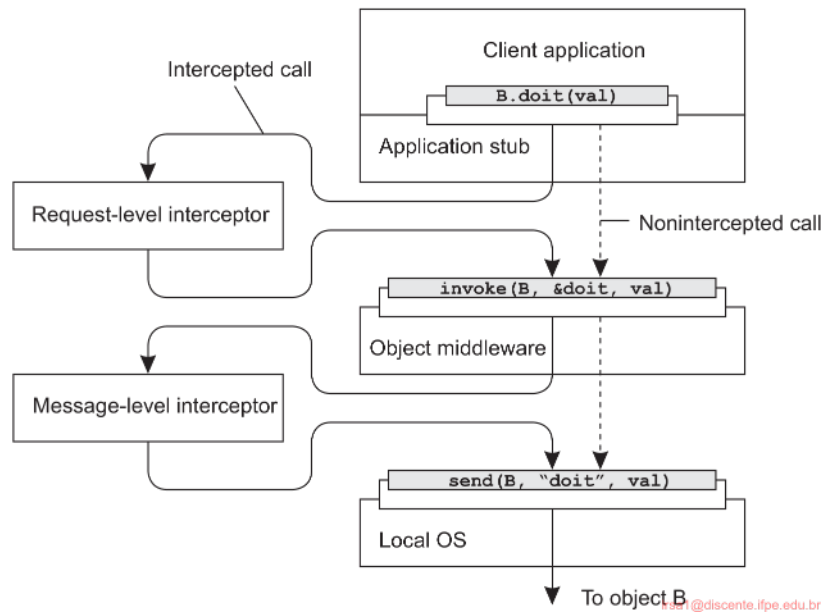


Figure 2.15: Using interceptors to handle remote-object invocations.

This scheme is shown in Figure 2.15. After the first step, the call `B.doit(val)` is transformed into a generic call, such as `invoke(B,&doit,val)` with a reference to B's method and the parameters that go along with the call. Now imagine that object B is replicated. In that case, each replica should actually be invoked. This is a clear point where interception can help. What the **request-level interceptor** will do, is simply call `invoke(B,&doit,val)` for each of the replicas. The beauty of this all is that the object A need not be aware of the replication of B, but also the object middleware need not have special components that deal with this replicated call. Only the request-level interceptor, which may be *added* to the middleware, needs to know about B's replication.

In the end, a call to a remote object will have to be sent over the network. In practice, this means that the messaging interface as offered by the local operating system will need to be invoked. At that level, a **message-level interceptor** may assist in transferring the invocation to the target object. For example, imagine that the parameter `val` actually corresponds to a huge array of data. In that case, it may be wise to fragment the data into smaller parts to have it assembled again at the destination. Such a fragmentation may improve performance or reliability. Again, the middleware need not be aware of this fragmentation; the lower-level interceptor will transparently handle the rest of the communication with the local operating system.

2.2.2 Modifiable middleware

What wrappers and interceptors offer are means to extend and adapt the middleware. The need for adaptation comes from the fact that the environment in which distributed applications are executed changes continuously. Changes include those resulting from mobility, a strong variance in the quality-of-service of networks, failing hardware, and battery drainage, among others. Rather than making applications responsible for reacting to changes, this task is placed in the middleware. Moreover, as the size of a distributed system increases, changing its parts can rarely be done by temporarily shutting it down. What is needed is being able to make changes on-the-fly.

These strong influences from the environment have brought many designers of middleware to consider the construction of *adaptive software*. We follow Parlavantzas and Coulson [2007] in speaking of **modifiable middleware** to express that middleware may not only need to be adaptive, but that we should be able to purposefully modify it without bringing it down. In this context, interceptors can be thought of offering a means to adapt the standard flow of control. Replacing software components at runtime is an example of modifying a system. And indeed, perhaps one of the most popular approaches toward modifiable middleware is that of dynamically constructing middleware from components.

Component-based design focuses on supporting modifiability through composition. A system may either be configured statically at design time, or dynamically at runtime. The latter requires support for late binding, a technique that has been successfully applied in programming language environments, but also for operating systems where modules can be loaded and unloaded at will. Automatically selecting the best implementation of a component during runtime is by now well understood [Yellin, 2003] but again, the process remains complex for distributed systems, especially when considering that replacement of one component requires to know exactly what the effect of that replacement on other components will be. Often, components are less independent as one may think.

The bottom line is that to accommodate dynamic changes to the software that makes up middleware, we need at least basic support to load and unload components at runtime. In addition, for each component explicit specifications of the interfaces it offers, as well the interfaces it requires, are needed. If state is maintained between calls to a component, then further special measures are needed. By-and-large, it should be clear that organizing middleware to be modifiable requires special attention.

2.3 Layered-system architectures

Let us now take a look at how many distributed systems are actually organized by considering where software components are placed. Deciding on software

components, their interaction, and their placement leads to an instance of a software architecture, also known as a **system architecture** [Bass et al., 2021]. We start with discussing layered architectures. Other forms follow later.

Despite the lack of consensus on many distributed systems issues, there is one issue that many researchers and practitioners agree upon: thinking in terms of *clients* that request services from *servers* helps to understand and manage the complexity of distributed systems [Saltzer and Kaashoek, 2009]. In the following, we first consider a simple layered organization, followed by looking at multi-layered organizations.

2.3.1 Simple client-server architecture

In the basic client-server model, processes in a distributed system are divided into two (possibly overlapping) groups. A **server** is a process implementing a specific service, for example, a file system service or a database service. A **client** is a process that requests a service from a server by sending it a request and subsequently waiting for the server's reply. This client-server interaction, also known as **request-reply behavior** is shown in Figure 2.16.

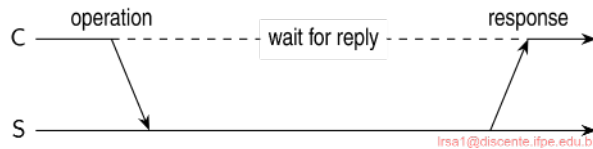


Figure 2.16: General interaction between a client C and a server S. C sends the operation oper and waits for the response from S.

Communication between a client and a server can be implemented by a simple connectionless protocol when the underlying network is fairly reliable, as in many local-area networks. In these cases, when a client requests a service, it simply packages a message for the server, identifying the service it wants, along with the necessary input data. The message is then sent to the server. The latter, in turn, will always wait for an incoming request, subsequently process it, and package the results in a reply message that is then sent to the client.

Using a connectionless protocol has the obvious advantage of being efficient. As long as messages do not get lost or corrupted, the request/reply protocol just sketched works fine. Unfortunately, making the protocol resistant to occasional transmission failures is not trivial. The only thing we can do is possibly let the client resend the request when no reply message comes in. The problem, however, is that the client cannot detect whether the original request message was lost, or that transmission of the reply failed. If the reply was lost, then resending a request may result in performing the operation twice. If the

operation was something like “transfer \$10,000 from my bank account,” then clearly, it would have been better that we simply reported an error instead. On the other hand, if the operation was “tell me how much money I have left,” it would be perfectly acceptable to resend the request. When an operation can be repeated multiple times without harm, it is said to be **idempotent**. Since some requests are idempotent and others are not, it should be clear that there is no single solution for dealing with lost messages. We defer a detailed discussion on handling transmission failures to Section 8.3.

As an alternative, many client-server systems use a reliable connection-oriented protocol. Although this solution is not entirely appropriate in a local-area network due to relatively low performance, it works perfectly fine in wide-area systems in which communication is inherently unreliable. For example, virtually all Internet application protocols are based on reliable TCP/IP connections. In this case, whenever a client requests a service, it first sets up a connection to the server before sending the request. The server generally uses that same connection to send the reply message, after which the connection is torn down. The trouble may be that setting up and tearing down a connection is relatively costly, especially when the request and reply messages are small.

The client-server model has been subject to many debates and controversies over the years. One of the main issues was how to draw a clear distinction between a client and a server. Not surprisingly, there is often no clear distinction. For example, a server for a distributed database may continuously act as a client because it is forwarding requests to different file servers responsible for implementing the database tables. In such a case, the database server itself only processes the queries.

2.3.2 Multitiered Architectures

The distinction into three logical levels, as discussed so far, suggests several possibilities for physically distributing a client-server application across several machines. The simplest organization is to have only two types of machines:

1. A client machine containing only the programs implementing (part of) the user-interface level
2. A server machine containing the rest, that is, the programs implementing the processing and data level

In this organization everything is handled by the server while the client is essentially no more than a dumb terminal, possibly with only a convenient graphical interface. There are, however, many other possibilities. As explained in Section 2.1.1, many distributed applications are divided into three layers: (1) a user-interface layer, (2) a processing layer, and (3) a data layer. One

approach for organizing clients and servers is then to distribute these layers across different machines, as shown in Figure 2.17 (see also Umar [1997]). As a first step, we make a distinction between only two kinds of machines: client machines and server machines, leading to what is also referred to as a **(physically) two-tiered architecture**.

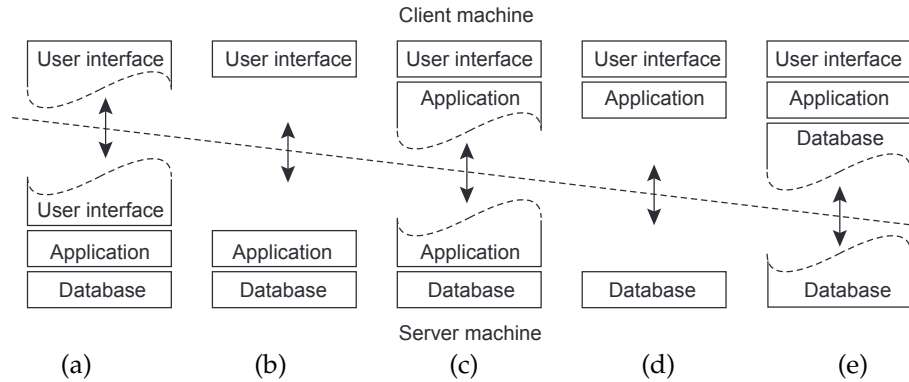


Figure 2.17: Client-server organizations in a two-tiered architecture.

One possible organization is to have only the terminal-dependent part of the user interface on the client machine, as shown in Figure 2.17(a), and give the applications remote control over the presentation of their data. An alternative is to place the entire user-interface software on the client side, as shown in Figure 2.17(b). In such cases, we essentially divide the application into a graphical front end, which communicates with the rest of the application (residing at the server) through an application-specific protocol. In this model, the front end (the client software) does no processing other than necessary for presenting the application's interface.

Continuing along this line of reasoning, we may also move part of the application to the front end, as shown in Figure 2.17(c). An example where this makes sense is where the application makes use of a form that needs to be filled in entirely before it can be processed. The front end can then check the correctness and consistency of the form, and where necessary interact with the user. Another example of the organization of Figure 2.17(c), is that of a word processor in which the basic editing functions execute on the client side where they operate on locally cached, or in-memory data, but where the advanced support tools such as checking the spelling and grammar execute on the server side.

In many client-server environments, the organizations shown in Figure 2.17(d) and Figure 2.17(e) are particularly popular. These organizations are used where the client machine is a PC or workstation, connected through a network to a distributed file system or database. Essentially, most of the

application is running on the client machine, but all operations on files or database entries go to the server. For example, many banking applications run on an end-user's machine, where the user prepares transactions and such. Once finished, the application contacts the database on the bank's server and uploads the transactions for further processing. Figure 2.17(e) represents the situation where the client's local disk contains part of the data. For example, when browsing the Web, a client can gradually build a huge cache on local disk of most recent inspected Web pages.

Note 2.5 (More information: Is there something like the best organization?)

We note that there has been a strong trend to move away from the configurations shown in Figure 2.17(d) and Figure 2.17(e) in those cases, that client software is placed at end-user machines. Instead, most of the processing and data storage is handled at the server side. The reason for this is simple: although client machines do a lot, they are also more problematic to manage. Having more functionality on the client machine means that a wide range of end users will need to be able to handle that software. This implies that more effort needs to be spent on making software resilient to end-user behavior. In addition, client-side software is dependent on the client's underlying platform (i.e., operating system and resources), which can easily mean that multiple versions will need to be maintained. From a systems-management perspective, having what are called **fat clients** is not optimal. Instead, the **thin clients** as represented by the organizations shown in Figure 2.17(a)–(c) are much easier, perhaps at the cost of less sophisticated user interfaces and client-perceived performance.

Does this mean the end of fat clients? Not in the least. For one thing, there are many applications for which a fat-client organization is often still the best. We already mentioned office suites, but also many multimedia applications require that processing is done on the client's side. Moreover, when end users need to operate offline, we see that installing applications will be necessary. Second, with the advent of advanced Web browsing technology, it is now much easier to dynamically place and manage client-side software by simply uploading (the sometimes very sophisticated) scripts to the client. Combined with the fact that this type of client-side software runs in well-defined commonly deployed environments, and thus that platform dependency is much less of an issue, we see that the counter-argument of management complexity is often no longer valid. This has led to the deployment of **virtual desktop environments**, which we discuss further in Chapter 3.

Finally, note that moving away from fat clients does not imply that we no longer need distributed systems. On the contrary, what we continue to see is that server-side solutions are becoming increasingly more distributed as a single server is being replaced by multiple servers running on different machines. Cloud computing is a good example in this case: the complete server side is being executed in data centers, and generally on multiple servers.

When distinguishing only client and server machines as we did so far, we miss the point that a server may sometimes need to act as a client, as shown

in Figure 2.18, leading to a **(physically) three-tiered architecture**.

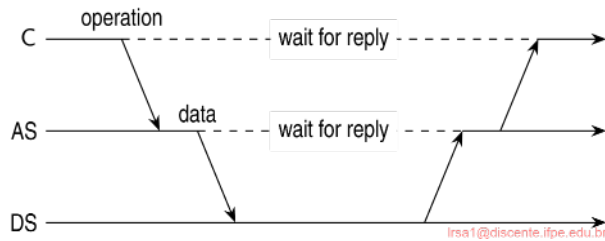


Figure 2.18: An example of an application server AS acting as client for a database server DS.

In this architecture, traditionally programs that form part of the processing layer are executed by a separate server, but may additionally be partly distributed across the client and server machines. A typical example of where a three-tiered architecture is used is in transaction processing. A separate process, called the transaction processing monitor, coordinates all transactions across possibly different data servers.

Another, but very different example where we often see a three-tiered architecture is in the organization of Websites. In this case, a Web server acts as an entry point to a site, passing requests to an application server where the actual processing takes place. This application server, in turn, interacts with a database server. We already came across such an organization when discussing this book's Website and the facilities for generating and downloading a personalized copy.

2.3.3 Example: The Network File System

Many distributed files systems are organized like client-server architectures, with Sun Microsystem's **Network File System (NFS)** being one of the most widely deployed ones for Unix systems [Callaghan, 2000; Haynes, 2015; Noveck and Lever, 2020].

The basic idea behind NFS is that each file server provides a standardized view of its local file system. In other words, it should not matter how that local file system is implemented; each NFS server supports the same model. This approach has been adopted for other distributed files systems as well. NFS comes with a communication protocol that allows clients to access the files stored on a server, thus allowing a heterogeneous collection of processes, possibly running on different operating systems and machines, to share a common file system.

The model underlying NFS and similar systems is that of a **remote file service**. In this model, clients are offered transparent access to a file system that is managed by a remote server. However, clients are normally unaware

of the actual location of files. Instead, they are offered an interface to a file system that is similar to the interface offered by a conventional local file system. In particular, the client is offered only an interface containing various file operations, but the server is responsible for implementing those operations. This model is therefore also referred to as the **remote access model**. It is shown in Figure 2.19(a).

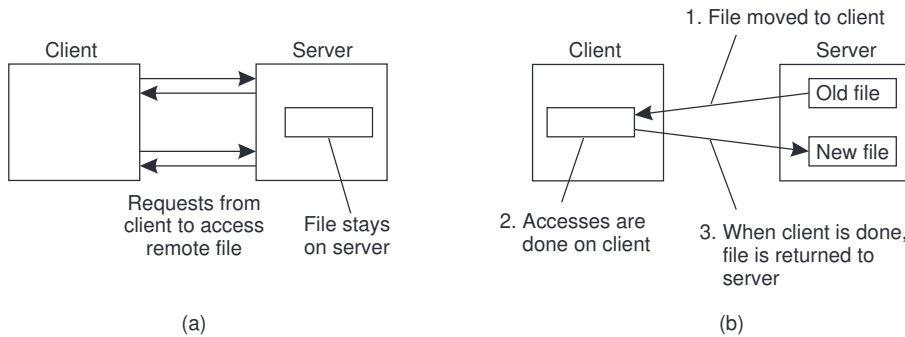


Figure 2.19: (a) The remote access model. (b) The upload/download model.

In contrast, in the **upload/download model** a client accesses a file locally after having downloaded it from the server, as shown in Figure 2.19(b). When the client is finished with the file, it is uploaded back to the server again so that it can be used by another client. The Internet's FTP service can be used this way when a client downloads a complete file, modifies it, and then puts it back.

NFS has been implemented for numerous operating systems, although the Unix versions are predominant. For virtually all modern Unix systems, NFS is generally implemented following the architecture shown in Figure 2.20.

A client accesses the file system using the system calls provided by its local operating system. However, the local Unix file system interface is replaced by an interface to the **Virtual File System (VFS)**, which by now is a de facto standard for interfacing to different (distributed) file systems [Kleiman, 1986]. Virtually all modern operating systems provide VFS, and not doing so more or less forces developers to largely reimplement huge parts of an operating system when adopting a new file-system structure. With NFS, operations on the VFS interface are either passed to a local file system, or passed to a separate component known as the **NFS client**, which takes care of handling access to files stored at a remote server. In NFS, all client-server communication is done through so-called **remote procedure calls (RPCs)**. As mentioned before, an RPC is essentially a standardized way to let a client on a machine A make an ordinary call to a procedure that is implemented on another machine B. We discuss RPCs extensively in Chapter 4. The NFS client implements the NFS file system operations as remote procedure calls to the server. Note that the operations offered by the VFS interface can be different from those offered by

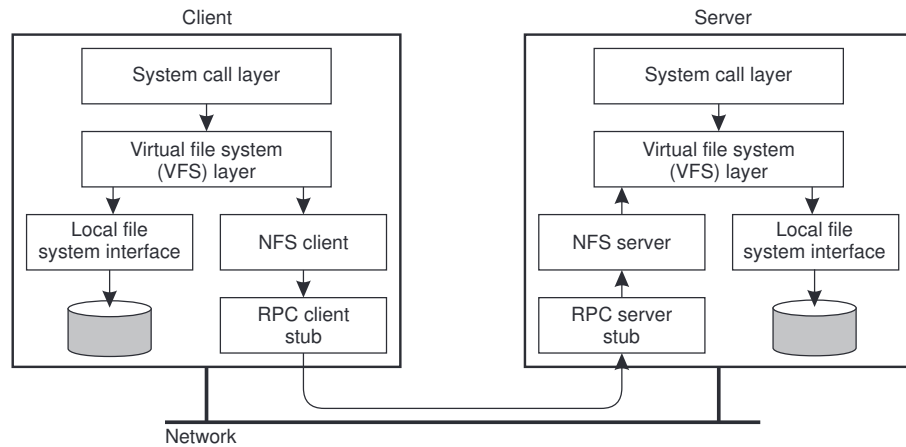


Figure 2.20: The basic NFS architecture for Unix systems.

the NFS client. The whole idea of the VFS is to hide the differences between various file systems.

On the server side, we see a similar organization. The **NFS server** is responsible for handling incoming client requests. The RPC component at the server converts incoming requests to regular VFS file operations that are subsequently passed to the VFS layer. Again, the VFS is responsible for implementing a local file system in which the actual files are stored.

An important advantage of this scheme is that NFS is largely independent of local file systems. It does not matter whether the operating system at the client or server uses a Unix file system, a Windows file system, or even an old MS-DOS file system. The only important issue is that these file systems are compliant with the file system model offered by NFS. For example, MS-DOS with its short file names cannot be used to implement an NFS server in a fully transparent way.

2.3.4 Example: The Web

The architecture of Web-based distributed systems is not fundamentally different from other distributed systems. However, it is interesting to see how the initial idea of supporting distributed documents has evolved since its inception in the 1990s. Documents turned from being purely static and passive to dynamically generated content. Furthermore, recently, many organizations have begun supporting services instead of just documents.

Simple Web-based systems

Many Web-based systems are still organized as relatively simple client-server architectures. The core of a Web site is formed by a process that has access to a

local file system storing documents. The simplest way to refer to a document is by a reference called a **uniform resource locator (URL)**. It specifies where a document is located by embedding the DNS name of its associated server along with a file name by which the server can look up the document in its local file system. Furthermore, a URL specifies the application-level protocol for transferring the document across the network.

A client interacts with Web servers through a **browser**, which is responsible for properly displaying a document. Furthermore, a browser accepts input from a user mostly by letting the user select a reference to another document, which it then subsequently fetches and displays. The communication between a browser and Web server is standardized: they both adhere to the **HyperText Transfer Protocol (HTTP)**. This leads to the overall organization shown in Figure 2.21.

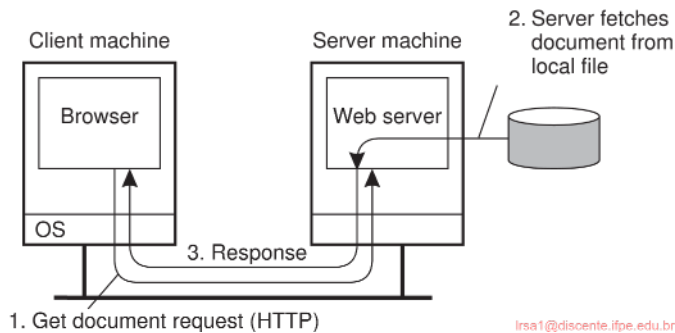


Figure 2.21: The overall organization of a traditional Web site.

Let us zoom in a bit into what a document actually is. Perhaps the simplest form is a standard text file. In that case, the server and browser have barely anything to do: the server copies the file from the local file system and transfers it to the browser. The latter, in turn, merely displays the content of the file ad verbatim without further ado.

More interesting are Web documents that have been marked up, which is usually done in the **HyperText Markup Language**, or simply **HTML**. In that case, the document includes various instructions expressing how its content should be displayed, similar to what one can expect from any decent word-processing system (although those instructions are normally hidden from the end user). For example, instructing text to be emphasized is done by the following markup:

```
<emph>Emphasize this text</emph>
```

There are many more of such markup instructions. The point is that the browser understands these instructions and will act accordingly when displaying the text.

Documents can contain much more than just markup instructions. In particular, they can have complete programs embedded, of which **JavaScript** is the one most often deployed. In this case, the browser is warned that there is some code to execute as in:

```
<script type="text/javascript">...</script>
```

and as long as the browser has an appropriate embedded interpreter for the specified language, everything between “<script>” and “</script>” will be executed as any other program. The main benefit of including scripts is that it allows for much better interaction with the end user, including sending information back to the server. (The latter, by the way, has always been supported in HTML through **forms**.)

Much more can be said about Web documents, but this is not the place to do so. A good introduction on how to build Web-based applications can be found in [Sebesta, 2015].

Multitiered architectures

The Web started out as the relatively simple two-tiered client-server system shown in Figure 2.21. By now, this simple architecture has been extended to support much more sophisticated means of documents. In fact, one could justifiably argue that the term “document” is no longer appropriate. For one, most things that we get to see in our browser has been generated on the spot as the result of sending a request to a Web server. Content is stored in a database at the server’s side, along with client-side scripts and such, to be composed on-the-fly into a document which is then subsequently sent to the client’s browser. Documents have thus become completely dynamic.

One of the first enhancements to the basic architecture was support for simple user interaction by the **Common Gateway Interface** or simply **CGI**. CGI defines a standard way by which a Web server can execute a program taking user data as input. Usually, user data come from an HTML form; it specifies the program that is to be executed at the server side, along with parameter values that are filled in by the user. Once the form has been completed, the program’s name and collected parameter values are sent to the server, as shown in Figure 2.22.

When the server sees the request, it starts the program named in the request and passes it the parameter values. At that point, the program simply does its work and generally returns the results in the form of a document that is sent back to the user’s browser to be displayed.

CGI programs can be as sophisticated as a developer wants. For example, as shown in Figure 2.22 many programs operate on a database local to the Web server. After processing the data, the program generates an HTML document and returns that document to the server. The server will then pass the document to the client. An interesting observation is that to the server,

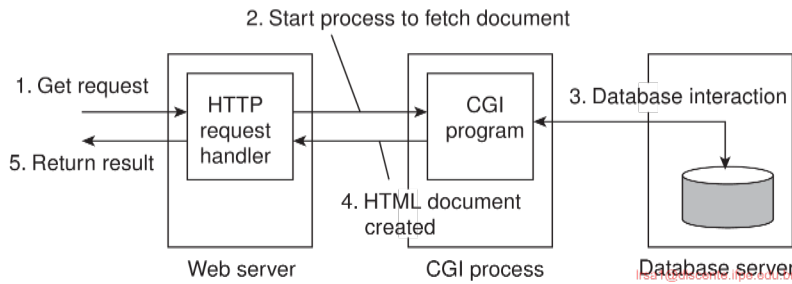


Figure 2.22: The principle of using server-side CGI programs.

it appears as if it is asking the CGI program to fetch a document. In other words, the server does nothing but delegate the fetching of a document to an external program.

The main task of a server used to be handling client requests by simply fetching documents. With CGI programs, fetching a document could be delegated in such a way that the server would remain unaware of whether a document had been generated on the fly, or actually read from the local file system. Note that we have just described a two-tiered organization of server-side software.

However, servers nowadays do much more than just fetching documents. One of the most important enhancements is that servers can also process a document before passing it to the client. In particular, a document may contain a **server-side script**, which is executed by the server when the document has been fetched locally. The result of executing a script is sent along with the rest of the document to the client. The script itself is not sent. In other words, using a server-side script changes a document by essentially replacing the script with the results of its execution. To make matters concrete, take a look at a simple example of dynamically generating a document. Assume a file is stored at the server with the following content:

```
<strong> <?php echo $_SERVER['REMOTE_ADDR']; ?> </strong>
```

The server will examine the file and subsequently process the PHP code (between “<?php” and “?>”) replacing the code with the address of the requesting client. Much more sophisticated settings are possible, such as accessing a local database and subsequently fetching content from that database to be combined with other dynamically generated content.

2.4 Symmetrically distributed system architectures

Multitiered client-server architectures are a direct consequence of dividing distributed applications into a user interface, processing components, and

data-management components. The different tiers correspond directly with the logical organization of applications. In many business environments, distributed processing is equivalent to organizing a client-server application as a multitiered architecture. We refer to this type of distribution as **vertical distribution**. The characteristic feature of vertical distribution is that it is achieved by placing logically different components on different machines. The term is related to the concept of *vertical fragmentation* as used in distributed relational databases, where it means that tables are split columnwise, and subsequently distributed across multiple machines [Özsu and Valduriez, 2020].

Again, from a systems-management perspective, having a vertical distribution can help: functions are logically and physically split across multiple machines, where each machine is tailored to a specific group of functions. However, vertical distribution is only one way of organizing client-server applications. In modern architectures, it is often the distribution of the clients and the servers that counts, which we refer to as **horizontal distribution**. In this type of distribution, a client or server may be physically split up into logically equivalent parts, but each part is operating on its own share of the complete data set, thus balancing the load. In this section, we will take a look at a class of modern system architectures that support horizontal distribution, known as **peer-to-peer systems**.

From a high-level perspective, the processes that constitute a peer-to-peer system are all equal. This means that the functions that need to be carried out are represented by every process that constitutes the distributed system. As a consequence, much of the interaction between processes is symmetric: each process will act as a client and a server at the same time (which is also referred to as acting as a **servant**).

Given this symmetric behavior, peer-to-peer architectures revolve around the question of how to organize the processes in an **overlay network** [Tarkoma, 2010]: a network in which the nodes are formed by the processes and the links represent the possible communication channels (which are often realized as TCP connections). A node may not be able to communicate directly with an arbitrary other node, but is required to send messages through the available communication channels. Two types of overlay networks exist: those that are structured and those that are not. These two types are surveyed extensively in Lua et al. [2005] along with numerous examples. Buford and Yu [2010] additionally includes an extensive list of various peer-to-peer systems. Aberer et al. [2005] provide a reference architecture that allows for a more formal comparison of the different types of peer-to-peer systems. A survey taken from the perspective of content distribution is provided by Androutsellis-Theotokis and Spinellis [2004]. Finally, Buford et al. [2009], Tarkoma [2010] and Vu et al. [2010] go beyond the level of surveys and form adequate textbooks for initial or further study.

2.4.1 Structured peer-to-peer systems

As its name suggests, in a **structured peer-to-peer system** the nodes (i.e., processes) are organized in an overlay that adheres to a specific, deterministic topology: a ring, a binary tree, a grid, etc. This topology is used to efficiently look up data. Characteristic for structured peer-to-peer systems, is that they are generally based on using a so-called semantic-free index. What this means is that each data item that is to be maintained by the system, is uniquely associated with a key, and that this key is subsequently used as an index. To this end, it is common to use a hash function so that we get:

$$\text{key}(\text{data item}) = \text{hash}(\text{data item's value}).$$

The peer-to-peer system as a whole is now responsible for storing $(\text{key}, \text{value})$ pairs. To this end, each node is assigned an identifier from the same set of all possible hash values, and each node is made responsible for storing data associated with a specific subset of keys. In essence, the system is thus seen to implement a **distributed hash table**, generally abbreviated to a **DHT** [Balakrishnan et al., 2003].

Following this approach now reduces the essence of structured peer-to-peer systems to being able to look up a data item by its key. That is, the system provides an efficient implementation of a function *lookup* that maps a key to an *existing* node:

$$\text{existing node} = \text{lookup}(\text{key}).$$

This is where the topology of a structured peer-to-peer system plays a crucial role. Any node can be asked to look up a given key, which then boils down to efficiently *routing* that lookup request to the node responsible for storing the data associated with the given key.

To clarify these matters, let us consider a simple peer-to-peer system with a *fixed* number of nodes, organized into a hypercube. A **hypercube** is an n -dimensional cube. The hypercube shown in Figure 2.23 is four-dimensional. It can be thought of as two ordinary cubes, each with 8 vertices and 12 edges. To expand the hypercube to five dimensions, we would add another set of two interconnected cubes to the figure, connect the corresponding edges in the two halves, and so on.

For this (admittedly naive) system, each data item is associated with one of the 16 nodes. This can be achieved by hashing the value of a data item to a key $k \in \{0, 1, 2, \dots, 2^4 - 1\}$. Now suppose that the node with identifier 0111 is requested to look up the data having key 14, corresponding to the binary value 1110. In this example, we assume that the node with identifier 1110 is responsible for storing all data items that have key 14. What node 0111 can simply do, is forward the request to a neighbor who is closer to node 1110. In this case, this is either node 0110 or node 1111. If it picks a node 0110, that

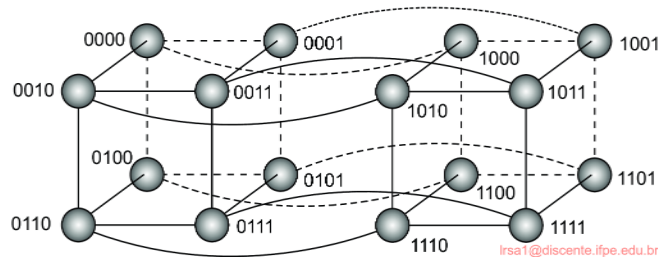


Figure 2.23: A simple peer-to-peer system organized as a four-dimensional hypercube.

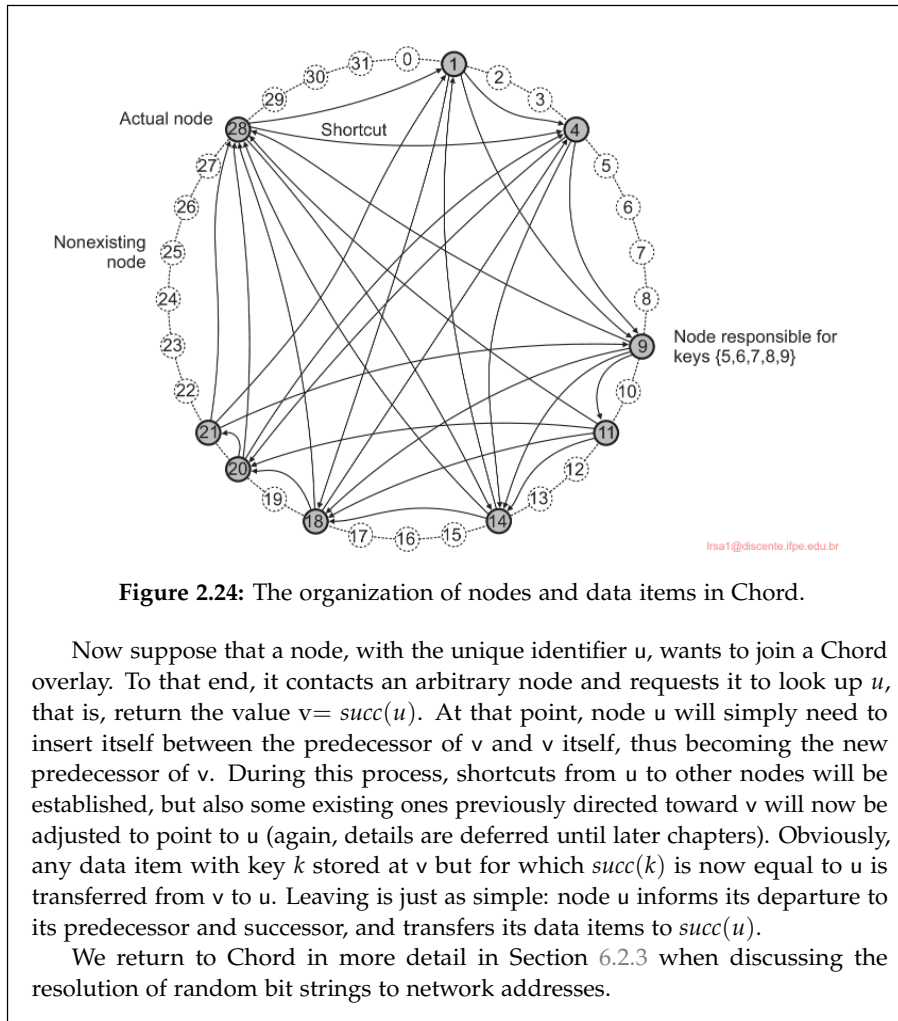
node will then forward the request directly to a node 1110 from where the data can be retrieved.

Note 2.6 (Example: The Chord system)

The previous example illustrates two things: (1) the use of a hashing function to identify the node responsible for storing some data item, and (2) the routing along the topology of a peer-to-peer system when looking up a data item given its key. However, it is not a very realistic example, if only for the reason that we assumed that the total set of nodes is fixed. Let us therefore consider a more realistic example of a structured peer-to-peer system that is considered as belonging to the foundations for many other such systems.

In the **Chord system** [Stoica et al., 2003] the nodes are logically organized in a ring such that a data item with an m -bit key k is mapped to the node with the smallest (again, also m bit) identifier $id \geq k$. This node is referred to as the **successor** of key k and denoted as $succ(k)$. Keys and identifiers are typically 128 or 160 bits long. Figure 2.24 shows a much smaller Chord ring, where $m = 5$ and with nine nodes $\{1, 4, 9, 11, 14, 18, 20, 21, 28\}$. The successor of key 7 is equal to 9. Likewise, $succ(5) = 9$, but also $succ(9) = 9$. In Chord, each node maintains shortcuts to other nodes. A shortcut appears as a directed edge from one node to another. How these shortcuts are constructed is explained in Chapter 6. The construction is done in such a way that the length of the shortest path between any pair of nodes is of order $\mathcal{O}(\log N)$, where N is the total number of nodes.

To look up a key, a node will try to forward the request “as far as possible,” but without passing it beyond the node responsible for that key. To clarify, suppose that in our example Chord ring, node 9 is asked to look up the node responsible for key 3 (which is node 4). Node 9 has four shortcuts: to nodes 11, 14, 18, and 28, respectively. As the node 28 is the farthest node 9 knows about and still preceding the one responsible for key 3, it will get the lookup request. Node 28 has three shortcuts: to nodes 1, 4, and 14, respectively. Note that node 28 has no knowledge about the existence of nodes between nodes 1 and 4. For this reason, the best what it can do is forward the request to the node 1. The latter knows that its successor in the ring is node 4, and thus that this is the node responsible for the key 3, to which it will subsequently forward the request.



2.4.2 Unstructured peer-to-peer systems

Structured peer-to-peer systems attempt to maintain a specific, deterministic overlay network. In contrast, in an **unstructured peer-to-peer system**, each node maintains an ad hoc list of neighbors. The resulting overlay resembles what is known as a **random graph**: a graph in which an edge $\langle u, v \rangle$ between two nodes u and v exists only with a certain probability $\mathbb{P}[\langle u, v \rangle]$. Ideally, this probability is the same for all pairs of nodes, but in practice a wide range of distributions is observed.

In an unstructured peer-to-peer system, when a node joins, it often contacts a well-known node to obtain a starting list of other peers in the system. This list can then be used to find more peers, and perhaps ignore others, and so

on. In practice, a node generally changes its local list almost continuously. For example, a node may discover that a neighbor is no longer responsive and that it needs to be replaced. There may be other reasons, which we will describe shortly.

Unlike structured peer-to-peer systems, looking up data cannot follow a predetermined route when lists of neighbors are constructed in an ad hoc fashion. Instead, in an unstructured peer-to-peer systems, we really need to resort to *searching* for data [Rissov and Moors, 2006]. Let us look at two extremes and consider the case in which we are requested to search for specific data (e.g., identified by keywords).

Flooding: In the case of **flooding**, an issuing node u simply passes a request for a data item to all its neighbors. A request will be ignored when its receiving node, say v , had seen it before. Otherwise, v searches locally for the requested data item. If v has the required data, it can either respond directly to the issuing node u , or send it back to the original forwarder, who will then return it to *its* original forwarder, and so on. If v does not have the requested data, it forwards the request to all of its own neighbors.

Obviously, flooding can be expensive, for which reason a request often has an associated **time-to-live** or **TTL** value, giving the maximum number of hops a request is allowed to be forwarded. Choosing the right TTL value is crucial: too small means that a request will stay close to the issuer and may thus not reach a node having the data. Too large incurs high communication costs.

As an alternative to setting TTL values, a node can also start a search with an initial TTL value of 1, meaning that it will first query only its neighbors. If no, or not enough results are returned, the TTL is increased, and a new search is initiated.

Random walks: At the other end of the search spectrum, an issuing node u can simply try to find a data item by asking a randomly chosen neighbor, say v . If v does not have the data, it forwards the request to one of its randomly chosen neighbors, and so on. The result is known as a **random walk** [Gkantsidis et al., 2006; Lv et al., 2002]. Obviously, a random walk imposes much less network traffic, yet it may take much longer before a node is reached that has the requested data. To decrease the waiting time, an issuer can simply start n random walks simultaneously. Indeed, studies show that in this case, the time it takes before reaching a node that has the data drops approximately by a factor n . Lv et al. [2002] reports that relatively small values of n , such as 16 or 64, turn out to be effective.

A random walk also needs to be stopped. To this end, we can either again use a TTL, or alternatively, when a node receives a lookup request, check

with the issuer whether forwarding the request to another randomly selected neighbor is still needed.

Note that neither method relies on a specific comparison technique to decide when requested data has been found. For structured peer-to-peer systems, we assumed the use of keys for comparison; for the two approaches just described, any comparison technique would suffice.

Between flooding and random walks lie **policy-based search methods**. For example, a node may decide to keep track of peers who responded positively, effectively turning them into preferred neighbors for succeeding queries. Likewise, we may want to restrict flooding to fewer neighbors, but in any case give preference to neighbors having many neighbors themselves.

Note 2.7 (Advanced: Flooding versus random walks)

When giving the matter some thought, it may come as a surprise that people have even considered a random walk as an alternative way to search. At first instance, it would seem like a technique resembling the search for a needle in a haystack. However, we need to realize that in practice we are dealing with *replicated* data, and even for minimal replication factors and different replication distributions, studies show that deploying random walks is not only effective, it can also be much more efficient in comparison to flooding.

To see why, we closely follow the model described in Lv et al. [2002] and Cohen and Shenker [2002]. Assume there are a total of N nodes and that each data item is replicated across r randomly chosen nodes. A search consists of repeatedly selecting a node at random until the item is found. If $\mathbb{P}[k]$ is the probability that the item is found after k attempts, we have

$$\mathbb{P}[k] = \frac{r}{N} \left(1 - \frac{r}{N}\right)^{k-1}.$$

Let the average search size S be the expected number of nodes that need to be probed before finding the requested data item:

$$S = \sum_{k=1}^N k \cdot \mathbb{P}[k] = \sum_{k=1}^N k \cdot \frac{r}{N} \left(1 - \frac{r}{N}\right)^{k-1} \approx N/r \text{ for } 1 \ll r \leq N.$$

By simply replicating every data item to each node, $S = 1$ and it is clear that a random walk will always outperform flooding even for TTL values of 1. More realistically, however, is to assume that r/N is relatively low, such as 0.1%, meaning that the average search size would be approximately 1000 nodes.

To compare this to flooding, assume that each node, on average, forwards a request to d randomly selected neighbors. After one step, the request will have arrived at d nodes, each of who will forward it to another $d - 1$ nodes (assuming that the node from where the request came is skipped), and so on. In other words, after k steps, and considering that a node can receive the request more than once, we will have reached (at most) the following number of nodes:

$$R(k) = d(d - 1)^{k-1}$$

Various studies show that $R(k)$ is a good estimate for the actual number of nodes reached, as long as we have only a few number of flooding steps. Of these nodes, we can expect a fraction of r/N to have the requested data item, meaning that when $\frac{r}{N} \cdot R(k) \geq 1$, we will most likely have found a node that has the data item.

To illustrate, let $r/N = 0.001 = 0.1\%$, which means that $S \approx 1000$. With flooding to, on average, $d = 10$ neighbors, we would require at least 4 flooding steps, reaching some 7290 nodes, which is considerably more than the 1000 nodes required when using a random walk. Only with $d = 33$ will we need to contact approximately also 1000 nodes in $k = 2$ flooding steps and having $r/N \cdot R(k) \geq 1$.

The obvious drawback of deploying random walks, is that it may take much longer before an answer is returned.

2.4.3 Hierarchically organized peer-to-peer networks

Notably in unstructured peer-to-peer systems, locating relevant data items can become problematic as the network grows. The reason for this scalability problem is simple: as there is no deterministic way of routing a lookup request to a specific data item, essentially the only technique a node can resort to is *searching* for the request by flooding or randomly walking through the network. As an alternative, many peer-to-peer systems have proposed to make use of special nodes that maintain an index of data items.

There are other situations in which abandoning the symmetric nature of peer-to-peer systems is sensible. Consider a collaboration of nodes that offer resources to each other. For example, in a collaborative **Content Delivery Network** (CDN), nodes may offer storage for hosting copies of Web documents allowing Web clients to access pages nearby, and thus to access them quickly. What is needed is a means to find out where documents can be stored best. In that case, making use of a **broker** that collects data on resource usage and availability for several nodes that are in each other's proximity allows selecting a node quickly with sufficient resources.

Nodes such as those maintaining an index or acting as a broker are generally referred to as **super peers**. As the name suggests, super peers are often also organized in a peer-to-peer network, leading to a hierarchical organization, as explained in Yang and Garcia-Molina [2003]. A simple example of such an organization is shown in Figure 2.25. In this organization, every regular peer, now referred to as a **weak peer**, is connected as a client to a super peer. All communication from and to a weak peer proceeds through that peer's associated super peer.

Often, the association between a weak peer and its super peer is fixed: whenever a weak peer joins the network, it attaches to one of the super peers and remains attached until it leaves the network. Obviously, it is expected that super peers are long-lived processes with high availability. To compensate for potential unstable behavior of a super peer, backup schemes can be deployed,

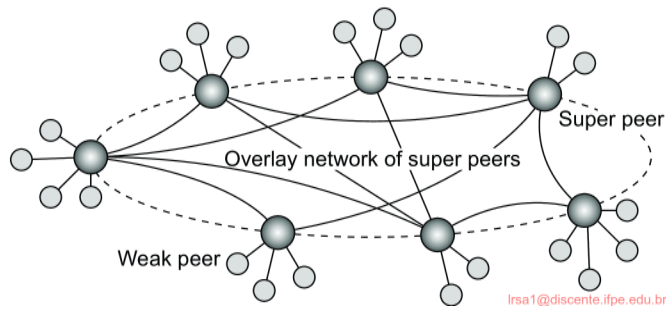


Figure 2.25: A hierarchical organization of nodes into a super-peer network.

such as pairing every super peer with another one and requiring weak peers to attach to both.

Having a fixed association with a super peer may not always be the best solution. For example, in the case of file-sharing networks, it may be better for a weak peer to attach to a super peer that maintains an index of files that the weak peer is currently interested in. In that case, chances are bigger that when a weak peer is looking for a specific file, its super peer will know where to find it. Garbacki et al. [2010] describe a relatively simple scheme in which the association between weak peer and strong peer can change as weak peers discover better super peers to associate with. In particular, a super peer returning the result of a lookup operation is given preference over other super peers.

As we have seen, peer-to-peer networks offer a flexible means for nodes to join and leave the network. However, with super-peer networks a new problem is introduced, namely how to select the nodes that are eligible to become super peer. This problem is closely related to the **leader-election problem**, which we discuss in Section 5.4.

2.4.4 Example: BitTorrent

Let us consider the widely popular **BitTorrent** file-sharing system [Cohen, 2003] as an example of a (largely) unstructured peer-to-peer system. BitTorrent is a file-downloading system. Its principle working is shown in Figure 2.26. The basic idea is that when an end user is looking for a file, she downloads chunks of the file from other users until the downloaded chunks can be assembled, yielding the complete file. An important design goal was to ensure collaboration. In most file-sharing systems, a significant fraction of participants merely download files but otherwise contribute close to nothing [Adar and Huberman, 2000; Saroiu et al., 2003; Yang et al., 2005], a phenomenon referred to as **free riding**. To prevent this situation, in BitTorrent a file can

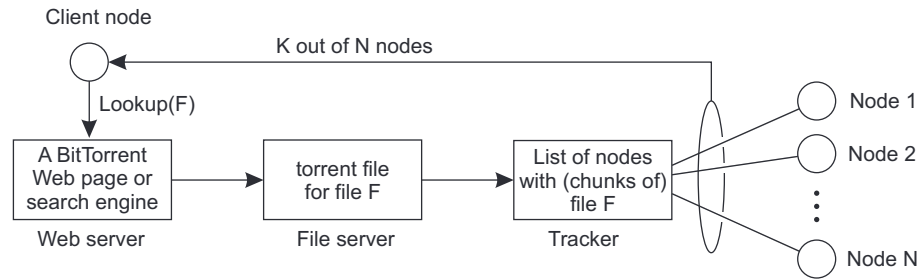


Figure 2.26: The principle working of BitTorrent [adapted with permission from Pouwelse et al. [2005].

be downloaded only when the downloading client is providing content to someone else.

To download a file, a user needs to access a global directory, which is generally just one of a few well-known Websites. Such a directory contains references to what are called torrent files. A **torrent file** contains the information that is needed to download a specific file. In particular, it contains a link to what is known as a **tracker**, which is a server that is keeping an accurate account of *active* nodes that have (chunks of) the requested file. An active node is one that is currently downloading the file as well. Obviously, there will be many trackers, although there will generally be only a single tracker per file (or collection of files).

Once the nodes have been identified from where chunks can be downloaded, the downloading node effectively becomes active. At that point, it will be forced to help others, for example by providing chunks of the file it is downloading that others do not yet have. This enforcement comes from a simple rule: if a node *P* notices that a node *Q* is downloading more than it is uploading, *P* can decide to decrease the rate at which it sends data to *Q*. This scheme works well, provided *P* has something to download from *Q*. For this reason, nodes are often supplied with references to many other nodes, putting them in a better position to trade data.

Clearly, BitTorrent combines centralized with decentralized solutions. As it turns out, the bottleneck of the system is easily formed by the trackers. In an alternative implementation of BitTorrent, a node also joins a separate structured peer-to-peer system (i.e., a DHT) to assist in tracking file downloads. In effect, a central tracker's load is now distributed across the participating nodes, with each node acting as a tracker for a relatively small set of torrent files. The original function of the tracker coordinating the collaborative downloading of a file is retained. However, we note that in many BitTorrent systems used today, the tracking functionality has actually been minimized to a one-time provisioning of peers currently involved in downloading the file. From that moment on, the newly participating peer will communicate only

with those peers and no longer with the initial tracker. The initial tracker for the requested file is looked up in the DHT through a so-called **magnet link**. We return to DHT-based lookups in Section 6.2.3.

2.5 Hybrid system architectures

Real-world distributed systems are complex in the sense that they combine a myriad of architectures: centralized features are combined with peer-to-peer features are combined with hierarchical organizations, etc. The complexity is aggravated by the fact that many distributed systems cross organizational boundaries, leading to truly decentralized solutions in which even no single organization can take responsibility for a system's operation. In this section, we will take a closer look into these complex, hybrid system architectures.

2.5.1 Cloud computing

Organizations in charge of running data centers have been seeking ways for opening up their resources to customers. Eventually, this led to the concept of **utility computing** by which a customer could upload tasks to a data center and be charged on a per-resource basis. Utility computing formed the basis for what is now commonly referred to as **cloud computing**.

Following Vaquero et al. [2008], cloud computing is characterized by an easily usable and accessible pool of *virtualized* resources. Which and how resources are used can be configured dynamically, providing the basis for scalability: if more work needs to be done, a customer can simply acquire more resources. The link to utility computing is formed by the fact that cloud computing is generally based on a pay-per-use model in which guarantees are offered by customized **service-level agreements (SLAs)**. Keeping it simple, clouds are organized into four layers, as shown in Figure 2.27.

Hardware: The lowest layer is formed by the means to manage the necessary hardware: processors, routers, but also power and cooling systems. It is generally implemented at data centers and contains the resources that customers normally never get to see directly.

Infrastructure: This is an important layer forming the backbone for most cloud computing platforms. It deploys virtualization techniques (discussed in Section 3.2) to provide customers an infrastructure consisting of virtual storage and computing resources. Indeed, nothing is what it seems: cloud computing evolves around allocating and managing virtual storage devices and virtual servers.

Platform: One could argue that the platform layer provides to a cloud-computing customer what an operating system provides to application developers, namely the means to easily develop and deploy applications

that need to run in a cloud. In practice, an application developer is offered a vendor-specific API, which includes calls to uploading and executing a program in that vendor's cloud. In a sense, this is comparable to the Unix `exec` family of system calls, which take an executable file as a parameter and pass it to the operating system to be executed.

Furthermore, like operating systems, the platform layer provides higher-level abstractions for storage and such. For example, as we discussed, the **Amazon S3 storage system** [Murty, 2008; Culkin and Zazon, 2022] is offered to the application developer in the form of an API allowing (locally created) files to be organized and stored in **buckets**. By storing a file in a bucket, that file is automatically uploaded to the Amazon cloud.

Application: Actual applications run in this layer and are offered to users for further customization. Well-known examples include those found in office suites (text processors, spreadsheet applications, presentation applications, and so on). It is important to realize that these applications are again executed in the vendor's cloud. As before, they can be compared to the traditional suite of applications that are shipped when installing an operating system.

Cloud-computing providers offer these layers to their customers through various interfaces (including command-line tools, programming interfaces, and Web interfaces), leading to three different types of services:

- **Infrastructure-as-a-Service (IaaS)** covering the hardware and infrastructure layer.
- **Platform-as-a-Service (PaaS)** covering the platform layer.
- **Software-as-a-Service (SaaS)** in which their applications are covered.

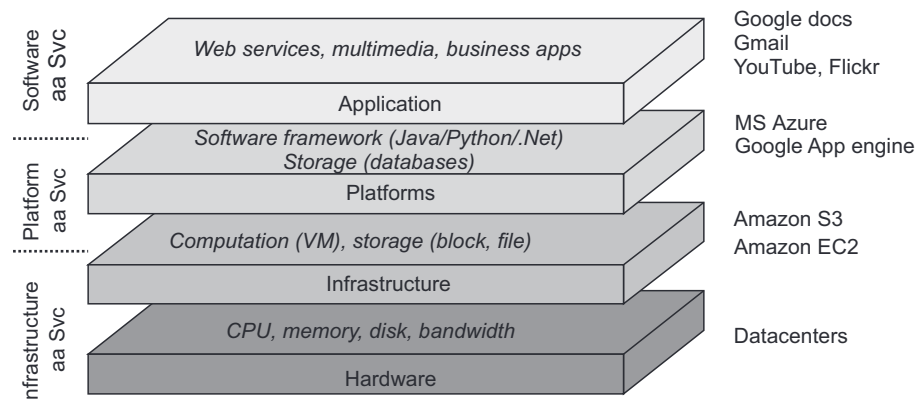


Figure 2.27: The organization of clouds (adapted from Zhang et al. [2010]).

As of now, making use of clouds is relatively easy, and we discuss in later chapters more concrete examples of interfaces to cloud providers. As a consequence, cloud computing as a means for outsourcing local computing infrastructures has become a serious option for many enterprises.

From the perspective of a system architecture, which deals with configuring (micro)services across some infrastructure, one may argue that in the case of cloud computing, we are dealing with a highly advanced client-server architecture. However, let it be noted that the actual implementation of a server is generally completely hidden from the client: it is often unclear *where* the server actually is, and even whether the server is actually implemented in a fully distributed manner (which it often is). To further illustrate this point, the notion of a **Function-as-a-Service**, or simply **FaaS**, allows a client to execute code without bothering even with starting a server to handle the code (see also Shahradi et al. [2019]).

2.5.2 The edge-cloud architecture

In the advent of increasingly more network-connected devices and the emergence of the **Internet-of-Things (IoT)** many became aware of the fact that we may need more than just cloud computing. **Edge computing** was born. There is a lot to say about edge computing, and much has already been said. And as is the case with so many topics in distributed systems, it simply takes a few years before things settle down a bit. In this section, we take a look at edge computing from an architectural perspective and will return to various elements throughout the book, often without even explicitly mentioning edge computing. An excellent overview of edge computing is given by Yousefpour et al. [2019] and the interested reader is referred to that paper to get a better grip on its nomenclature. We deliberately take a simplified and broad view of edge computing, using it as a general term for most of the things sitting between the devices that comprise the Internet-of-Things and the services typically offered through cloud computing. In this sense, we follow the discussion as presented by Horner [2021].

As its name suggests, edge computing deals with the placement of services “at the edge” of the network. This edge is often formed by the boundary between enterprise networks and the actual Internet, for example, as provided by an **Internet Service Provider (ISP)**. For example, many universities reside on a campus consisting of various buildings, each having their own local network, in turn connected through a campuswide network. As part of the campus, there may be multiple on-premise services for storage, computing, security, lectures, and so on. On-premise means that the local IT department is responsible for hosting those services on servers directly hooked up to the campus network. Much of the traffic related to those services will never leave the campus network, and the network together with its servers and services form a typical **edge infrastructure**.

At the same time, such servers may be connected to those of other universities and perhaps even making use of, again, other servers. In other words, instead of setting up connections between universities in a peer-to-peer fashion, we also see configurations in which various universities share services through a logically centralized infrastructure. Such an infrastructure may be situated “in the cloud,” but it may equally well have been set up through a regional infrastructure using locally available data centers. As we move closer to cloud infrastructures, the term **fog computing** is often used. We thus see an overall picture emerge as the one shown in Figure 2.28, where the boundaries between cloud and edge are becoming blurred.

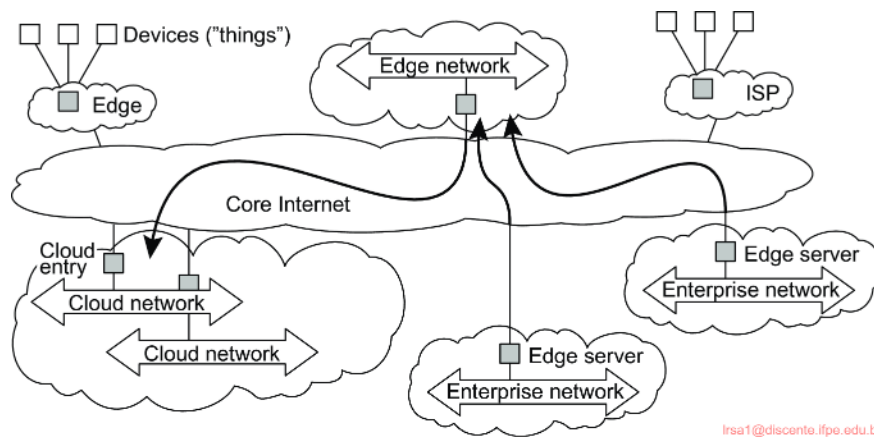


Figure 2.28: A collection of infrastructures involving edge devices, edge infrastructures and cloud infrastructures, and a possible setup between two enterprise edge infrastructures, an intermediate edge infrastructure, and a cloud infrastructure.

Many configurations for an edge infrastructure easily come to mind, ranging from infrastructures needed to keep track of your activities, layered video-streaming infrastructures, gaming infrastructures, etc. What all of these have in common is that there is some smart end device that one way or the other needs to (eventually) connect to a service hosted somewhere in the cloud. The question then pops up why an edge infrastructure is needed at all. Logically, it seems much simpler to just connect to the cloud service directly using existing and often excellent networking facilities. Let us take a critical look at a few arguments.

Latency and bandwidth What should have become clear from our examples is that edge infrastructures are considered to be close to the end devices. Closeness can be measured in terms of latency and often also bandwidth. Throughout the decades, bandwidth, or actually lack of bandwidth, has

always been used as an argument for introducing solutions close to specific devices. However, if anything has become clear all this time, is that available bandwidth continues to increase, now reaching the point that one should seriously question how problematic it actually is, and whether installing and maintaining edge infrastructures for having insufficient bandwidth is a good reason. Nevertheless, there are situations in which closeness to end devices is actually needed to guarantee quality of service. The canonical example is formed by video services: the closer the video sources are, the better bandwidth guarantees can be given, reducing issues such as jitter.

More problematic is when Mother Nature gets in our way. This may easily happen when dealing with latency. It may take 100 ms to reach a cloud, rendering many interactive applications quite useless. One such important application is (semi-)autonomous driving. A car will need to continuously observe its environment through a myriad of sensors and react accordingly. Having to coordinate its movements through the cloud is not acceptable from a real-time aspect alone. This example also illustrates that cars may need to detect each other beyond the capabilities of their sensors, for example, when heading toward a junction with clear visibility. In a real-time system, cars may be able to provide their current position to a local edge infrastructure and reveal themselves to each other when approaching the junction.

Other examples in which latency plays a crucial role easily come to mind. Overcoming latency is one of the most compelling reasons for developing edge infrastructures.

Reliability Many argue that cloud connectivity is simply not reliable enough for many applications, for which reason edge infrastructures should be deployed. To what extent this is a valid argument remains to be seen. The fact is that for many networked applications, connectivity is generally good and reliable, if not excellent. Of course, there are situations in which relying on 24/7 reliability is not an option. This may be the case for hospitals, factories, and other critical settings in general. Yet, in those cases, measures have been traditionally already taken and to what extent edge computing brings in anything new is not always clear.

Security and privacy Finally, many argue that edge solutions enhance security and privacy. It all depends. One could argue that if a cloud solution is not secure, then there is no reason why an edge solution would be. An implicit assumption that many people make is that an edge infrastructure is owned by a specific organization and operates within the (protected) network boundaries of that organization. In that case, it often does indeed become simpler to protect data and operations, yet one should ask whether such protection is sufficient. As we shall discuss in Chapter 9, simply trying to protect assets by developing a secure wall around an organization does not help against

insider attacks, whether they are intended or not. A same reasoning holds for privacy: if we cannot protect personal data in the cloud, then why would an edge infrastructure suffice for privacy? A thorough discussion on the role and position of privacy in the edge and edge devices is given by Hong [2017]. From that discussion, it is clear that there is still considerable work to be done.

However, there may be another reason related to security and privacy why edge infrastructures are needed. In many cases, organizations are simply not allowed, for whatever regulatory reasons, to place data in the cloud or have data be processed by a cloud service. For example, medical records may have to be kept on premise on certified servers and with strict audit procedures in place. In this case, an organization will have to resort to maintaining an edge infrastructure.

Introducing additional layers between end devices and cloud infrastructures opens a whole can of worms compared to the relatively simple situation of just having to deal with cloud computing. For the latter, one can argue that the cloud provider to a considerable extent decides where and how a service is actually implemented. In practice, we will be dealing with a data center in which the (micro)services that make up the entire service are distributed across multiple machines.

Matters become more intricate in the case of edge computing. In this case, the client organization will now have to make informed decisions on what to do where. Which services need to be placed on premise on a local edge infrastructure, and which can be moved to the cloud? To what extent does an edge infrastructure offer facilities for virtual resources, akin to the facilities offered in cloud computing? Moreover, where we may be able to assume that computational and storage resources are in abundance when dealing with a cloud, this is not necessarily the case for an edge infrastructure. In practice, the latter simply have less hardware resources available, but often also offer less flexibility in terms of available platforms.

By and large, allocating resources in the case of edge computing appears to be much more challenging in comparison to clouds. As summarized by Hong and Varghese [2019], we are dealing with limitations when it comes to resources, higher degrees of hardware heterogeneity, and much more dynamic workloads, which, when taken together, have led to a higher demand of **orchestration**. Moreover, where from a client's perspective the cloud appears to be hiding many of its internal intricacies, this is necessarily no longer the case, making it much more difficult to do the orchestration [Bittencourt et al., 2018]. Orchestration boils down to the following (see also Taleb et al. [2017]):

- **Resource allocation:** specific services require specific resources. The question is then to guarantee the availability of the resources required to perform a service. Typically, resources amount to CPU, storage, memory, and networking facilities.

- **Service placement:** regardless the availability of resources, it is important to decide *when* and *where* to place a service. This is notably relevant for mobile applications, for in that case finding the edge infrastructure that is closest to that application may be crucial. A typical use case is that of video conferencing, for which the encoding is often not done on the mobile device, but at an edge infrastructure. In practice, one needs to decide at which edges the service should be installed. An extensive overview of service placement in the case of edge computing is provided by Salaht et al. [2020].
- **Edge selection:** related to service placement is deciding which edge infrastructure should be used when the service needs to be offered. It may seem logical to use the edge infrastructure closest to the end device, but all kinds of circumstances may ask for an alternative solution, for example the connectivity of that edge to the cloud provider.

Other issues play a role as well, but it should be clear by now that the edge-cloud architecture is much more demanding than one might initially think it to be. Moreover, the different perspectives on how the continuum between end devices and the cloud should be filled with edge components and solutions has still to converge [Antonini et al., 2019].

2.5.3 Blockchain architectures

An upcoming and much debated type of distributed system is that of so-called **blockchains**. Blockchain systems enable the registration of transactions, for which reason they are also referred to as **distributed ledgers**. The latter is actually more accurate, with blockchains forming one of different ways for implementing distributed ledgers.

The key issue in transaction systems is that a transaction is validated, effectuated, and subsequently stored for various auditing purposes. For example, Alice may decide to create a transaction stating that she transfers \$10 to Bob's account. Normally, she would go to a bank, where she would have to sign the transaction to prove that she really wants it to be carried out. Whether this all happens physically or digitally does not really matter. The bank will check whether she has enough credit, whether Bob is eligible for receiving the money, and assuming everything is fine, will subsequently transfer the money. A record of the transaction is kept for all kinds of auditing purposes. Note that transactions in blockchains systems are taken very broad. Besides monetary transactions, systems have been developed for registering identification documents, registering resource usage and allocation, electronic voting, and sharing health records, to name a few.

The bank operates as a **trusted third party**. An important design assumption for blockchains is that participating parties can, in principle, not be trusted. This also excludes having a trusted third party that handles all

transactions. We will return to trust in Section 9.4, and concentrate on the implications lack of trust has for the architecture of a system.

In the case of blockchains, we assume there is a (potentially very large) set of participants who jointly register transactions among them in a publicly available ledger. In this way, any participant can see what has happened and also verify the validity of a transaction. For example, in a blockchain system for digital coins, each having a unique and unforgeable ID, any participant should be able to check whether a coin has already been spent by checking all transactions that have taken place since the beginning.

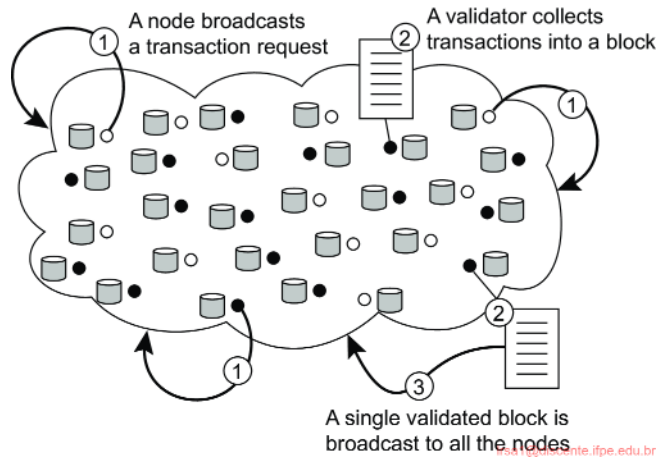


Figure 2.29: The principle operation of a blockchain.

To this end, when Alice wants to transfer \$10 to Bob, she essentially tells all the participants in the blockchain system about this intent, thus allowing volunteers to validate the intended transaction. This is shown as Step 1 in Figure 2.29. To avoid having to check every transaction separately one-by-one as they are submitted, a validator groups several transactions into a block to increase efficiency, shown as Step 2 in Figure 2.29. If everything goes well, i.e., the transactions in the block are considered to be valid, the validator securely protects the block against any modifications, and appends the now immutable block to the chain of other blocks with validated transactions. It does so by broadcasting the validated block to all participants, shown as Step 3 in Figure 2.29.

An important observation is that there is logically only a single chain of blocks with validated transactions. Each block is immutable, in the sense that if an adversary decides to modify any transaction from any block in that chain, the modification can never go unnoticed. Securely protecting blocks of transactions against modifications, but also securely appending a block to an existing list, are well-understood techniques, which we will explain further

in Section 9.4.3. The immutability of a block makes it an ideal fit for massive replication: it will never be changed anyway, so someone may just as well store it locally to make verification as simple as possible. Effectively, the logically single chain of immutable blocks may be physically massively replicated across the Internet among all participating parties. This is precisely what happens: each block is broadcast to every participating node in a blockchain, as we just explained.

What differentiates so many blockchain systems from each other is deciding on which node(s) may actually carry out validation tasks. In other words, we need to figure out who is allowed to append a block of validated transactions to the existing chain. Appending such a block means that there is global consensus on fulfilled transactions. It is therefore important that we also reach consensus on which validator can move ahead. All others will have to do their validation over again, as their transactions may be affected by the newly appended ones and thus may need to be revisited.

Deciding on which validator can move ahead requires **(distributed) consensus**. In principle, there are three options:

1. A centralized solution, in which a trusted third party validates transactions as before.
2. A distributed solution, in which a small, preselected group of processes takes over the role of a trusted third party.
3. A fully decentralized solution, in which, in principle, all participating nodes in the blockchain jointly reach consensus without any (distributed) third party.

These three options are shown in Figure 2.30. As mentioned, each node participating in the blockchain is assumed to have a full copy locally available.

Obviously, a centralized architecture for a blockchain does not fit its design goals, which state that there is essentially no place for a trusted third party. The distributed architecture is an interesting one. In this case, there is a relatively small group of nodes that are *permissioned* to validate transactions. For blockchains, it is important to realize that none of these permissioned nodes are assumed to be trusted, yet they are assumed to run a consensus protocol that can withstand malicious behavior. Specifically, if there are n permissioned nodes, then it is assumed that at most $k \leq (n - 1)/3$ will fail and perhaps act maliciously. One problem with such so-called **permissioned blockchains** is that n is quite limited, in practice, to less than a few tens of nodes.

Finally, in so-called **permissionless blockchains** all nodes collectively participate to validate transactions. In practice, this means that all nodes who want to validate transactions are engaged in a process called **leader election**. The process elected as leader appends a block to the current chain (and is

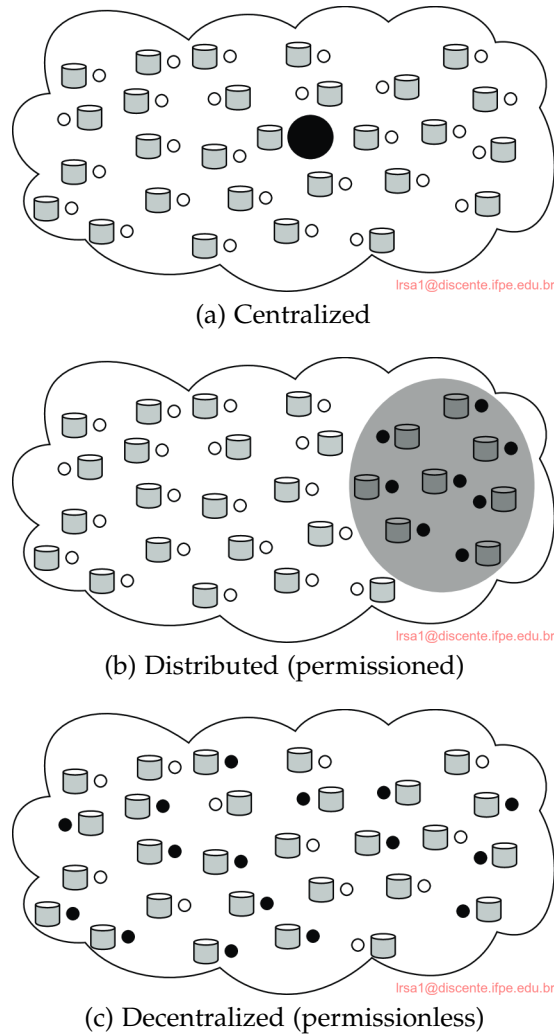


Figure 2.30: The three different organizations of blockchains: (a) centralized, (b) distributed, (c) fully decentralized. Filled nodes represent validators; other nodes are participants not engaged in validation.

often rewarded for that). In practice, not all participating nodes will want to act as validator, if only because the leader-election algorithm is costly in terms of resources. We return to leader elections in Section 5.4.

The architecture of a blockchain system is thus seen to be quite complex. In a permissioned system, we have a few tens of nodes for validating transactions. None of these nodes needs to be individually trusted beforehand, yet it may be argued that they form a centralized, fault-tolerant distributed group. Trust

is needed in so far that we need to assume that not too many of those nodes act maliciously or collude against any decisions they should make. On the other hand, permissionless blockchains may be viewed as being fully decentralized, but here we see that special measures are needed to guarantee some form of fairness among willing validators. In fact, through the dynamics of permissionless blockchains, we often see that, in practice, only a relatively few number of nodes can carry out validation tasks, effectively also leading to a more centralized system. An overview of the various settings in blockchains from the perspective of architectures is given by Xu et al. [2017].

2.6 Summary

Distributed systems can be organized in many ways. We can make a distinction between software architecture and system architecture. The latter considers where the components that constitute a distributed system are placed across the various machines. The former is more concerned about the logical organization of the software: how do components interact, in what ways can they be structured, how can they be made independent, and so on?

A keyword when talking about architectures is architectural style. A style reflects the basic principle that is followed in organizing the interaction between the software components comprising a distributed system. Important styles include layering, service-oriented styles, and styles in which handling events are prominent, exemplified by are known as publish-subscribe styles.

There are many organizations of distributed systems. An important class is where machines are divided into clients and servers. A client sends a request to a server, who will then produce a result that is returned to the client. The client-server architecture reflects the traditional way of modularizing software, in which a module calls the functions available in another module. By placing different components on different machines, we obtain a natural physical distribution of functions across a collection of machines.

Client-server architectures are often highly centralized. In decentralized architectures, we often see an equal role played by the processes that constitute a distributed system, also known as peer-to-peer systems. In peer-to-peer systems, the processes are organized into an overlay network, which is a logical network in which every process has a local list of other peers that it can communicate with. The overlay network can be structured, in which case deterministic schemes can be deployed for routing messages between processes. In unstructured networks, the list of peers is more or less random, implying that search algorithms need to be deployed for locating data or other processes.

In hybrid architectures, elements from centralized and decentralized organizations are combined. A typical example is that of cloud computing, which logically follows a client-server architecture, but where the server is generally

completely distributed across a data center. In the last decade, we have seen a strong emergence of what is known as edge computing. Edge infrastructures form several steps between end devices and clouds and are demanding from the view point of organizing and configuring distributed systems. Finally, as an example in which decentralization plays a prominent role, the increasing popular blockchain architecture illustrates yet another class of hybrid system architectures.

Interprocess communication is at the heart of all distributed systems. It makes no sense to study distributed systems without carefully examining the ways that processes on different machines can exchange information. Communication in distributed systems has traditionally always been based on low-level message passing as offered by the underlying network. Expressing communication through message passing is more difficult than using primitives based on shared memory, as available for nondistributed platforms. Modern distributed systems often consist of thousands or even millions of processes scattered across a network with unreliable communication, such as the Internet. Unless the primitive communication facilities of computer networks are replaced by something else, development of large-scale distributed applications is extremely difficult.

In this chapter, we start by discussing the rules that communicating processes must adhere to, known as protocols, and concentrate on structuring those protocols in the form of layers. We then look at two widely used models for communication: Remote Procedure Call (RPC), and Message-Oriented Middleware (MOM). We also discuss the general problem of sending data to multiple receivers, called multicasting.

Our first model for communication in distributed systems is the remote procedure call (RPC). An RPC aims at hiding most of the intricacies of message passing, and is ideal for client-server applications. However, realizing RPCs transparently is easier said than done. We look at several important details that cannot be ignored, while diving into actual code to illustrate to what extent distribution transparency can be realized such that performance is still acceptable.

In many distributed applications, communication does not follow the rather strict pattern of client-server interaction. In those cases, it turns out that thinking in terms of messages is more appropriate. The low-level communication facilities of computer networks are in many ways not suitable, again due to their lack of distribution transparency. An alternative is to use a high-level message-queuing model, in which communication proceeds much the same as in e-mail systems. Message-oriented communication is a subject important enough to warrant a section of its own. We look at numerous aspects, including application-level routing.

Finally, since our understanding of setting up multicast facilities has improved, novel and elegant solutions for data dissemination have emerged. We pay separate attention to this subject in the last section of this chapter, discussing traditional deterministic means of multicasting, as well as probabilistic approaches as used in flooding and gossiping. The latter have been receiving much attention over the past years due to their elegance, reliability, and simplicity.

4.1 Foundations

Before we start our discussion on communication in distributed systems, we first recapitulate some fundamental issues related to communication. In the next section, we briefly discuss network communication protocols, as these form the basis for any distributed system. Thereafter, we take a different approach by classifying the different types of communication that usually occur in distributed systems.

4.1.1 Layered Protocols

Due to the absence of shared memory, all communication in distributed systems is based on sending and receiving (low level) messages. When process P wants to communicate with process Q, it first builds a message in its own address space. Then it executes a system call that causes the operating system to send the message over the network to Q. Although this basic idea sounds simple enough, to prevent chaos, P and Q have to agree on the meaning of the bits being sent.

The OSI reference model

To make it easier to deal with the numerous levels and issues involved in communication, the International Standards Organization (ISO) developed a reference model that clearly identifies the various levels involved, gives them standard names, and points out which level should do which job. This model is called the **Open Systems Interconnection Reference Model** [Day and Zimmerman, 1983] usually abbreviated as **ISO OSI** or sometimes just the **OSI model**. It should be emphasized that the protocols that were developed as part of the OSI model were never widely used and are essentially dead. However, the underlying model itself has proved to be quite useful for understanding computer networks. Although we do not intend to give a full description of this model and all of its implications here, a brief introduction will be helpful. For more details, see [Tanenbaum et al., 2021].

The OSI model is designed to allow open systems to communicate. An open system is one that is prepared to communicate with any other open system by using standard rules that govern the format, contents, and meaning of the messages sent and received. These rules are formalized in what are called **communication protocols**. To allow a group of computers to communicate over a network, they must all agree on the protocols to be used. A protocol is said to provide a **communication service**. There are two types of such services. In the case of a **connection-oriented service**, before exchanging data the sender and receiver first explicitly establish a connection, and possibly negotiate specific parameters of the protocol they will use. When they are done, they release (terminate) the connection. The telephone is a

typical connection-oriented communication service. With a **connectionless services**, no setup in advance is needed. The sender just transmits the first message when it is ready. Dropping a letter in a mailbox is an example of making use of connectionless communication service. With computers, both connection-oriented and connectionless communication are common.

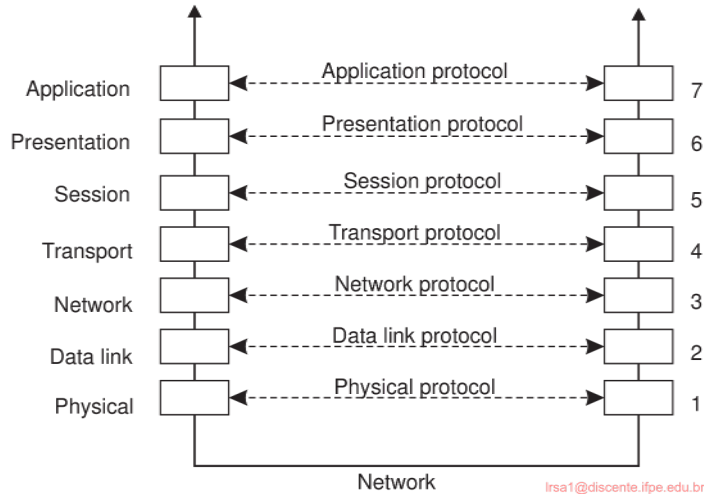


Figure 4.1: Layers, interfaces, and protocols in the OSI model.

In the OSI model, communication is divided into seven levels or layers, as shown in Figure 4.1. Each layer offers one or more specific communication services to the layer above it. In this way, the problem of getting a message from A to B can be divided into manageable pieces, each of which can be solved independently of the others. Each layer provides an **interface** to the one above it. The interface consists of a set of operations that together define the service the layer is prepared to offer. The seven OSI layers are:

Physical layer Deals with standardizing how two computers are connected and how 0s and 1s are represented.

Data link layer Provides the means to detect and possibly correct transmission errors, as well as protocols to keep a sender and receiver in the same pace.

Network layer Contains the protocols for routing a message through a computer network, as well as protocols for handling congestion.

Transport layer Mainly contains protocols for directly supporting applications, such as those that establish reliable communication, or support real-time streaming of data.

Session layer Provides support for sessions between applications.

Presentation layer Prescribes how data is represented in a way that is independent of the hosts on which communicating applications are running.

Application layer Essentially, everything else: e-mail protocols, Web access protocols, file-transfer protocols, and so on.

When a process P wants to communicate with some remote process Q, it builds a message and passes that message to the application layer as offered to it through an interface. This interface will typically appear in the form of a library procedure. The application layer software then adds a *header* to the front of the message and passes the resulting message across the layer 6/7 interface to the presentation layer. The presentation layer, in turn, adds its own header and passes the result down to the session layer, and so on. Some layers add not only a header to the front, but also a trailer to the end. When it hits the bottom, the physical layer actually transmits the message (which by now might look as shown in Figure 4.2) by putting it onto the physical transmission medium.

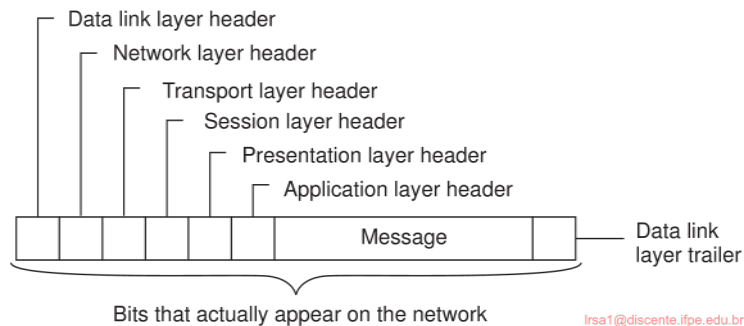


Figure 4.2: A typical message as it appears on the network.

When the message arrives at the remote machine hosting Q, it is passed upward, with each layer stripping off and examining its own header. Finally, the message arrives at the receiver, process Q, which may reply to it using the reverse path. The information in the layer-*n* header is used for the layer-*n* protocol.

In the OSI model, there are not two layers, but seven, as we saw in Figure 4.1. The collection of protocols used in a particular system is called a **protocol suite** or **protocol stack**. It is important to distinguish a *reference model* from its actual *protocols*. As said, the OSI protocols were never popular, in contrast to protocols developed for the Internet, such as TCP and IP.

Note 4.1 (More information: Protocols in the OSI model)

Let us briefly examine each of the OSI layers in turn, starting at the bottom. Instead of giving examples of OSI protocols, where appropriate, we will point out some Internet protocols used in each layer.

Lower-level protocols The three lowest layers of the OSI protocol suite implement the basic functions that encompass a computer network.

The **physical layer** is concerned with transmitting the 0s and 1s. How many volts to use for 0 and 1, how many bits per second can be sent, and whether transmission can take place in both directions simultaneously are key issues in the physical layer. In addition, the size and shape of the network connector (plug), as well as the number of pins and meaning of each, are of concern here.

The physical layer protocol deals with standardizing the electrical, optical, mechanical, and signaling interfaces so that when one machine sends a 0 bit it is actually received as a 0 bit and not a 1 bit. Many physical layer standards have been developed (for different media), for example, the USB standard for serial communication lines.

The physical layer just sends bits. As long as no errors occur, all is well. However, real communication networks are subject to errors, so some mechanism is needed to detect and correct them. This mechanism is the main task of the **data link layer**. What it does is to group the bits into units, also called **frames**, and see that each frame is correctly received.

The data link layer does its work by putting a special bit pattern on the start and end of each frame to mark them, as well as computing a **checksum** by adding up all the bytes in the frame in a certain way. The data link layer appends the checksum to the frame. When the frame arrives, the receiver recomputes the checksum from the data and compares the result to the checksum following the frame. If the two agree, the frame is considered correct and is accepted. If they disagree, the receiver asks the sender to retransmit it. Frames are assigned sequence numbers (in the header), so everyone can tell which is which.

On a LAN, there is usually no need for the sender to locate the receiver. It just puts the message out on the network and the receiver takes it off. A wide-area network, however, consists of numerous machines, each with some number of lines to other machines, rather like a large-scale map showing major cities and roads connecting them. For a message to get from the sender to the receiver it may have to make a number of hops, at each one choosing an outgoing line to use. The question of how to choose the best path is called **routing**, and is essentially the primary task of the **network layer**.

The problem is complicated by the fact that the shortest route is not always the best route. What really matters is the amount of delay on a given route, which, in turn, is related to the amount of traffic and the number of messages queued up for transmission over the various lines. The delay can thus change over the course of time. Some routing algorithms try to adapt to changing loads, whereas others are content to make decisions based on long-term averages.

At present, the most widely used network protocol is the connectionless **IP (Internet Protocol)**, which is part of the Internet protocol suite. An **IP packet** (the technical term for a message in the network layer) can be sent without any setup. Each IP packet is routed to its destination independent of all others. No internal path is selected and remembered.

Transport protocols The **transport layer** forms the last part of what could be called a basic network protocol stack, in the sense that it implements all those services that are not provided at the interface of the network layer, but which are reasonably needed to build network applications. In other words, the transport layer turns the underlying network into something that an application developer can use.

Packets can be lost on the way from the sender to the receiver. Although some applications can handle their own error recovery, others prefer a reliable connection. The job of the transport layer is to provide this service. The idea is that the application layer should be able to deliver a message to the transport layer with the expectation that it will be delivered without loss.

Upon receiving a message from the application layer, the transport layer breaks it into pieces small enough for transmission, assigns each one a sequence number, and then sends them all. The discussion in the transport layer header concerns which packets have been sent, which have been received, how many more the receiver has room to accept, which should be retransmitted, and similar topics.

Reliable transport connections (which by definition are connection-oriented) can be built on top of connection-oriented or connectionless network services. In the former case all the packets will arrive in the correct sequence (if they arrive at all), but in the latter case it is possible for one packet to take a different route and arrive earlier than the packet sent before it. It is up to the transport layer software to put everything back to maintain the illusion that a transport connection is like a big tube—you put messages into it, and they come out undamaged and in the same order in which they went in. Providing this end-to-end communication behavior is an important aspect of the transport layer.

The Internet transport protocol is called **TCP (Transmission Control Protocol)** and is described in detail by Comer [2013]. The combination TCP/IP is now used as a de facto standard for network communication. The Internet protocol suite also supports a connectionless transport protocol called **UDP (Universal Datagram Protocol)**, which is essentially just IP with some minor additions. User programs that do not need a connection-oriented protocol normally use UDP.

Additional transport protocols are regularly proposed. For example, to support real-time data transfer, the **Real-time Transport Protocol (RTP)** has been defined. RTP is a framework protocol in the sense that it specifies packet formats for real-time data without providing the actual mechanisms for guaranteeing data delivery. In addition, it specifies a protocol for monitoring and controlling data transfer of RTP packets [Schulzrinne et al., 2003]. Likewise, the **Streaming Control Transmission Protocol (SCTP)** has been proposed as an alternative to

TCP [Stewart, 2007]. The main difference between SCTP and TCP is that SCTP groups data into messages, whereas TCP merely moves bytes between processes. Doing so may simplify application development.

Higher-level protocols Above the transport layer, OSI distinguishes three additional layers. In practice, only the **application layer** is ever used. In fact, in the Internet protocol suite, everything above the transport layer is grouped together. In the face of middleware systems, we shall see that neither the OSI nor the Internet approach is really appropriate.

The **session layer** is essentially an enhanced version of the transport layer. It provides dialog control, to keep track of which party is currently talking, and it provides synchronization facilities. The latter are useful to allow users to insert checkpoints into long transfers, so that in the event of a crash, it is necessary to go back only to the last checkpoint, rather than all the way back to the beginning. In practice, few applications are interested in the session layer, and it is rarely supported. It is not even present in the Internet protocol suite. However, in the context of developing middleware solutions, the concept of a session and its related protocols has turned out to be quite relevant, notably when defining higher-level communication protocols.

Unlike the lower layers, which are concerned with getting the bits from the sender to the receiver reliably and efficiently, the **presentation layer** is concerned with the meaning of the bits. Most messages do not consist of random bit strings, but more structured information such as people's names, addresses, amounts of money, and so on. In the presentation layer, it is possible to define records containing fields like these and then have the sender notify the receiver that a message contains a particular record in a certain format. This makes it easier for machines with different internal representations to communicate with each other.

The OSI application layer was originally intended to contain a collection of standard network applications such as those for electronic mail, file transfer, and terminal emulation. By now, it has become the container for all applications and protocols that in one way or the other do not fit into one of the underlying layers. From the perspective of the OSI reference model, virtually all distributed systems are just applications.

What is missing in this model is a clear distinction between applications, application-specific protocols, and general-purpose protocols. For example, the Internet **File Transfer Protocol (FTP)** [Postel and Reynolds, 1985; Horowitz and Lunt, 1997] defines a protocol for transferring files between a client and server machine. The protocol should not be confused with the ftp program, which is an end-user application for transferring files and which also (not entirely coincidentally) happens to implement the Internet FTP.

Another example of a typical application-specific protocol is the **HyperText Transfer Protocol (HTTP)** [Fielding and Reschke, 2014] which is designed to remotely manage and handle the transfer of Web pages. The protocol is implemented by applications such as Web browsers and Web servers. However, HTTP is now also used by systems that are not intrinsically tied to the Web. For example,

Java's object-invocation mechanism can use HTTP to request the invocation of remote objects that are protected by a firewall.

There are also many general-purpose protocols that are useful to many applications, but which cannot be qualified as transport protocols. Often, such protocols fall into the category of middleware protocols.

Middleware protocols

Middleware is an application that logically lives (mostly) in the OSI application layer, but which contains many general-purpose protocols that warrant their own layers, independent of other, more specific applications. Let us briefly look at some examples.

The **Domain Name System (DNS)** [Liu and Albitz, 2006] is a distributed service that is used to look up a network address associated with a name, such as the address of a so-called **domain name** like `www.distributed-systems.net`. In terms of the OSI reference model, DNS is an application and therefore is logically placed in the application layer. However, it should be quite obvious that DNS is offering a general-purpose, application-independent service. Arguably, it forms part of the middleware.

As another example, there are various ways to establish **authentication**, that is, provide proof of a claimed identity. Authentication protocols are not closely tied to any specific application, but instead, can be integrated into a middleware system as a general service. Likewise, **authorization protocols** by which authenticated users and processes are granted access only to those resources for which they have authorization, tend to have a general, application-independent nature. Being labeled as applications in the OSI reference model, these are clear examples that belong in the middleware.

Distributed commit protocols establish that in a group of processes, possibly spread out across several machines, either all processes carry out a particular operation, or that the operation is not carried out at all. This phenomenon is also referred to as **atomicity** and is widely applied in transactions. As it turns out, commit protocols can present an interface independently of specific applications, thus providing a general-purpose transaction service. In such a form, they typically belong to the middleware and not to the OSI application layer.

As a last example, consider a **distributed locking protocol**, by which a resource can be protected against simultaneous access by a collection of processes that are distributed across multiple machines. It is not hard to imagine that such protocols can be designed in an application-independent fashion, and accessible through a relatively simple, again application-independent interface. As such, they generally belong in the middleware.

These protocol examples are not directly tied to communication, yet there

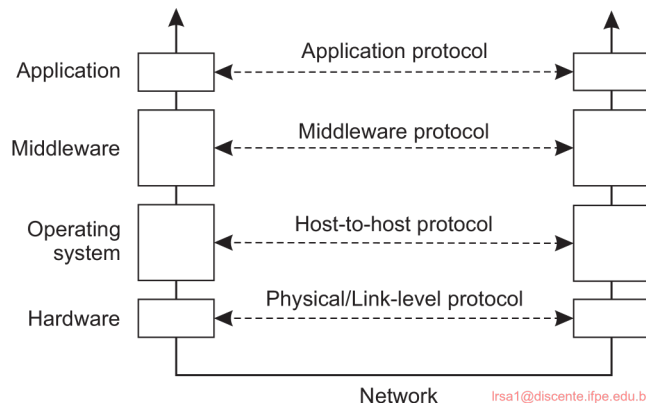


Figure 4.3: An adapted reference model for networked communication.

are also many middleware communication protocols. For example, with a so-called **remote procedure call**, a process is offered a facility to *locally* call a procedure that is effectively implemented on a *remote* machine. This communication service belongs to one of the oldest types of middleware services and is used for realizing access transparency. In a similar vein, there are high-level communication services for setting and synchronizing streams for transferring real-time data, such as needed for multimedia applications. As a last example, some middleware systems offer reliable multicast services that scale to thousands of receivers spread across a wide-area network.

Taking this approach to layering leads to the adapted and simplified reference model for communication, as shown in Figure 4.3. Compared to the OSI model, the session and presentation layer have been replaced by a single middleware layer that contains application-independent protocols. These protocols do not belong in the lower layers we just discussed. Network and transport services have been grouped into communication services as normally offered by an operating system, which, in turn, manages the specific lowest-level hardware used to establish communication.

4.1.2 Types of Communication

In the remainder of this chapter, we concentrate on high-level middleware communication services. Before doing so, there are other general criteria for distinguishing (middleware) communication. To understand the various alternatives in communication that middleware can offer to applications, we view the middleware as an additional service in client-server computing, as shown in Figure 4.4. Consider, for example, an e-mail system. In principle, the core of the mail delivery system can be seen as a middleware communication service. Each host runs a user agent allowing users to compose, send, and receive e-mail. A sending user agent passes such mail to the mail delivery

system, expecting it, in turn, to eventually deliver the mail to the intended recipient. Likewise, the user agent at the receiver's side connects to the mail delivery system to see whether any mail has come in. If so, the messages are transferred to the user agent so that they can be read by the user.

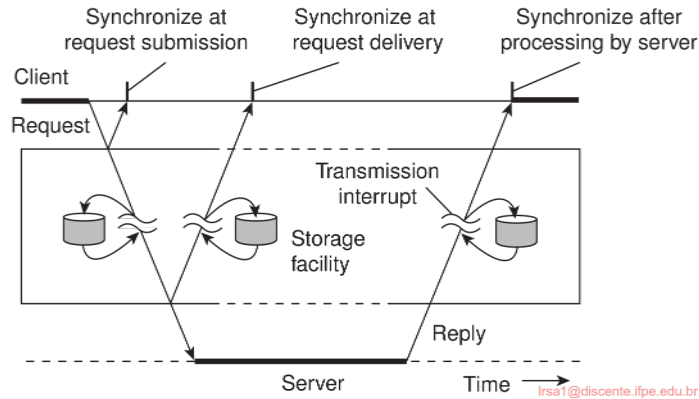


Figure 4.4: Viewing middleware as an intermediate (distributed) service in application-level communication.

An e-mail system is a typical example in which communication is persistent. With **persistent communication**, a message that has been submitted for transmission is stored by the communication middleware as long as it takes to deliver it to the receiver. In this case, the middleware will store the message at one or several of the storage facilities shown in Figure 4.4. As a consequence, it is not necessary for the sending application to continue execution after submitting the message. Likewise, the receiving application need not be executing when the message is submitted.

In contrast, with **transient communication**, a message is stored by the communication system only as long as the sending and receiving application are executing. More precisely, in terms of Figure 4.4, if the middleware cannot deliver a message due to a transmission interrupt, or because the recipient is currently not active, it will simply be discarded. Typically, all transport-level communication services offer only transient communication. In this case, the communication system consists of traditional store-and-forward routers. If a router cannot deliver a message to the next one or the destination host, it will simply drop the message.

Besides being persistent or transient, communication can also be asynchronous or synchronous. The characteristic feature of **asynchronous communication** is that a sender continues immediately after it has submitted its message for transmission. This means that the message is (temporarily) stored immediately by the middleware upon submission. With **synchronous communication**, the sender is blocked until its request is known to be accepted. There are essentially three points where synchronization can take place. First,

the sender may be blocked until the middleware notifies that it will take over transmission of the request. Second, the sender may synchronize until its request has been delivered to the intended recipient. Third, synchronization may take place by letting the sender wait until its request has been fully processed, that is, up to the time that the recipient returns a response.

Various combinations of persistence and synchronization occur in practice. Popular ones are persistence in combination with synchronization at request submission, which is a common scheme for many message-queuing systems, which we discuss later in this chapter. Likewise, transient communication with synchronization after the request has been fully processed is also widely used. This scheme corresponds with remote procedure calls, which we discuss in the following section.

4.2 Remote procedure call

Many distributed systems have been based on explicit message exchange between processes. However, the operations send and receive do not conceal communication at all, which is important to achieve access transparency in distributed systems. This problem has long been known, but little was done about it until researchers in the 1980s [Birrell and Nelson, 1984] introduced an entirely different way of handling communication. Although the idea is refreshingly simple (once someone has thought of it), the implications are often subtle. In this section we will examine the concept, its implementation, its strengths, and its weaknesses.

In a nutshell, the proposal was to allow programs to call procedures located on other machines. When a process on a machine A calls a procedure on a machine B, the calling process on A is suspended, and execution of the called procedure takes place on B. Information can be transported from the caller to the callee in the parameters and can come back in the procedure result. No message passing at all is visible to the programmer. This method is known as **remote procedure call**, or often just **RPC**.

While the basic idea sounds simple and elegant, subtle problems exist. To start with because the calling and called procedures run on different machines, they execute in different address spaces, which causes complications. Parameters and results also have to be passed, which can be complicated, especially if the machines are not identical. Finally, either or both machines can crash, and each of the possible failures causes different problems. Still, most of these can be dealt with, and RPC is a widely used technique that underlies many distributed systems.

4.2.1 Basic RPC operation

The idea behind RPC is to make a remote procedure call look as much as possible as a local one. In other words, we want RPC to be transparent—the

calling procedure should not be aware that the called procedure is executing on a different machine or vice versa. Suppose that a program has access to a database that allows it to append data to a stored list, after which it returns a reference to the modified list. The operation is made available to a program by a routine `append`:

```
newlist = append(data, dbList)
```

In a traditional (single processor) system, `append` is extracted from a library by the linker and inserted into the object program. In principle, it can be a short procedure, which could be implemented by a few file operations for accessing the database.

Even though `append` eventually does only a few basic file operations, it is called in the usual way, by pushing its parameters onto the stack. The programmer does not know the implementation details of `append`, and this is, of course, how it is supposed to be.

Note 4.2 (More information: Conventional procedure calls)

To understand how RPC works and some of its pitfalls, it may help to first understand how a conventional (i.e., single machine) procedure call works. Consider the operation `newlist = append(data, dbList)`;

We assume that the purpose of this call is to take a *globally* defined list object, referred here to as `dbList`, and append a simple data element to it, represented by the variable `data`. An important observation is that in various programming languages such as C, `dbList` is implemented as a *reference* to a list object (i.e., a pointer), whereas `data` may be represented directly by its value (which we assume to be the case here). When calling `append`, both the representations of `data` and `dbList` are pushed onto the stack, making those representations accessible to the implementation of `append`. For `data`, this means the variable follows a **call-by-value** policy, the policy for `dbList` is **call-by-reference**. What happens before and during the call is shown in Figure 4.5.

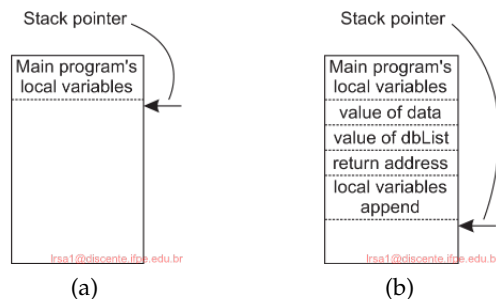


Figure 4.5: (a) Parameter passing in a local procedure call: the stack before the call to `append`. (b) The stack while the called procedure is active.

Several things are worth noting. For one, a value parameter, such as `data`, is just an initialized local variable. The called procedure may modify it, but such changes do not affect the original value at the calling side.

When a parameter like `dbList` is actually a pointer to a variable rather than the value of the variable, something else happens. What is pushed onto the stack is the address of the list object as stored in main memory. When the value of `data` is appended to the list, a call to `append` *does* modify the list object. The difference between call-by-value and call-by-reference is quite important for RPC.

One other parameter passing mechanism also exists, although it is not used in most programming languages. It is called **call-by-copy/restore**. It consists of having the variable copied to the stack by the caller, as in call-by-value, and then copied back after the call, overwriting the caller's original value. Under most conditions, this achieves the same effect as call-by-reference, but in some situations, such as the same parameter being present multiple times in the parameter list, the semantics are different.

The decision of which parameter passing mechanism to use is normally made by the language designers and is a fixed property of the language. Every so often, it depends on the data type being passed. In C, for example, integers and other scalar types are always passed by value, whereas arrays are always passed by reference. Some Ada compilers use copy/restore for inout parameters, but others use call-by-reference. The language definition permits either choice, which makes the semantics a bit fuzzy. In Python, all variables are passed by reference, but some actually get copied to local variables, thus mimicking the behavior of copy-by-value.

RPC achieves its transparency analogously. When `append` is actually a remote procedure, a different version of `append`, called a **client stub**, is offered to the calling client. Like the original one, it, too, is called using a normal calling sequence. However, unlike the original one, it does not perform an append operation. Instead, it packs the parameters into a message and requests that message to be sent to the server, as illustrated in Figure 4.6. Following the call to send, the client stub calls `receive`, blocking itself until the reply comes back.

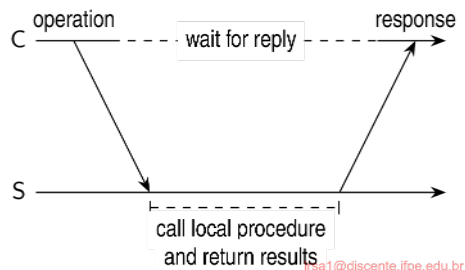


Figure 4.6: The principle of RPC between a client and server program.

When the message arrives at the server, the server's operating system passes it to a **server stub**. A server stub is the server-side equivalent of a client stub: it is a piece of code that transforms requests coming in over the network into local procedure calls. Typically, the server stub will have called `receive` and be blocked waiting for incoming messages. The server stub unpacks the parameters from the message and then calls the server procedure in the usual way. From the server's perspective, it is as though it is being called directly by the client—the parameters and return address are all on the stack where they belong and nothing seems unusual. The server performs its work and then returns the result to the caller (in this case the server stub) in the usual way.

When the server stub gets control back after the call has completed, it packs the result in a message and calls `send` to return it to the client. Thereafter, the server stub usually does a call to `receive` again, to wait for the next incoming request.

When the result message arrives at the client's machine, the operating system passes it through the `receive` operation, which had been called previously, to the client stub, and the client process is subsequently unblocked. The client stub inspects the message, unpacks the result, copies it to its caller, and returns in the usual way. When the caller gets control following the call to `append`, all it knows is that it appended some data to a list. It has no idea that the work was done remotely at another machine.

This blissful ignorance for the client is the beauty of the whole scheme. As far as it is concerned, remote services are accessed by making ordinary (i.e., local) procedure calls, not by calling `send` and `receive`. All the details of the message passing are hidden away in the two library procedures, just as the details of actually making system calls are hidden away in traditional libraries. To summarize, a remote procedure call occurs in the following steps:

1. The client procedure calls the client stub in the normal way.
2. The client stub builds a message and calls the local operating system.
3. The client's OS sends the message to the remote OS.
4. The remote OS gives the message to the server stub.
5. The server stub unpacks the parameter(s) and calls the server.
6. The server does the work and returns the result to the stub.
7. The server stub packs the result in a message and calls its local OS.
8. The server's OS sends the message to the client's OS.
9. The client's OS gives the message to the client stub.
10. The stub unpacks the result and returns it to the client.

The first steps are shown in Figure 4.7 for an abstract two-parameter procedure `doit(a,b)`, where we assume that parameter `a` is of type `type1`, and `b` of type `type2`. The net effect of all these steps is to convert the local call by the client procedure to the client stub, to a local call to the server procedure,

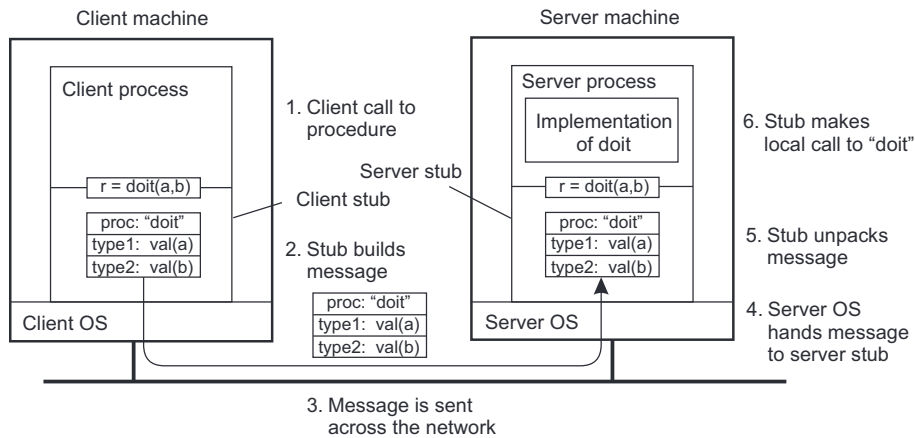


Figure 4.7: The steps involved in calling a remote procedure `doit(a,b)`. The return path for the result is not shown.

without either client or server being aware of the intermediate steps or the existence of the network.

Note 4.3 (More information: An example in Python)

To make matters concrete, let us consider how a remote procedure call could be implemented for the operation `append` discussed previously. Take a look at the Python code shown in Figure 4.8 (from which we omit nonessential code fragments).

The class `DBList` is a simple representation of a list object, mimicking what one would expect to see in a version that would be found in a database environment. The client stub, represented by the class `Client`, consists of an implementation of `append`. When called with parameters `data` and `dbList`, the following happens. The call is transformed into a tuple `(APPEND, data, dbList)` containing all the information the server would need to do its work. The client stub then sends the request off to the server, and subsequently waits for the response. In the `channel` package, a `recvFrom` operation always returns a `(sender,message)` pair, allowing the caller to identify the process who had sent the message. In our case, when the response comes in, the client stub finishes the call to `append` by simply passing the returned message from the server to the program that initially called the stub.

On the server side, we see that in the server stub, the server waits for any incoming message, and inspects which operation it is required to call. Again, the `channel` package returns the identifier of the original sender (line 24). Assuming the server received a request to call `append`, it then simply does a *local* call to its implementation of `append` with the appropriate parameters as also found in the request tuple. The result is then sent back to the identified client.

Note that an actual client would simply call `c.append(...)` where `c` is an instance of the class `Client`. Indeed, the call truly seems to take place locally.

```

1 import channel, pickle
2
3 class DBList:
4     def append(self, data):
5         self.value.extend(data)
6         return self
7
8 class Client:
9     def append(self, data, dbList):
10        msglst = (APPEND, data, dbList)           # message payload
11        self.channel.sendTo(self.server, msglst)  # send message to server
12        msgrcv = self.channel.recvFrom(self.server) # wait for an incoming message
13
14        # A call to recvFrom returns a [senderID, message] pair
15        return msgrcv[1]                          # pass returned message to caller
16
17 class Server:
18     def append(self, data, dbList):
19         return dbList.append(data)
20
21     def run(self):
22         while True:
23             msgreq = self.channel.recvFromAny() # wait for any request
24             client = msgreq[0]                  # see who is the caller
25             msgrpc = msgreq[1]                  # fetch the actual request
26
27             # At this point, msgreq should have the form (operation, data, list)
28             if APPEND == msgrpc[0]:             # check what is being requested
29                 result = self.append(msgrpc[1], msgrpc[2]) # do local call
30                 self.channel.sendTo(client, result)        # return response

```

Figure 4.8: A simple RPC example for operation append.

4.2.2 Parameter passing

The function of the client stub is to take its parameters, pack them into a message, and send them to the server stub. While this sounds straightforward, it is not quite as simple as it at first appears.

Packing parameters into a message is called **parameter marshaling**. Returning to our append operation, we need to ensure that its two parameters (data and dbList) are sent over the network and correctly interpreted by the server. The thing to realize, is that the server will just be seeing a series of bytes coming in that constitute the original message sent by the client. However, no additional information on what those bytes mean is normally provided with the message. Also, we would be facing the same problem again: how should the meta-information be recognized as such by the server?

Besides this interpretation problem, we also need to handle the case that the placement of bytes in memory may differ between machine architectures. In particular, we need to account for the fact that some machines, such as Intel processors, number their bytes from right to left, whereas many others, such as the older ARM processors, number them the other way (ARM now supports both). The Intel format is called **little endian** and the (older) ARM format is called **big endian**. Byte ordering is also important for networking: also here we can witness that machines may use a different ordering when transmitting (and thus receiving) bits and bytes. However, big endian is what is normally used for transferring bytes across a network.

The solution to this problem is to transform data that is to be sent to a machine- and network-independent format, next to making sure that both communicating parties expect the same *message data type* to be transmitted. The latter can typically be solved at the level of programming languages. The former is accomplished by using machine-*dependent* routines that transform data to and from machine- and network-independent formats.

Marshaling and unmarshaling is all about this transformation to neutral formats and forms an essential part of remote procedure calls.

Note 4.4 (More information: An example in Python revisited)

```

1  import channel, pickle
2
3  class Client:
4      def append(self, data, dbList):
5          msglst = (APPEND, data, dbList)          # message payload
6          msgsnd = pickle.dumps(msglst)            # wrap call
7          self.channel.sendTo(self.server, msgsnd) # send request to server
8          msgrcv = self.channel.recvFrom(self.server) # wait for response
9          retval = pickle.loads(msgrcv[1])          # unwrap return value
10         return retval                             # pass it to caller
11
12     class Server:
13         def run(self):
14             while True:
15                 msgreq = self.channel.recvFromAny() # wait for any request
16                 client = msgreq[0]                  # see who is the caller
17                 msgrpc = pickle.loads(msgreq[1])    # unwrap the call
18                 if APPEND == msgrpc[0]:             # check what is being requested
19                     result = self.append(msgrpc[1], msgrpc[2]) # do local call
20                     msgres = pickle.dumps(result)    # wrap the result
21                     self.channel.sendTo(client, msgres) # send response

```

Figure 4.9: A simple RPC example for operation `append`, but now with proper marshaling.

It is not difficult to see that the solution to remote procedure calling as shown in Figure 4.8 will not work in general. Only if the client and server are operating on machines that obey the same byte-ordering rules *and* have the same machine representations for data structures, will the exchange of messages as shown lead to correct interpretations. A robust solution is shown in Figure 4.9 (where we again have omitted code for brevity).

In this example, we use the Python `pickle` library for marshaling and unmarshaling data structures. Note that the code hardly changes in comparison to what we have shown in Figure 4.8. The only changes occur just before sending, and after receiving a message. Also note that both client and server are programmed to work on the same data structures, as we discussed above.

(We do note that, behind the scene, whenever a class instance is passed as a parameter, Python generally takes care of *pickling* and later *unpickling* the instance. In this sense, our explicit use of `pickle` is, strictly speaking, not necessary.)

We now come to a difficult problem: How are pointers, or in general, references passed? The answer is: only with the greatest of difficulty, if at all. A pointer is meaningful only within the address space of the process in which it is being used. Getting back to our `append` example, we stated that the second parameter, `dbList`, is implemented through a *reference* to a list stored in a database. If that reference is just a pointer to a local data structure somewhere in the caller's main memory, we cannot simply pass it to the server. The transferred pointer value will most likely be referring to something entirely different.

One solution is just to forbid pointers and reference parameters in general. However, these are so important that this solution is highly undesirable. In fact, it is often not necessary either. First, reference parameters are often used with fixed-sized data types, such as static arrays, or with dynamic data types for which it is easy to compute their size at runtime, such as strings or dynamic arrays. In such cases, we can simply copy the entire data structure to which the parameter is referring, effectively replacing the copy-by-reference mechanism by copy-by-value/restore. Although this is semantically not always identical, it frequently is good enough. An obvious optimization is that when the client stub knows the referred data will be only read, there is no need to copy it back when the call has finished. Copy-by-value is thus good enough.

More intricate data types can often be supported as well, and certainly if a programming language supports those data types. For example, a language such as Python or Java supports user-defined classes, allowing a language system to provide fully automated marshaling and unmarshaling of those data types. Note, however, that as soon as we are dealing with large, nested, or otherwise intricate dynamic data structures, automatic (un)marshaling may not be available, or even desirable.

The problem with pointers and references, as discussed so far, is that they make sense only locally: they refer to memory locations that have

meaning only to the calling process. Problems can be alleviated by using *global* references: references that are meaningful to the calling and the called process. For example, if the client and the server have access to the same file system, passing a file handle instead of a pointer may do the trick. There is one important observation: both processes need to know exactly what to do when a global reference is passed. In other words, if we consider a global reference having an associated data type, the calling and called process should have the same picture of the operations that can be performed. Moreover, both processes should have agreement on exactly what to do when a file handle is passed. Again, these are typically issues that can be solved by proper programming-language support. We will return to this subject shortly.

Note 4.5 (Advanced: Parameter passing in object-based systems)

Object-based systems often use global references. Consider the situation that all objects in the system can be accessed from remote machines. In that case, we can consistently use **object references** as parameters in method invocations. References are passed by value, and thus copied from one machine to the other. When a process is given an object reference as the result of a method invocation, it can simply bind to the object referred to when needed later (see also Section 2.1.2).

Unfortunately, using only distributed objects can be highly inefficient, especially when objects are small, such as integers, or worse yet, Booleans. Each invocation by a client that is not co-located in the same server as the object, generates a request between different address spaces or, even worse, between different machines. Therefore, references to remote objects and those to local objects are often treated differently.

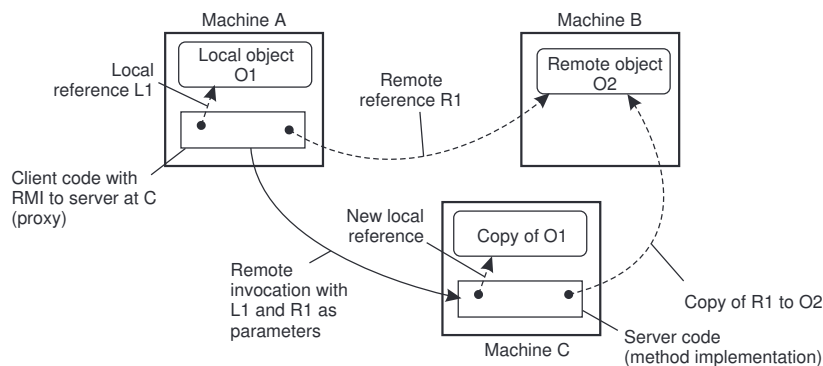


Figure 4.10: Passing an object by reference or by value.

When invoking a method with an object reference as parameter, that reference is copied and passed as a value parameter only when it refers to a remote object. In this case, the object is literally passed by reference. However, when the reference refers to a local object, that is an object in the same address space as the client, the referred object is copied as a whole and passed along with the invocation. In

other words, the object is passed by value.

These two situations are illustrated in Figure 4.10 which shows a client program running on a machine A, and a server program on a machine C. The client has a reference to a local object O1 that it uses as a parameter when calling the server program on machine C. In addition, it holds a reference to a remote object O2 residing at machine B, which is also used as a parameter. When calling the server, a copy of O1 is passed to the server on machine C, along with only a copy of the reference to O2.

Note that whether we are dealing with a reference to a local object or a reference to a remote object can be highly transparent, such as in Java. In Java, the distinction is visible only because local objects are essentially of a different data type than remote objects. Otherwise, both types of references are treated very much the same (see also [Wollrath et al., 1996]). On the other hand, when using conventional programming languages such as C, a reference to a local object can be as simple as a pointer, which can never be used to refer to a remote object.

The side effect of invoking a method with an object reference as parameter is that we may be *copying* an object. Obviously, hiding this aspect is unacceptable so that we are consequently forced to make an explicit distinction between local and distributed objects. Clearly, this distinction not only violates distribution transparency, but also makes it harder to write distributed applications.

We can now also easily explain how global references can be implemented when using portable, interpreted languages such as Python or Java: use the entire client stub as a reference. The key observation is that a client stub is often just another data structure that is compiled into (portable) bytecode. That compiled code can actually be transferred across the network and executed at the receiver's side. In other words, there is no need for explicit binding anymore; simply copying the client stub to the recipient is enough to allow the latter to invoke the associated server-side object. Of course, it may be necessary to marshall that code before shipping it to the other machine, although strictly speaking, there is no reason why that other machine would work with different layouts.

4.2.3 RPC-based application support

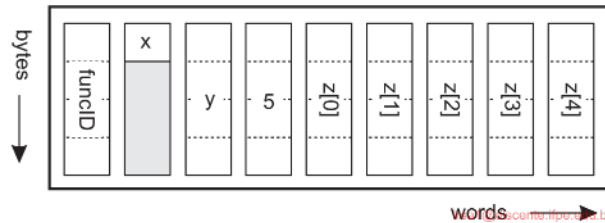
From what we have explained so far, it is clear that hiding a remote procedure call requires that the caller and the callee agree on the format of the messages they exchange and that they follow the same steps when it comes to, for example, passing complex data structures. In other words, both sides in an RPC should follow the same protocol or the RPC will not work correctly. There are at least two ways in which RPC-based application development can be supported. The first one is to let a developer specify exactly what needs to be called remotely, from which complete client-side and server-side stubs can be generated. A second approach is to embed remote procedure calling as part of a programming-language environment.

Stub generation

Consider the function `someFunction` of Figure 4.11(a). It has three parameters, a character, a floating-point number, and an array of five integers. Assuming a word is four bytes, the RPC protocol might prescribe that we should transmit a character in the rightmost byte of a word (leaving the next three bytes empty), a float as a whole word, and an array as a group of words whose size is equal to the array length, preceded by a word giving the length, as shown in Figure 4.11(b). Thus given these rules, the client stub for `someFunction` knows that it must use the format of Figure 4.11(b), and the server stub knows that incoming messages for `someFunction` will have the format of Figure 4.11(b).

```
void someFunction(char x; float y; int z[5])
```

(a)



(b)

Figure 4.11: (a) A function. (b) The corresponding message, and the order in which bytes and words are sent across the network.

Defining the message format is one aspect of an RPC protocol, but it is not sufficient. What we also need is the client and the server to agree on the representation of simple data structures, such as integers, characters, Booleans, etc. For example, the protocol could prescribe that integers are represented in two's complement, characters in 16-bit Unicode, and floats in the IEEE standard #754 format, with everything stored in little endian. With this additional information, messages can be unambiguously interpreted.

With the encoding rules now pinned down to the last bit, the only thing that remains to be done is that the caller and callee agree on the actual exchange of messages. For example, it may be decided to use a connection-oriented transport service, such as TCP/IP. An alternative is to use an unreliable datagram service and let the client and server implement an error control scheme as part of the RPC protocol. In practice, several variants exist, and it is up to the developer to indicate the preferred underlying communication service.

Once the RPC protocol has been fully defined, the client and server stubs need to be implemented. Fortunately, stubs for the same protocol, but different

procedures, normally differ only in their interface to the applications. An interface consists of a collection of procedures that can be called by a client, and which are implemented by a server. An interface is usually available in the same programming language as the one in which the client or server is written (although this is, strictly speaking, not necessary). To simplify matters, interfaces are often specified through an **Interface Definition Language (IDL)**. An interface specified in such an IDL is then subsequently compiled into a client stub and a server stub, along with the appropriate compile-time or run-time interfaces. This process is sketched in Figure 4.12.

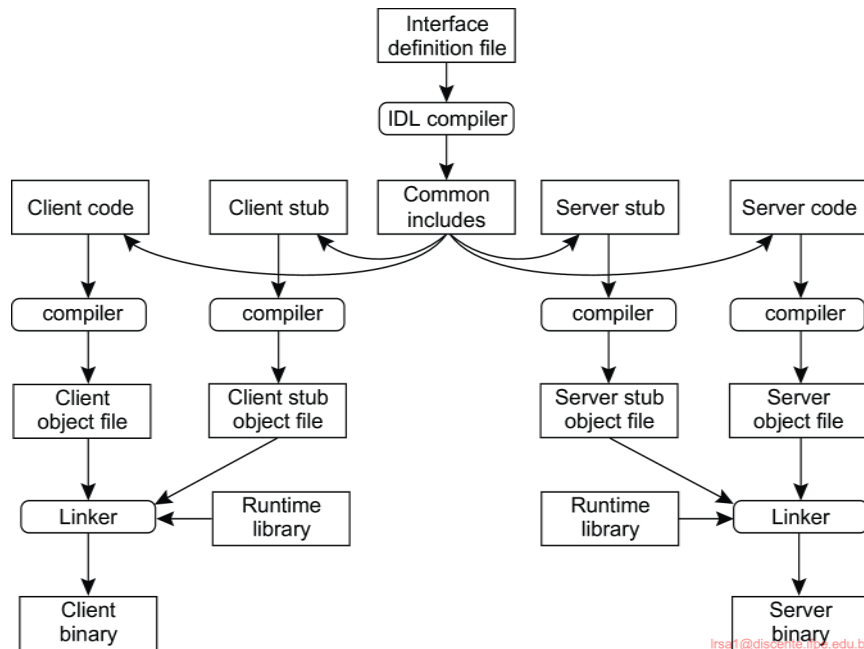


Figure 4.12: The principle of generating stubs from an interface definition file.

The figure shows the situation for compiled languages, yet it is not hard to imagine that a very similar situation holds for interpreted languages. Also note that it is possible that the client and server code are written in *different* languages. There is no principal reason why the client side cannot consist of components written in, for example, Python, while the server side is written in C. Of course, in that case, the box in Figure 4.12 representing common includes will then contain separate files for the client and server, respectively. Important in this scheme is the runtime library: it forms the interface to the actual runtime system, constituting the middleware.

Practice shows that using an interface definition language considerably simplifies client-server applications based on RPCs. Because it is easy to fully

generate client and server stubs, all RPC-based middleware systems offer an IDL to support application development. In some cases, using the IDL is even mandatory.

Note 4.6 (More information: Language-based RPC in Python)

Let us see by an example of how remote procedure calling can be integrated in a language. We have been using the Python language for most of our examples, and will continue to do so now as well. In Figure 4.13 we show a simple server for our DBList data structure. In this case, it has two *exposed* operations: `exposed_append` for appending elements, and `exposed_value` to display what is currently in the list. We use the Python RPyC package for embedding RPCs.

```

1  import rpyc
2  from rpyc.utils.server import ForkingServer
3
4  class DBList(rpyc.Service):
5      value = [] # Used to build a list of strings
6
7      def exposed_append(self, data):
8          self.value.extend(str(data)) # Extend the list with the data
9          return self.value           # Return the current list
10
11  class Server:
12      # Create a forking server at initialization time and immediately start it.
13      # For each incoming request, the server will spawn another process to handle
14      # that request. The process that started the (main) server can simply kill
15      # it when it's time to do so.
16      def __init__(self):
17          self.server = ForkingServer(DBList, hostname=SERVER, port=PORT)
18
19      def start(self):
20          self.server.start()
21
22  class Client:
23      def run(self):
24          conn = rpyc.connect(SERVER, PORT) # Connect to the server
25          conn.root.exposed_append(2)       # Call an exposed operation,
26          conn.root.exposed_append(4)       # and append two elements
27          print(conn.root.exposed_value())  # Print the result

```

Figure 4.13: Embedding RPCs in a language

The client is also shown in Figure 4.13. When a connection is made to the server, a new instance of `DBList` will be created and the client can immediately append values to the list. The exposed operations are called without further ado.

Note that as the client breaks the connection to the server, the list will be lost. It is a **transient object** and special measures will need to be taken to make it a **persistent object**.

Language-based support

The approach described up until now is largely independent of a specific programming language. As an alternative, we can also embed remote procedure calling into a language itself. The main benefit is that application development often becomes much simpler. Furthermore, reaching a high degree of access transparency is often simpler, as many issues related to parameter passing can be circumvented altogether.

A well-known example in which remote procedure calling is fully embedded is Java, where an RPC is referred to as a **remote method invocation (RMI)**. In essence, a client being executed by its own (Java) virtual machine can invoke a method of an object managed by another virtual machine. By simply reading an application's source code, it may be hard or even impossible to see whether a method invocation is to a local or to a remote object.

4.2.4 Variations on RPC

As in conventional procedure calls, when a client calls a remote procedure, the client will block until a reply is returned. This strict request-reply behavior is unnecessary when there is no result to return, or may hinder efficiency when multiple RPCs need to be performed. In the following, we look at two variations on the RPC scheme we have discussed so far.

Asynchronous RPC

To support situations in which there is simply no result to return to the client, RPC systems may provide facilities for what are called **asynchronous RPCs**. With asynchronous RPCs, the server, in principle, immediately sends a reply back to the client the moment the RPC request is received, after which it locally calls the requested procedure. The reply acts as an acknowledgment to the client that the server is going to process the RPC. The client will continue without further blocking as soon as it has received the server's acknowledgment. Figure 4.14(b) shows how client and server interact in the case of asynchronous RPCs. For comparison, Figure 4.14(a) shows the normal request-reply behavior.

Asynchronous RPCs can also be useful when a reply will be returned, but the client is not prepared to wait for it and do nothing in the meantime. A typical case is when a client needs to contact several servers independently. In that case, it can send the call requests one after the other, effectively establishing that the servers operate more or less in parallel. After all call requests have been sent, the client can start waiting for the various results to be returned. In cases such as these, it makes sense to organize the communication between the client and server through an asynchronous RPC combined with a **callback**, as shown in Figure 4.15. In this scheme, also referred to as **deferred synchronous RPC**, the client first calls the server, waits for the acceptance,

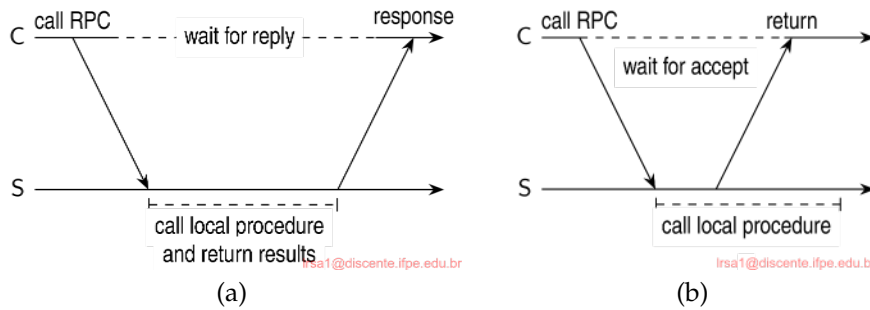


Figure 4.14: (a) The interaction between client and server in a traditional RPC. (b) The interaction using asynchronous RPC.

and continues. When the results become available, the server sends a response message that leads to a callback at the client's side. A callback is a user-defined function that is invoked when a special event happens, such as an incoming message. A straightforward implementation is to spawn a separate thread and let it block on the occurrence of the event while the main process continues. When the event occurs, the thread is unblocked and calls the function.

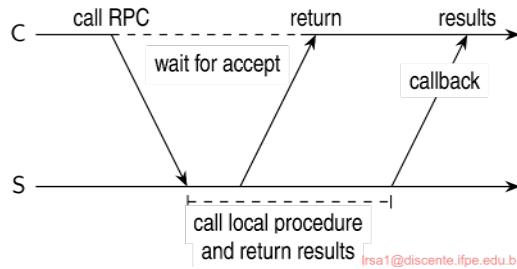


Figure 4.15: A client and server interacting through a deferred synchronous RPC.

It should be noted that variants of asynchronous RPCs exist in which the client continues executing immediately after sending the request to the server. In other words, the client does not wait for an acknowledgment of the server's acceptance of the request. We refer to such RPCs as **one-way RPCs**. The problem with this approach is that when reliability is not guaranteed, the client cannot know for sure whether its request will be processed. We return to these matters in Chapter 8. Likewise, in the case of deferred synchronous RPC, the client may poll the server to see whether the results are available yet, instead of letting the server calling back the client.

Multicast RPC

Asynchronous and deferred synchronous RPCs facilitate another alternative to remote procedure calls, namely executing multiple RPCs at the same time. Adopting the one-way RPCs (i.e., when a server does not tell the client it has accepted its call request but immediately starts processing it, while the client continues just after issuing the RPC), a **multicast RPC** boils down to sending an RPC request to a group of servers. This principle is shown in Figure 4.16. In this example, the client sends a request to two servers, who subsequently process that request independently and in parallel. When done, the result is returned to the client where a callback takes place.

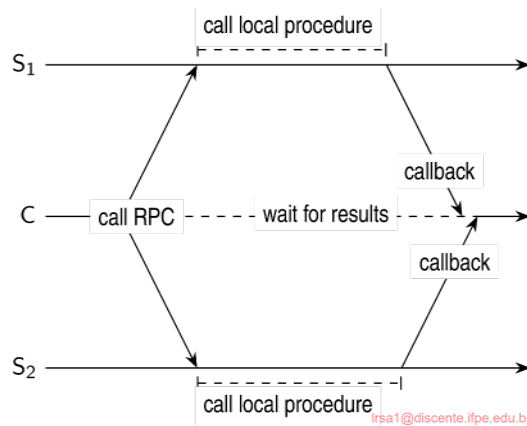


Figure 4.16: The principle of a multicast RPC.

There are several issues that we need to consider. First, as before, the client application may be unaware of the fact that an RPC is actually being forwarded to more than one server. For example, to increase fault tolerance, we may decide to have all operations executed by a backup server who can take over when the main server fails. That a server has been replicated can be completely hidden from a client application by an appropriate stub. Yet even the stub need not be aware that the server is replicated, for example because we are using a transport-level multicast address.

Second, we need to consider what to do with the responses. In particular, will the client proceed after all responses have been received, or wait just for one? It all depends. When the server has been replicated for fault tolerance, we may decide to wait for just the first response, or perhaps until a majority of the servers returns the same result. On the other hand, if the servers have been replicated to do the same work but on different parts of the input, their results may need to be merged before the client can continue. Again, such matters can be hidden in the client-side stub, yet the application developer will, at the very least, have to specify the purpose of the multicast RPC.

4.3 Message-oriented communication

Remote procedure calls and remote object invocations contribute to hiding communication in distributed systems, that is, they enhance access transparency. Unfortunately, neither mechanism is always appropriate. In particular, when it cannot be assumed that the receiving side is executing at the time a request is issued, alternative communication services are needed. Likewise, the inherent synchronous nature of RPCs, by which a client is blocked until its request has been processed, may need to be replaced by something else.

That something else is messaging. In this section, we concentrate on message-oriented communication in distributed systems by first taking a closer look at what exactly synchronous behavior is and what its implications are. Then, we discuss messaging systems that assume that parties are executing at the time of communication. Finally, we will examine message-queuing systems that allow processes to exchange information, even if the other party is not executing at the time communication is initiated.

4.3.1 Simple transient messaging with sockets

Many distributed systems and applications are built directly on top of the simple message-oriented model offered by the transport layer. To better understand and appreciate the message-oriented systems as part of middleware solutions, we first discuss messaging through transport-level sockets.

Special attention has been paid to standardizing the interface of the transport layer to allow programmers to make use of its entire suite of (messaging) protocols through a simple set of operations. Furthermore, standard interfaces make it easier to port an application to a different machine. As an example, we briefly discuss the **socket interface** as introduced in the 1970s in Berkeley Unix, and which has been adopted as a POSIX standard (with only very few adaptations).

Conceptually, a **socket** is a communication end point to which an application can write data that are to be sent out over the underlying network, and from which incoming data can be read. A socket forms an abstraction over the actual port that is used by the local operating system for a specific transport protocol. In the following text, we concentrate on the socket operations for TCP, which are shown in Figure 4.17.

Servers generally execute the first four operations, normally in the order given. When calling the socket operation, the caller creates a new communication end point for a specific transport protocol. Internally, creating a communication end point means that the local operating system reserves resources for sending and receiving messages for the specified protocol.

The bind operation associates a local address with the newly created socket. For example, a server should bind the IP address of its machine together with a (possibly well-known) port number to a socket. Binding tells the operating

Operation	Description
socket	Create a new communication end point
bind	Attach a local address to a socket
listen	Tell operating system what the maximum number of pending connection requests should be
accept	Block caller until a connection request arrives
connect	Actively attempt to establish a connection
send	Send some data over the connection
receive	Receive some data over the connection
close	Release the connection

Figure 4.17: The socket operations for TCP/IP.

system that the server wants to receive messages only on the specified address and port. In the case of connection-oriented communication, the address is used to receive incoming connection requests.

The listen operation is called only in the case of connection-oriented communication. It is a nonblocking call that allows the local operating system to reserve enough buffers for a specified maximum number of pending connection requests that the caller is willing to accept.

A call to accept blocks the caller until a connection request arrives. When a request arrives, the local operating system creates a new socket with the same properties as the original one, and returns it to the caller. This approach will allow the server to, for example, fork a process that will subsequently handle the actual communication through the new connection. The server can go back and wait for another connection request on the original socket.

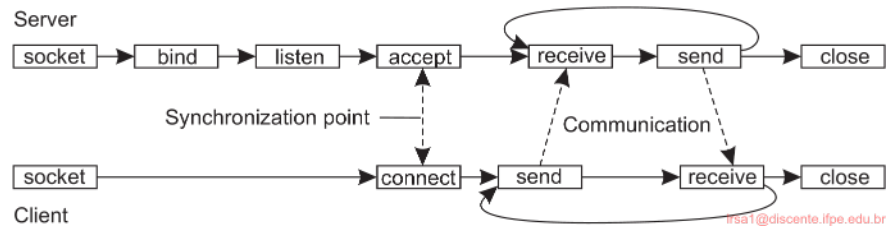


Figure 4.18: Connection-oriented communication pattern using sockets.

Let us now take a look at the client side. Here, too, a socket must first be created using the `socket` operation, but explicitly binding the socket to a local address is not necessary, since the operating system can dynamically allocate a port when the connection is set up. The `connect` operation requires that the caller specifies the transport-level address to which a connection request is to be sent. The client is blocked until a connection has been set up successfully, after which both sides can start exchanging information through the `send` and `receive` operations. Finally, closing a connection is symmetric when using

sockets, and is established by having both the client and server call the `close` operation. Although there are many exceptions to the rule, the general pattern followed by a client and server for connection-oriented communication using sockets is as shown in Figure 4.18. Details on network programming using sockets and other interfaces in Unix can be found in [Stevens, 1998].

Note 4.7 (Example: A simple socket-based client-server system)

As an illustration of the recurring pattern in Figure 4.18, consider the simple socket-based client-server system shown in Figure 4.19 (see also Note 2.1). We see the server starting by creating a socket, and subsequently binding an address to that socket. It calls the `listen` operation, and waits for an incoming connection request. When the server accepts a connection, the socket library creates a separate connection, `conn`, which is used to receive data and send a response to the connected client. The server enters a loop receiving and sending messages, until no more data has been received. It then closes the connection.

```

1 from socket import *
2
3 class Server:
4     def run(self):
5         s = socket(AF_INET, SOCK_STREAM)
6         s.bind((HOST, PORT))
7         s.listen(1)
8         (conn, addr) = s.accept() # returns new socket and addr. client
9         while True:               # forever
10            data = conn.recv(1024) # receive data from client
11            if not data: break      # stop if client stopped
12            conn.send(data+b"*")   # return sent data plus an "*"
13            conn.close()           # close the connection

```

(a)

```

1 class Client:
2     def run(self):
3         s = socket(AF_INET, SOCK_STREAM)
4         s.connect((HOST, PORT)) # connect to server (block until accepted)
5         s.send(b"Hello, world") # send same data
6         data = s.recv(1024)     # receive the response
7         print(data)             # print what you received
8         s.send(b"")             # tell the server to close
9         s.close()               # close the connection

```

(b)

Figure 4.19: A simple socket-based client-server system.

The client again follows the pattern from Figure 4.18. It creates a socket, and calls `connect` to request a connection with the server. Once the connection has been established, it sends a single message, waits for the response, and after printing the result, closes the connection.

Note 4.8 (Advanced: Implementing stubs as global references revisited)

To provide a more in-depth insight in the working of sockets, let us look at a more elaborate example, namely the use of stubs as global references.

```

1 class DBClient:
2     def __sendrecv(self, message):          # this is a private method
3         sock = socket()                   # create a socket
4         sock.connect((self.host, self.port)) # connect to server
5         sock.send(pickle.dumps(message))    # send some data
6         result = pickle.loads(sock.recv(1024)) # receive the response
7         sock.close()                       # close the connection
8         return result
9
10    def create(self):
11        self.listID = self.__sendrecv([CREATE])
12        return self.listID
13
14    def getValue(self):
15        return self.__sendrecv([GETVALUE, self.listID])
16
17    def appendData(self, data):
18        return self.__sendrecv([APPEND, data, self.listID])

```

(a)

```

1 class Server:
2     self.setOfLists = {}                  # init: no lists to manage
3
4     def run(self):
5         while True:
6             (conn, addr) = self.sock.accept() # accept incoming call
7             data = conn.recv(1024)           # fetch data from client
8             request = pickle.loads(data)      # unwrap the request
9
10            if request[0] == CREATE:           # create a list
11                listID = len(self.setOfLists) + 1 # allocate listID
12                self.setOfLists[listID] = []     # initialize to empty
13                conn.send(pickle.dumps(listID))  # return ID
14
15            elif request[0] == APPEND:         # append request
16                listID = request[2]             # fetch listID
17                data = request[1]              # fetch data to append
18                self.setOfLists[listID].append(data) # append it to the list
19                conn.send(pickle.dumps(OK))     # return an OK
20
21            elif request[0] == GETVALUE:       # read request
22                listID = request[1]            # fetch listID
23                result = self.setOfLists[listID] # get the elements
24                conn.send(pickle.dumps(result)) # return the list

```

(b)

Figure 4.20: Implementing a list server in Python.

We return to our example of implementing a shared list, which we now do using a list server, implemented in the form of the Python class shown in Figure 4.20(b). Figure 4.20(a) shows the stub implementation of a shared list. Again, we have omitted code for readability.

The `DBClient` class represents a client-side stub that, once marshaled, can be passed between processes. It provides three operations associated with a list: `create`, `getValue`, and `append`, with obvious semantics. A `DBClient` is assumed to be associated with one specific list as managed by the server. An identifier for that list is returned when the list is created. Note how the (internal) `sendrecv` operation follows the client-side pattern explained in Figure 4.18.

The server maintains lists, as shown in Figure 4.20(b). Its internal data structure is a `setOfLists` with each element being a previously created list. The server simply waits for incoming requests, unmarshals the request, and checks which operation is being requested. Results are sent back to the requesting client (which always issues the `sendrecv` operation implemented as part of `DBClient`). Again, we see that the server follows the pattern shown in Figure 4.18: it creates a socket, binds an address to it, informs the operating system to how many connections it should listen, and then waits to accept an incoming connection request. Once a connection has been established, the server receives data, sends a response, and closes the connection again.

```

1 class Client:
2     def __init__(self, port):
3         self.host = 'localhost'           # this machine
4         self.port = port                  # port it will listen to
5         self.sock = socket()              # socket for incoming calls
6         self.sock.bind((self.host, self.port)) # bind socket to an address
7         self.sock.listen(2)               # max num connections
8
9     def sendTo(self, host, port, data):
10        sock = socket()
11        sock.connect((host, port))        # connect to server (blocking call)
12        sock.send(pickle.dumps(data))     # send some data
13        sock.close()
14
15    def recvAny(self):
16        (conn, addr) = self.sock.accept()
17        return conn.recv(1024)

```

Figure 4.20: (c) Implementing a list server in Python: the client.

To use a stub as a global reference, we represent each client application by the class `Client` shown in Figure 4.20(c). The class is instantiated in the same process running the application (exemplified by the value of `self.host`), and will be listening on a specific port for messages from other applications, as well as the server. Otherwise, it merely sends and receives messages, coded through the operations `sendTo` and `recvAny`, respectively.

Now consider the code shown in Figure 4.20(d), which mimics two client applications. The first one creates a new list and appends data to it. Then note

how `dbClient1` is simply sent to the other client. Under the hood, we now know that it is marshaled in the operation `sendTo` (line 12) of class `Client` shown in Figure 4.20(c).

```

1 def client1():
2     c1 = Client(PORTC1)           # create client
3     dbC1 = DBClient(HOSTS,PORTS)  # create reference
4     dbC1.create()                 # create new list
5     dbC1.appendData('Client 1')   # append some data
6     c1.sendTo(HOSTC2,PORTC2,dbC1) # send to other client
7
8 def client2():
9     c2 = Client(PORTC2)           # create a new client
10    data = c2.recvAny()            # block until data is sent
11    dbC2 = pickle.loads(data)      # receive reference
12    dbC2.appendData('Client 2')    # append data to same list

```

Figure 4.20: (d) Passing stubs as references.

The second client simply waits for an incoming message (line 12), unmarshals the result, knowing that it is a `DBClient` instance, and subsequently appends some more data to the same list as the one the first client appended data. Indeed, an instance of `DBClient` is seen to be passed as a global reference, seemingly along with all the operations that go with the associated class.

4.3.2 Advanced transient messaging

The standard socket-based approach toward transient messaging is very basic and, as such, rather brittle: a mistake is easily made. Furthermore, sockets essentially support only TCP or UDP, meaning that any extra facility for messaging needs to be implemented separately by an application programmer. In practice, we do often need more advanced approaches for message-oriented communication to make network programming easier, to expand beyond the functionality offered by existing networking protocols, to make better use of local resources, and so on.

Using messaging patterns: ZeroMQ

One approach toward making network programming easier is based on the observation that many messaging applications, or their components, can be effectively organized according to a few simple communication patterns. By subsequently providing enhancements to sockets for each of these patterns, it may become easier to develop a networked, distributed application. This approach has been followed in ZeroMQ and documented in [Hintjens, 2013; Akgul, 2013].

Like in the Berkeley approach, ZeroMQ also provides sockets through which all communication takes place. Actual message transmission generally takes place over TCP connections, and like TCP, all communication is essentially connection-oriented, meaning that a connection will first be set up between a sender and receiver before message transmission can take place. However, setting up, and maintaining connections is kept mostly under the hood: an application programmer need not bother with those issues. To further simplify matters, a socket may be bound to multiple addresses, effectively allowing a server to handle messages from very different sources through a single interface. For example, a server can listen to multiple ports using a single blocking receive operation. ZeroMQ sockets can thus support *many-to-one* communication instead of just *one-to-one* communication, as is the case with standard Berkeley sockets. To complete the story: ZeroMQ sockets also support *one-to-many* communication, i.e., multicasting.

Essential to ZeroMQ is that communication is **asynchronous**: a sender will normally continue after having submitted a message to the underlying communication subsystem. An interesting side effect of combining asynchronous with connection-oriented communication, is that a process can request a connection setup, and subsequently send a message even if the recipient is not yet up-and-running and ready to accept incoming connection requests, let alone incoming messages. What happens, of course, is that a connection request and subsequent messages are queued at the sender's side, while a separate thread as part of ZeroMQ's library will take care that eventually the connection is set up and messages are transmitted to the recipient.

Simplifying matters, ZeroMQ establishes a higher level of abstraction in socket-based communication by *pairing* sockets: a specific type of socket used for sending messages is paired with a corresponding socket type for receiving messages. Each pair of socket types corresponds to a **communication pattern**. The three most important communication patterns supported by ZeroMQ are *request-reply*, *publish-subscribe*, and *pipeline*.

The **request-reply pattern** is used in traditional client-server communication, like the ones normally used for remote procedure calls. A client application uses a *request socket* (of type REQ) to send a request message to a server and expects the latter to respond with an appropriate response. The server is assumed to use a *reply socket* (of type REP). The request-reply pattern simplifies matters for developers by avoiding the need to call the `listen` operation, as well as the `accept` operation. Moreover, when a server receives a message, a subsequent call to `send` is automatically targeted toward the original sender. Likewise, when a client calls the `recv` operation (for receiving a message) after having sent a message, ZeroMQ assumes the client is waiting for a response from the original recipient. Note that this approach was effectively encoded in the local `sendrecv` operation of Figure 4.20(b), which we discussed in Note 4.8.

Note 4.9 (Example: The request-reply pattern)

Let us look at a simple programming example to illustrate the request-reply pattern. Figure 4.21 shows a server that appends an asterisk to a received message. As before, it creates a socket, and binds it to a combination of a protocol (in this case TCP), and a host and port. In our example, the server is willing to accept incoming connection requests on two different ports. It then waits for incoming messages. The request-reply pattern effectively ties the receipt of a message to the subsequent response. In other words, when the server calls send, it will transmit a message to the same client from which it previously had received a message. Of course, this simplicity can be achieved only if the programmer indeed abides to the request-reply pattern.

```

1 import zmq
2
3 def server():
4     context = zmq.Context()
5     socket = context.socket(zmq.REP)      # create reply socket
6     socket.bind("tcp://*:12345")         # bind socket to address
7
8     while True:
9         message = socket.recv()          # wait for incoming message
10        if not "STOP" in str(message):    # if not to stop...
11            reply = str(message.decode())+'*' # append "*" to message
12            socket.send(reply.encode())     # send it away (encoded)
13        else:
14            break                          # break out of loop and end
15
16 def client():
17     context = zmq.Context()
18     socket = context.socket(zmq.REQ)     # create request socket
19
20     socket.connect("tcp://localhost:12345") # block until connected
21     socket.send(b"Hello world")           # send message
22     message = socket.recv()               # block until response
23     socket.send(b"STOP")                  # tell server to stop
24     print(message.decode())               # print result

```

Figure 4.21: A ZeroMQ client-server system.

The client, also shown in Figure 4.21, creates a socket and connects to the associated server. When it sends a message, it can expect to receive, from that same server, a response. By sending the string “STOP”, it tells the server it is done, after which the server will actually stop.

Interestingly, the asynchronous nature of ZeroMQ allows one to start the client *before* starting the server. An implication is that if, in this example, we would start the server, then a client, and after a while a second client, that the latter will be blocked until the server is restarted. Furthermore, note that ZeroMQ does not require the programmer to specify how many bytes are expected to be received. Unlike TCP, ZeroMQ uses messages instead of byte streams.

In the case of a **publish-subscribe pattern**, clients *subscribe* to specific messages that are *published* by servers. We came across this pattern in Section 2.1.3 when discussing coordination. In effect, only the messages to which the client has subscribed will be transmitted. If a server is publishing messages to which no one has subscribed, these messages will be lost. In its simplest form, this pattern establishes multicasting messages from a server to several clients. The server is assumed to use a socket of type PUB, while each client must use SUB type sockets. Each client socket is connected to the socket of the server. By default, a client subscribes to no specific message. This means that as long as no explicit subscription is provided, a client will not receive a message published by the server.

Note 4.10 (Example: The publish-subscribe pattern)

Again, let us make this pattern more concrete through a simple example. Figure 4.22 shows an admittedly naive time server that publishes its current, local time, through a PUB socket. The local time is published every five seconds, for any interested client.

```

1 import multiprocessing
2 import zmq, time
3
4 def server():
5     context = zmq.Context()
6     socket = context.socket(zmq.PUB)          # create a publisher socket
7     socket.bind("tcp://*:12345")             # bind socket to the address
8     while True:
9         time.sleep(5)                        # wait every 5 seconds
10        t = "TIME " + time.asctime()
11        socket.send(t.encode())               # publish the current time
12
13 def client():
14     context = zmq.Context()
15     socket = context.socket(zmq.SUB)          # create a subscriber socket
16     socket.connect("tcp://localhost:12345")   # connect to the server
17     socket.setsockopt(zmq.SUBSCRIBE, b"TIME") # subscribe to TIME messages
18
19     for i in range(5):                       # Five iterations
20         time = socket.recv()                 # receive a message related to subscription
21         print(time.decode())                 # print the result

```

Figure 4.22: A multicasting socket-based setup.

A client is equally simple, as also shown in Figure 4.22. It first creates a SUB socket which it connects to the corresponding PUB socket of the server. To receive the appropriate messages, it needs to subscribe to messages that have TIME as their tag. In our example, a client will simply print the first five messages received from the server. Note that we can have as many clients as we want: the server's message will be multicasted to all subscribers. Most important in this example, is

that it illustrates that the *only* messages received through the SUB socket, are the ones the client had subscribed to.

Finally, the **pipeline pattern** is characterized by the fact that a process wants to *push out* its results, assuming that there are other processes that want to *pull in* those results. The essence of the pipeline pattern is that a pushing process does not really care which other process pulls in its results: the first available one will do just fine. Likewise, any process pulling in results from multiple other processes will do so from the first pushing process making its results available. The intention of the pipeline pattern is thus seen to keep as many processes working as possible, pushing results through a pipeline of processes as quickly as possible.

Note 4.11 (Example: The pipeline pattern)

As our last example, consider the following template for keeping a collection of worker tasks busy. Figure 4.23 shows the code for a so-called **farmer task**: a process producing tasks to be picked up by others. In this example, we simulate the task by letting the producer pick a random number modeling the duration, or load, of the work to be done. This workload is then sent to the PUSH socket, effectively being queued until another process picks it up.

```

1  def producer():
2      context = zmq.Context()
3      socket = context.socket(zmq.PUSH)    # create a push socket
4      socket.bind("tcp://127.0.0.1:12345")  # bind socket to address
5
6      while True:
7          workload = random.randint(1, 100) # compute workload
8          socket.send(pickle.dumps(workload)) # send workload to worker
9          time.sleep(workload/NWORKERS)      # balance production by waiting
10
11  def worker(id):
12      context = zmq.Context()
13      socket = context.socket(zmq.PULL)    # create a pull socket
14      socket.connect("tcp://localhost:12345") # connect to the producer
15
16      while True:
17          work = pickle.loads(socket.recv()) # receive work from a source
18          time.sleep(work)                  # pretend to work

```

Figure 4.23: A producer-worker pattern.

Such other processes are known as **worker tasks**, of which a sketch is also given in Figure 4.23. A worker task connects to a single PULL socket. Once it picks up some work, it simulates that it is actually doing something by sleeping for some time proportional to the received workload.

The semantics of this push-pull pattern is such that the first available worker will pick up work from the producer, and likewise, if there are multiple workers ready to pick up work, each one of them will be provided with a task. How work distribution is actually done fairly requires some specific attention, which we will not discuss further here.

The Message-Passing Interface (MPI)

With the advent of high-performance multicomputers, developers have been looking for message-oriented operations that would allow them to easily write highly efficient applications. This means that the operations should be at a convenient level of abstraction (to ease application development), and that their implementation incurs only minimal overhead. Sockets were deemed insufficient for two reasons. First, they were at the wrong level of abstraction by supporting only simple send and receive operations. Second, sockets had been designed to communicate across networks using general-purpose protocol stacks such as TCP/IP. They were not considered suitable for the proprietary protocols developed for high-speed interconnection networks, such as those used in high-performance server clusters. Those protocols required an interface that could handle more advanced features, such as different forms of buffering and synchronization.

The result was that most interconnection networks and high-performance multicomputers were shipped with proprietary communication libraries. These libraries offered a wealth of high-level and generally efficient communication operations. Of course, all libraries were mutually incompatible, so that application developers now had a portability problem.

The need to be hardware and platform independent eventually lead to the definition of a standard for message passing, simply called the **Message-Passing Interface** or **MPI**. MPI is designed for parallel applications and as such is tailored to transient communication. It makes direct use of the underlying network. Furthermore, it assumes that serious failures such as process crashes or network partitions are fatal and do not require automatic recovery.

MPI assumes communication takes place within a known group of processes. Each group is assigned an identifier. Each process within a group is also assigned a (local) identifier. A (groupID, processID) pair therefore uniquely identifies the source or destination of a message, and is used instead of a transport-level address. There may be several, possibly overlapping, groups of processes involved in a computation and that are all executing at the same time.

At the core of MPI are messaging operations to support transient communication, of which the most intuitive ones are summarized in Figure 4.25. To understand their semantics, it helps to keep Figure 4.4 in mind, with the

middleware formed entirely by an MPI implementation. In particular, the middleware maintains its own buffers and synchronization is often related to when the necessary data has been copied from the calling application to the middleware.

Operation	Description
MPI_BSEND	Append outgoing message to a local send buffer
MPI_SEND	Send a message and wait until copied to local or remote buffer
MPI_SSEND	Send a message and wait until transmission starts
MPI_SENDRECV	Send a message and wait for reply
MPI_ISEND	Pass reference to outgoing message, and continue
MPI_IRESEND	Pass reference to outgoing message, and wait until receipt starts
MPI_RECV	Receive a message; block if there is none
MPI_IRECV	Check if there is an incoming message, but do not block

Figure 4.24: Some of the more intuitive message-passing operations of MPI.

Transient asynchronous communication is supported by the MPI_BSEND operation. The sender submits a message for transmission, which is generally first copied to a local buffer in the MPI runtime system. When the message has been copied, the sender continues. The local MPI runtime system will remove the message from its local buffer and take care of transmission as soon as a receiver has called a receive operation.

There is also a blocking send operation, called MPI_SEND, of which the semantics are implementation dependent. The operation MPI_SEND may either block the caller until the specified message has been copied to the MPI runtime system at the sender's side, or until the receiver has initiated a receive operation. Synchronous communication by which the sender blocks until its request is accepted for further processing is available through the MPI_SSEND operation. Finally, the strongest form of synchronous communication is also supported: when a sender calls MPI_SENDRECV, it sends a request to the receiver and blocks until the latter returns a reply. Basically, this operation corresponds to a normal RPC.

Both MPI_SEND and MPI_SSEND have variants that avoid copying messages from user buffers to buffers internal to the local MPI runtime system. These variants essentially correspond to a form of asynchronous communication. With MPI_ISEND, a sender passes a pointer to the message, after which the MPI runtime system takes care of communication. The sender immediately continues. To prevent overwriting the message before communication completes, MPI offers operations to check for completion, or even to block if required. As with MPI_SEND, whether the message has actually been transferred to the receiver or that it has merely been copied by the local MPI runtime system to an internal buffer is left unspecified.

Likewise, with `MPI_ISSEND`, a sender also passes only a pointer to the MPI runtime system. When the runtime system indicates it has processed the message, the sender is then guaranteed that the receiver has accepted the message and is now working on it.

The operation `MPI_RECV` is called to receive a message; it blocks the caller until a message arrives. There is also an asynchronous variant, called `MPI_IRECV`, by which a receiver indicates that it is prepared to accept a message. The receiver can check whether a message has indeed arrived, or block until one does.

The semantics of MPI communication operations are not always straightforward, and different operations can sometimes be interchanged without affecting the correctness of a program. The official reason why so many different forms of communication are supported is that it gives implementers of MPI systems enough possibilities for optimizing performance. Cynics might say the committee could not make up its collective mind, so it threw in everything. By now, MPI is in its fourth version with over 650 operations available. Being designed for high-performance parallel applications, it is perhaps easier to understand its diversity. More on MPI can be found in [Gropp et al., 2016]. The complete MPI-4 reference can be found in [Message Passing Interface Forum, 2021].

4.3.3 Message-oriented persistent communication

We now come to an important class of message-oriented middleware services, generally known as **message-queuing systems**, or just **message-oriented middleware (MOM)**. Message-queuing systems provide extensive support for persistent asynchronous communication. The essence of these systems is that they offer intermediate-term storage capacity for messages, without requiring either the sender or receiver to be active during message transmission. An important difference with sockets and MPI is that message-queuing systems are typically targeted to support message transfers that are allowed to take minutes instead of seconds or milliseconds.

Message-queuing model

The basic idea behind a message-queuing system is that applications communicate by inserting messages in specific queues. These messages are forwarded over a series of communication servers and are eventually delivered to the destination, even if it was down when the message was sent. In practice, most communication servers are directly connected to each other. In other words, a message is generally transferred directly to a destination server. In principle, each application has its own private queue to which other applications can send messages. A queue can be read only by its associated application, but it is also possible for multiple applications to share a single queue.

An important aspect of message-queuing systems is that a sender is generally given only the guarantees that its message will eventually be inserted in the recipient's queue. No guarantees are given about when, or even if the message will actually be read, which is completely determined by the behavior of the recipient.

These semantics permit communication to be decoupled in time, as also discussed in Section 2.1.3. There is thus no need for the receiver to be executing when a message is being sent to its queue. Likewise, there is no need for the sender to be executing the moment its message is picked up by the receiver. The sender and receiver can execute completely independently of each other. In fact, once a message has been deposited in a queue, it will remain there until it is removed, irrespective of whether its sender or receiver is executing. This gives us four combinations regarding the execution mode of the sender and receiver, as shown in Figure 4.25.

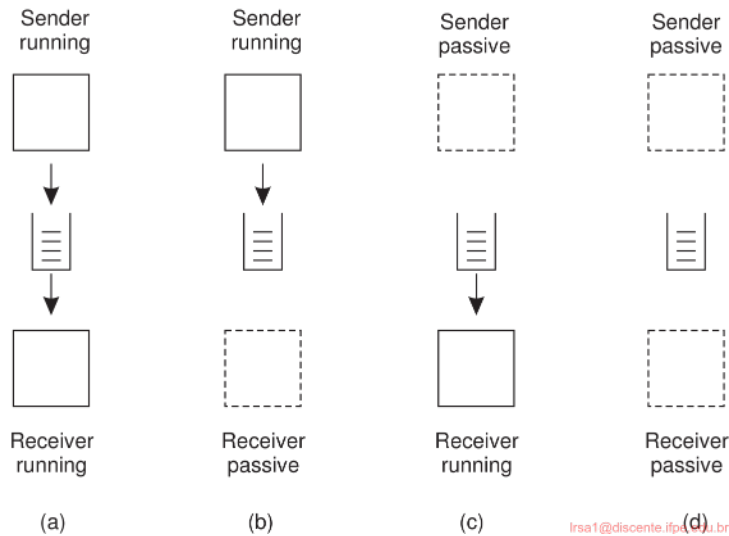


Figure 4.25: Four combinations for loosely coupled communication using queues.

In Figure 4.25(a), both the sender and receiver are in execution during the entire transmission of a message. In Figure 4.25(b), only the sender is in execution, while the receiver is passive, that is, in a state in which message delivery is not possible. Nevertheless, the sender can still send messages. The combination of a passive sender and an active receiver is shown in Figure 4.25(c). In this case, the receiver can read messages that were sent to it, but it is not necessary that their respective senders are in execution as well. Finally, in Figure 4.25(d), we see the situation that the system is storing (and possibly transmitting) messages even while sender and receiver

are passive. One may argue that only if this last configuration is supported, the message-queuing system truly provides persistent messaging.

Messages can, in principle, contain any data. The only important aspect from the perspective of middleware is that messages are properly addressed. In practice, addressing is done by providing a systemwide unique name of the destination queue. In some cases, message size may be limited, although it is also possible that the underlying system takes care of fragmenting and assembling large messages in a way that is completely transparent to applications. An effect of this approach is that the basic interface offered to applications can be simple, as shown in Figure 4.26.

Operation	Description
PUT	Append a message to a specified queue
GET	Block until the specified queue is nonempty, and remove the first message
POLL	Check a specified queue for messages, and remove the first. Never block
NOTIFY	Install a handler to be called when a message is put into the specified queue

Figure 4.26: Basic interface to a queue in a message-queuing system.

The PUT operation is called by a sender to pass a message to the underlying system that is to be appended to the specified queue. As we explained, this is a nonblocking call. The GET operation is a blocking call by which an authorized process can remove the longest pending message in the specified queue. The process is blocked only if the queue is empty. Variations on this call allow searching for a specific message in the queue, for example, using a priority, or a matching pattern. The nonblocking variant is given by the POLL operation. If the queue is empty, or if a specific message could not be found, the calling process simply continues.

Finally, most queuing systems also allow a process to install a handler as a *callback function*, which is automatically invoked whenever a message is put into the queue. Arranging this scheme is handled through a NOTIFY operation. Callbacks can also be used to automatically start a process that will fetch messages from the queue if no process is currently in execution. This approach is often implemented through a daemon on the receiver's side that continuously monitors the queue for incoming messages and handles accordingly.

General architecture of a message-queuing system

Let us now take a closer look at what a general message-queuing system looks like. First, queues are managed by **queue managers**. A queue manager is either a separate process, or is implemented through a library that is linked

with an application. Secondly, as a rule of thumb, an application can put messages only into a *local* queue. Likewise, getting a message is possible by extracting it from a *local* queue only. As a consequence, if a queue manager QM_A handling the queues for an application A runs as a separate process, both processes QM_A and A will generally be placed on the same machine, or at worst on the same LAN. Also note that if *all* queue managers are linked into their respective applications, we can no longer speak of a persistent asynchronous messaging system.

If applications can put messages only into local queues, then clearly each message will have to carry information concerning its destination. It is the queue manager's task to make sure that a message reaches its destination. This brings us to several issues.

In the first place, we need to consider how the destination queue is addressed. Obviously, to enhance location transparency, it is preferable that queues have logical, location-independent names. Assuming that a queue manager is implemented as a separate process, using logical names implies that each name should be associated with a **contact address**, such as a (host,port)-pair, and that the name-to-address mapping is readily available to a queue manager, as shown in Figure 4.27. In practice, a contact address carries more information, notably the protocol to be used, such as TCP or UDP. We came across such contact addresses in our examples of advanced sockets in, for example, Note 4.9.

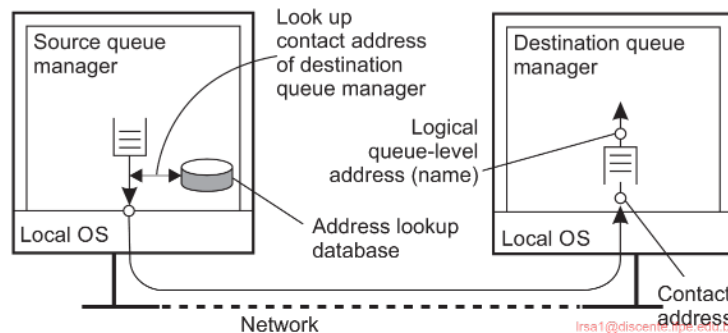


Figure 4.27: The relationship between queue-level naming and network-level addressing.

A second issue that we need to consider is how the name-to-address mapping is actually made available to a queue manager. A common approach is to simply implement the mapping as a lookup table and copy that table to all managers. Obviously, this leads to a maintenance problem, for every time that a new queue is added or named, many, if not all tables, need to be updated. There are various ways to alleviate such problems, which we will discuss in Chapter 6.

This brings us to a third issue, related to the problems of efficiently maintaining name-to-address mappings. We have implicitly assumed that if a destination queue at manager QM_B is known to queue manager QM_A , then QM_A can directly contact QM_B to transfer messages. In effect, this means that (the contact address of) each queue manager should be known to all others. Obviously, when dealing with large message-queuing systems, we will have a scalability problem. In practice, there are often special queue managers that operate as **routers**: they forward incoming messages to other queue managers. In this way, a message-queuing system may gradually grow into a complete, application-level, **overlay network**.

If only a few routers need to know about the network topology, then a source queue manager need only to know to which adjacent router, say R , it should forward a message, given a destination queue. The router R , in turn, may need to keep track only of its adjacent routers to see where to forward the message to, and so on. Of course, we still need to have name-to-address mappings for all queue managers, including the routers, but it is not difficult to imagine that such tables can be much smaller and easier to maintain.

Message brokers

An important application area of message-queuing systems is integrating existing and new applications into a single, coherent distributed information system. If we assume that communication with an application takes place through messages, then integration requires that applications can understand the messages they receive. In practice, this requires the sender to have its outgoing messages in the same format as that of the receiver, but also that its messages adhere to the same semantics as those expected by the receiver. Sender and receiver essentially need to speak the same language, that is, adhere to the same messaging protocol.

The problem with this approach is that each time an application A is added to the system having its own messaging protocol, then for each other application B that is to communicate with A we will need to provide the means for converting their respective messages. In a system with N applications, we will thus need $N \times N$ messaging-protocol converters.

An alternative is to agree on a common messaging protocol, as is done with traditional network protocols. Unfortunately, this approach will generally not work for message-queuing systems. The problem is the level of abstraction at which these systems operate. A common messaging protocol makes sense only if the collection of processes that make use of that protocol indeed have enough in common. If the collection of applications that make up a distributed information system is highly diverse (which it often is), then inventing a one-size-fits-all solution is simply not going to work.

If we focus only on the format and meaning of messages, commonality can be achieved by lifting the level of abstraction, as is done with XML messages.

In this case, messages carry information on their own organization, and what has been standardized is the way that they can describe their content. As a consequence, an application can provide information on the organization of its messages that can be automatically processed. Of course, this information is generally not enough: we also need to make sure that the *semantics* of messages are well understood.

Given these problems, the general approach is to learn to live with differences, and try to provide the means to make conversions as simple as possible. In message-queuing systems, conversions are handled by special nodes in a queuing network, known as **message brokers**. A message broker acts as an application-level gateway in a message-queuing system. Its main purpose is to convert incoming messages so that they can be understood by the destination application. Note that to a message-queuing system, a message broker is just another application, as shown in Figure 4.28. In other words, a message broker is generally not considered to be an integral part of the queuing system.

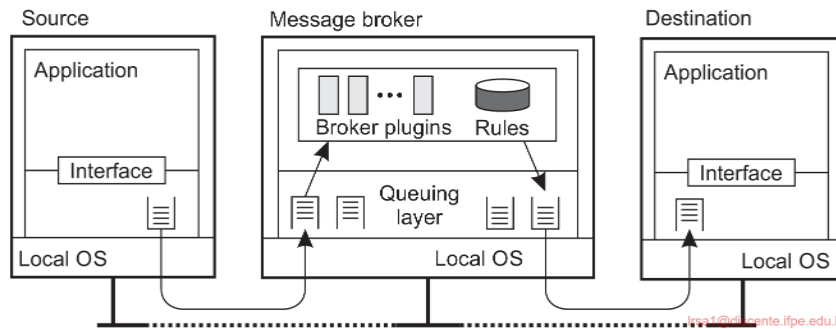


Figure 4.28: The general organization of a message broker in a message-queuing system.

A message broker can be as simple as a reformatter for messages. For example, assume an incoming message contains a table from a database in which records are separated by a special *end-of-record* delimiter and fields within a record have a known, fixed length. If the destination application expects a different delimiter between records, and also expects that fields have variable lengths, a message broker can be used to convert messages to the format expected by the destination.

In a more advanced setting, a message broker may act as an application-level gateway, in which information on the messaging protocol of several applications has been encoded. In general, for each pair of applications, we will have a separate subprogram capable of converting messages between the two applications. In Figure 4.28, this subprogram is drawn as a **plugin** to emphasize that such parts can be dynamically plugged in, or removed from a broker.

Finally, note that often a message broker is used for advanced **enterprise application integration (EAI)**, as we discussed in Section 1.3.2. In this case, rather than (only) converting messages, a broker is responsible for *matching* applications based on the messages that are being exchanged. In such a **publish-subscribe model**, applications send messages in the form of *publishing*. In particular, they may publish a message on topic X, which is then sent to the broker. Applications that have stated their interest in messages on topic X, that is, who have *subscribed* to those messages, will then receive these messages from the broker. More advanced forms of mediation are also possible.

At the heart of a message broker lies a repository of rules for transforming a message of one type to another. The problem is defining the rules and developing the plugins. Most message-broker products come with sophisticated development tools, but the bottom line is still that the repository needs to be filled by experts. Here we see a perfect example where commercial products are often misleadingly said to provide “intelligence,” where, in fact, the intelligence is to be found only in the heads of those experts. It may be better to consistently talk about “advanced” systems (but even then, we often see that this stands for complex or complicated).

Note 4.12 (More information: A note on message-queuing systems)

Considering what we have said about message-queuing systems, it would appear that they have long existed in the form of implementations for e-mail services. E-mail systems are generally implemented through a collection of mail servers that store and forward messages on behalf of the users on hosts directly connected to the server. Routing is generally left out, as e-mail systems can make direct use of the underlying transport services. For example, in the mail protocol for the Internet, SMTP [Postel, 1982], a message is transferred by setting up a direct TCP connection to the destination mail server.

What makes e-mail systems special compared to message-queuing systems is that they are primarily aimed at providing direct support for end users. This explains, for example, why several groupware applications are based directly on an e-mail system [Khoshafian and Buckiewicz, 1995]. In addition, e-mail systems may have very specific requirements such as automatic message filtering, support for advanced messaging databases (e.g., to easily retrieve previously stored messages), and so on.

General message-queuing systems are not aimed at supporting only end users. An important issue is that they are set up to enable persistent communication between processes, regardless of whether a process is running a user application, handling access to a database, performing computations, and so on. This approach leads to a different set of requirements for message-queuing systems than pure e-mail systems. For example, e-mail systems generally need not provide guaranteed message delivery, message priorities, logging facilities, efficient multicasting, load balancing, fault tolerance, and so on for general usage.

General-purpose message-queuing systems, therefore, have a wide range

of applications, including e-mail, workflow, groupware, and batch processing. However, as we have stated before, the most important application area is the integration of a (possibly widely dispersed) collection of databases and applications into a federated information system [Hohpe and Woolf, 2004]. For example, a query expanding several databases may need to be split into subqueries that are forwarded to individual databases. Message-queuing systems assist by providing the basic means to package each subquery into a message and routing it to the appropriate database. Other communication facilities we have discussed in this chapter are far less appropriate.

4.3.4 Example: Advanced Message Queuing Protocol (AMQP)

An interesting observation about message-queuing systems is that they have been developed in part to allow legacy applications to interoperate, yet at the same time we see that when it comes to operations between different message-queuing systems, we often hit a wall. As a consequence, once an organization chooses to use a message-queuing system from manufacturer X, they may have to settle for solutions that only X provides. Message-queuing solutions are thus in large part proprietary solutions. So much for openness.

In 2006, a working group was formed to change this situation, which resulted in the specification of the **Advanced Message-Queuing Protocol**, or simply **AMQP**. There are different versions of AMQP, with version 1.0 being the most recent one. There are also various implementations of AMQP, notably of versions before 1.0, which by the time version 1.0 was established, had gained considerable popularity. Because a pre-1.0 version is so different from the 1.0 version, yet has also a steady user base, we may see various pre-1.0 AMQP servers exist next to (their undeniably incompatible) 1.0 servers.

In this section, we will describe AMQP, but will more or less deliberately mix the pre-1.0 and 1.0 versions, sticking to the essentials and spirit of AMQP. Details can be found in the specifications [AMQP Working Group, 2008; OASIS, 2012]. Implementations of AMQP include RabbitMQ [Roy, 2018] and Apache's ActiveMQ.

Basics

AMQP revolves around applications, queue managers, and queues. Taking an approach that is common for many networking situations, we make a distinction between AMQP as a *messaging service*, the actual *messaging protocol*, and, finally, the *messaging interface* as offered to applications. To this end, it is easiest to consider having only a single queue manager, running as a single, separate server, forming the implementation of AMQP as a service. An application communicates with this queue manager through a local interface.

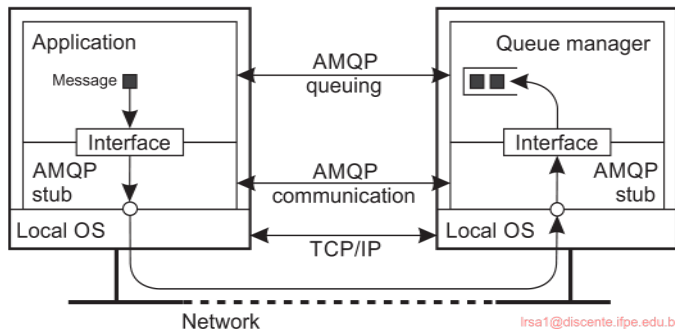


Figure 4.29: An overview of a single-server AMQP instance.

Between an application and the queue manager, communication proceeds according to the AMQP protocol.

This situation is shown in Figure 4.29 and should look familiar. The AMQP stub shields the application (as well as the queue manager) from the details concerning message transfer and communication in general. At the same time, it implements a message-queuing interface, allowing the application to make use of AMQP as a message-queuing service. Although the distinction between AMQP stub and queue manager is made explicit for queue managers, the strictness of the separation is left to an implementation. Nevertheless, if not strict, conceptually there is a distinction between handling queues and handling related communication as we shall make clear shortly.

AMQP communication

AMQP allows an application to set up a **connection** to a queue manager; a connection is a container for a number of one-way **channels**. Whereas the lifetime of a channel can be highly dynamic, connections are assumed to be relatively stable. This difference between connection and channel allows for efficient implementations, notably by using a single transport-layer TCP connection to multiplex many different channels between an application and a queue manager. In practice, AMQP assumes TCP is used for establishing AMQP connections.

Bidirectional communication is established through **sessions**: a logical grouping of two channels. A connection may have multiple sessions, but note that a channel need not necessarily be part of a session.

Finally, to actually transfer messages, a **link** is needed. Conceptually, a link, or rather its end points, keep track of the status of messages that are being transferred. It thus provides fine-grained flow control between an application and a queue manager, and, indeed, different control policies can be put simultaneously in place for different messages that are transferred

through the same session or connection. Flow control is established through credits: a receiver can tell the sender how many messages it is allowed to send over a specific link.

When a message is to be transferred, the application passes it to its local AMQP stub. As mentioned, each message transfer is associated with one specific link. Message transfer normally proceeds in three steps.

1. At the sender's side, the message is assigned a unique identifier and is recorded *locally* to be in an **unsettled state**. The stub subsequently transfers the message to the server, where the AMQP stub also records it as being in an unsettled state. At that point, the server-side stub passes it to the queue manager.
2. The receiving application (in this case the queue manager), is assumed to handle the message and normally reports back to its stub that it is finished. The stub passes this information to the original sender, at which point the message at the original sender's AMQP stub enters a **settled state**.
3. The AMQP stub of the original sender now tells the stub of the original receiver that message transfer has been settled (meaning that the original sender will forget about the message from now on). The receiver's stub can now also discard anything about the message, formally recording it as being settled as well.

Note that because the receiving application can indicate to the underlying AMQP communication layer that it is done with a message, AMQP enables true end-to-end communication reliability. In particular, the application, be it a client application or an actual queue manager, can instruct the AMQP communication layer to keep hold of a message (i.e., a message stays in the unsettled state).

AMQP messaging

Messaging in AMQP logically takes place at the layer above the one handling communication. It is here that an application can indicate what needs to be done with a message, but can also see what has happened so far. Messaging formally takes place between two **nodes**, of which there are three types: a producer, a consumer, or a queue. Typically, producer and consumer nodes represent regular applications, whereas queues are used to store and forward messages. Indeed, a queue manager will typically consist of multiple queue nodes. In order for message transfer to take place, two nodes will have to establish a link between them.

The receiver can indicate to the sender whether its message was accepted (meaning that it was successfully processed), or rejected. Note that this means that a notification is returned to the original sender. AMQP also

```

1 import rabbitpy
2
3 def producer():
4     connection = rabbitpy.Connection() # Connect to RabbitMQ server
5     channel = connection.channel()    # Create new channel on the connection
6
7     exchange = rabbitpy.Exchange(channel, 'exchange') # Create an exchange
8     exchange.declare()
9
10    queue1 = rabbitpy.Queue(channel, 'example1') # Create 1st queue
11    queue1.declare()
12
13    queue2 = rabbitpy.Queue(channel, 'example2') # Create 2nd queue
14    queue2.declare()
15
16    queue1.bind(exchange, 'example-key') # Bind queue1 to a single key
17    queue2.bind(exchange, 'example-key') # Bind queue2 to the same key
18
19    message = rabbitpy.Message(channel, 'Test message')
20    message.publish(exchange, 'example-key') # Publish the message using the key
21    exchange.delete()

```

(a)

```

1 import rabbitpy
2
3 def consumer():
4     connection = rabbitpy.Connection()
5     channel = connection.channel()
6
7     queue = rabbitpy.Queue(channel, 'example1')
8
9     # While there are messages in the queue, fetch them using Basic.Get
10    while len(queue) > 0:
11        message = queue.get()
12        print('Message Q1: %s' % message.body.decode())
13        message.ack()
14
15    queue = rabbitpy.Queue(channel, 'example2')
16
17    while len(queue) > 0:
18        message = queue.get()
19        print('Message Q2: %s' % message.body.decode())
20        message.ack()

```

(b)

Figure 4.30: A simple (a) producer and (b) consumer for RabbitMQ, adapted from [Roy, 2018].

supports fragmentation and assembly of large messages, for which additional notifications are sent.

Of course, an important aspect of AMQP is its support for **persistent messaging**. Achieving persistence is handled through several mechanisms. First, a message can be marked as durable, indicating that the source expects any intermediate node, such as a queue, to be able to recover in the case of a failure. An intermediate node that cannot guarantee such durability will have to reject a message. Second, a source or target node can also indicate its durability: if durable, will it maintain its state, or will it also maintain the (unsettled) state of durable messages? Combining the latter with durable messages effectively establishes reliable message transfer and persistent messaging.

AMQP is truly a messaging protocol in the sense that it does not by itself support, for example, publish-subscribe primitives. It expects that such issues are handled by more advanced, proprietary queue managers, akin to the message brokers discussed in Section 4.3.3.

Finally, there is no reason why a queue manager cannot be connected to another queue manager. In fact, it is quite common to organize queue managers into an overlay network in which messages are routed from producers to their consumers. AMQP does not specify how the overlay network should be constructed and managed, and, indeed, different providers of AMQP-based systems offer different solutions. Of particular importance is specifying how messages should be routed through the network. The bottom line is that administrators will need to do a lot of this specification manually. Only in cases where overlays have regular structures, such as cycles or trees, it becomes easier to provide the necessary routing details.

AMQP in practice

One of the more popular implementations of message brokering is the RabbitMQ server, extensively described by Roy [2018]. The RabbitMQ server is based on the 0.9 version of AMQP, while also fully supporting version 1.0. An important difference between these two versions is the use of **exchanges**. Instead of directly manipulating queues, producers contact an exchange, which, in turn, places messages in one or several queues. In essence, an exchange allows a producer to use a synchronous or asynchronous RPC to a queue manager. An exchange never stores messages; it simply makes sure that a message is placed in an appropriate queue.

To illustrate this concept, consider Figure 4.30(a) showing a simple producer. As mentioned previously, AMQP embeds channels as part of more durable connections, and the first we do is create a means for communicating with the server (we are omitting various details; in this case we establish a default connection with the server). In the following lines, an exchange is created within the realm of the server, along with two queues. In Lines 16

and 17, both queues are bound to the same exchange by a simple key named `example-key`. After creating a message, the producer tells the exchange that it should publish that message to any queue bound by the specified key.

As to be expected, the exchange will place the message in both queues, which is verified by the simple consumer shown in Figure 4.30(b). In this case, the consumer simply fetches messages from either queue and prints their content.

Exchanges and queues have many options, for which we refer the interested reader to Roy [2018]. It is important to note that using systems such as RabbitMQ, but also other messaging brokers, one can generally set up advanced overlay networks for application-level message routing. At the same time, maintenance of those networks can easily become a nightmare.

4.4 Multicast communication

An important topic in communication in distributed systems is the support for sending data to multiple receivers, also known as **multicast communication**. For many years, this topic has belonged to the domain of network protocols, where numerous proposals for network-level and transport-level solutions have been implemented and evaluated [Janic, 2005; Obraczka, 1998]. A major issue in all solutions was setting up the communication paths for information dissemination. In practice, this involved a huge management effort, often requiring human intervention. In addition, as long as there is no convergence of proposals, ISPs have shown to be reluctant to support multicasting [Diot et al., 2000].

With the advent of peer-to-peer technology, and notably structured overlay management, it became easier to set up communication paths. As peer-to-peer solutions are typically deployed at the application layer, various application-level multicasting techniques have been introduced. In this section, we will take a brief look at these techniques.

Multicast communication can also be accomplished in other ways than setting up explicit communication paths. As we also explore in this section, gossip-based information dissemination provides simple (yet often less efficient) ways for multicasting.

4.4.1 Application-level tree-based multicasting

The basic idea in **application-level multicasting** is that nodes are organized into an overlay network, which is then used to disseminate information to its members. An important observation is that network routers are not involved in group membership. As a consequence, the connections between nodes in the overlay network may cross several physical links, and as such, routing messages within the overlay may not be optimal in comparison to what could have been achieved by network-level routing.

A crucial design issue is the construction of the overlay network. In essence, there are two approaches [El-Sayed et al., 2003; Hosseini et al., 2007; Allani et al., 2009]. First, nodes may organize themselves directly into a tree, meaning that there is a unique (overlay) path between every pair of nodes. An alternative approach is that nodes organize into a mesh network in which every node will have multiple neighbors and, in general, there exist multiple paths between every pair of nodes. The main difference between the two is that the latter generally provides higher robustness: if a connection breaks (e.g., because a node fails), there will still be an opportunity to disseminate information without having to immediately reorganize the entire network.

Note 4.13 (Advanced: Constructing a multicast tree in Chord)

To make matters concrete, let us consider a relatively simple scheme for constructing a multicast tree in Chord, which we described in Note 2.6. This scheme was originally proposed for Scribe [Castro et al., 2002b] which is an application-level multicasting scheme built on top of Pastry [Rowstron and Druschel, 2001]. The latter is also a DHT-based peer-to-peer system.

Assume a node wants to start a multicast session. To this end, it simply generates a multicast identifier, say, mid which is just a randomly chosen 160-bit key. It then looks up $succ(mid)$, which is the node responsible for that key, and promotes it to become the root of the multicast tree that will be used for sending data to interested nodes. To join the tree, a node P simply executes the operation $lookup(mid)$ having the effect that a lookup message with the request to join the multicast group mid will be routed from P to $succ(mid)$. The routing algorithm itself will be explained in detail in Chapter 6.

On its way toward the root, the join request will pass several nodes. Assume it first reaches node Q . If Q had never seen a join request for mid before, it will become a **forwarder** for that group. At that point, P will become a child of Q whereas the latter will continue to forward the join request to the root. If the next node on the root, say, R is also not yet a forwarder, it will become one and record Q as its child and continue to send the join request.

On the other hand, if Q (or R) is already a forwarder for mid , it will also record the previous sender as its child (i.e., P or Q , respectively), but there will not be a need to send the join request to the root anymore, as Q (or R) will already be a member of the multicast tree.

Nodes such as P that have explicitly requested to join the multicast tree are, by definition, also forwarders. The result of this scheme is that we construct a multicast tree across the overlay network with two types of nodes: pure forwarders that act as helpers, and nodes that are also forwarders, but have explicitly requested to join the tree. Multicasting is now simple: a node merely sends a multicast message toward the root of the tree by again executing the $lookup(mid)$ operation, after which that message can be sent along the tree.

We note that this high-level description of multicasting in Scribe does not do complete justice to its original design. The interested reader is encouraged to take a look at the details, which can be found in [Castro et al., 2002b].

Performance issues in overlays

From the high-level description given above, it should be clear that although building a tree by itself is not that difficult once we have organized the nodes into an overlay, building an efficient tree may be a different story. Note that in our description so far, the selection of nodes that participate in the tree does not take into account any performance metrics: it is purely based on the (logical) routing of messages through the overlay.

To understand the problem at hand, take a look at Figure 4.31 which shows a small set of five nodes that are organized in a simple overlay network, with the node A forming the root of a multicast tree. The costs for traversing a physical link are also shown. Now, whenever A multicasts a message to the other nodes, the *logical* route, i.e., at the level of the overlay, is simply $A \rightarrow B \rightarrow E \rightarrow D \rightarrow C$. The actual route crosses the network-level links in the following order: $A \rightarrow Ra \rightarrow Rb \rightarrow B \rightarrow Rb \rightarrow Ra \rightarrow Re \rightarrow E \rightarrow Re \rightarrow Rc \rightarrow Rd \rightarrow D \rightarrow Rd \rightarrow Rc \rightarrow C$. It is thus seen that this message will traverse each of the links $\langle B, Rb \rangle$, $\langle Ra, Rb \rangle$, $\langle E, Re \rangle$, $\langle Rc, Rd \rangle$, and $\langle D, Rd \rangle$ twice. The overlay network would have been more efficient if we had not constructed *overlay links* $\langle B, E \rangle$, and $\langle D, E \rangle$, but instead $\langle A, E \rangle$ and $\langle C, E \rangle$. Such a configuration would have saved the double traversal across physical links $\langle Ra, Rb \rangle$ and $\langle Rc, Rd \rangle$.

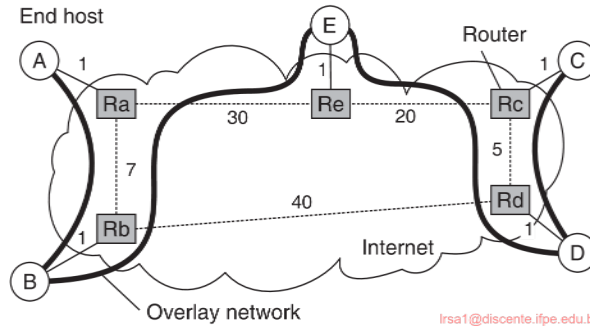


Figure 4.31: The relation between links in an overlay and network-level routes.

The quality of an application-level multicast tree is generally measured by three different metrics: link stress, stretch, and tree cost. **Link stress** is defined per link and counts how often a packet crosses the same link [Chu et al., 2002]. A link stress greater than 1 comes from the fact that although at a logical level a packet may be forwarded along two different connections, part of those connections may actually correspond to the same physical link, as we showed in Figure 4.31.

The **stretch** or **relative delay penalty (RDP)** measures the ratio of the delay between two nodes in the overlay, and the delay that those two nodes would experience in the underlying network. For example, messages from

B to C follow the route $B \rightarrow R_b \rightarrow R_a \rightarrow R_e \rightarrow E \rightarrow R_e \rightarrow R_c \rightarrow R_d \rightarrow D \rightarrow R_d \rightarrow R_c \rightarrow C$ in the overlay network, having a total cost of 73 units. However, messages would have been routed in the underlying network along the path $B \rightarrow R_b \rightarrow R_d \rightarrow R_c \rightarrow C$, with a total cost of 47 units, leading to a stretch of 1.55. Obviously, when constructing an overlay network, the goal is to minimize the aggregated stretch, or similarly, the average RDP measured over all node pairs.

Finally, the **tree cost** is a global metric, generally related to minimizing the aggregated link costs. For example, if the cost of a link is taken to be the delay between its two end nodes, then optimizing the tree cost boils down to finding a minimal spanning tree in which the total time for disseminating information to all nodes is minimal.

To simplify matters somewhat, assume that a multicast group has an associated and well-known node that keeps track of the nodes that have joined the tree. When a new node issues a join request, it contacts this **rendezvous node** to obtain a (potentially partial) list of members. The goal is to select the best member that can operate as the new node's parent in the tree. Who should it select? There are many alternatives, and different proposals often follow very different solutions.

Consider, for example, a multicast group with only a single source. In this case, the selection of the best node is obvious: it should be the source (because in that case, we can be assured that the stretch will be equal to 1). However, in doing so, we would introduce a star topology with the source in the middle. Although simple, it is not difficult to imagine the source may easily become overloaded. In other words, selection of a node will generally be constrained in such a way that only those nodes may be chosen who have k or fewer neighbors, with k being a design parameter. This constraint severely complicates the tree-establishment algorithm, as a suitable solution may require that part of the existing tree is reconfigured. Tan et al. [2003] provide an extensive overview and evaluation of various solutions to this problem.

Note 4.14 (Advanced: Switch-trees)

As an illustration, let us take a closer look at one specific family, known as **switch-trees** [Helder and Jamin, 2002]. The basic idea is simple. Assume we already have a multicast tree with a single source as root. In this tree, a node P can switch parents by dropping the link to its current parent in favor of a link to another node. The only constraints imposed on switching links is that the new parent can never be a member of the subtree rooted at P (as this would partition the tree and create a loop), and that the new parent will not have too many immediate children. This last requirement is needed to limit the load of forwarding messages by any single node.

There are different criteria for deciding to switch parents. A simple one is to

optimize the route to the source, effectively minimizing the delay when a message is to be multicast. To this end, each node regularly receives information on other nodes (we will explain one specific way of doing this below). At that point, the node can evaluate whether another node would be a better parent in terms of delay along the route to the source, and if so, initiates a switch.

Another criterion could be whether the delay to the potential other parent is lower than to the current parent. If every node takes this as a criterion, then the aggregated delays of the resulting tree should ideally be minimal. In other words, this is an example of optimizing the cost of the tree, as we explained above. However, more information would be needed to construct such a tree, but as it turns out, this simple scheme is a reasonable heuristic leading to a good approximation of a minimal spanning tree.

As an example, consider the case where a node P receives information on the neighbors of its parent. Note that the neighbors consist of P's grandparent, along with the other siblings of P's parent. Node P can then evaluate the delays to each of these nodes and subsequently choose the one with the lowest delay, say Q, as its new parent. To that end, it sends a switch request to Q. To prevent loops from being formed due to concurrent switching requests, a node that has an outstanding switch request will simply refuse to process any incoming requests. In effect, this leads to a situation where only completely independent switches can be carried out simultaneously. Furthermore, P will provide Q with enough information to allow the latter to conclude that both nodes have the same parent, or that Q is the grandparent.

An important problem that we have not yet addressed is node failure. In the case of switch-trees, a simple solution is proposed: whenever a node notices that its parent has failed, it attaches itself to the root. At that point, the optimization protocol can proceed as usual and will eventually place the node at a good point in the multicast tree. Experiments described in [Helder and Jamin, 2002] show that the resulting tree is indeed close to a minimal spanning one.

4.4.2 Flooding-based multicasting

So far, we have assumed that when a message is to be multicast, it is to be received by every node in the overlay network. Strictly speaking, this corresponds to **broadcasting**. In general, multicasting refers to sending a message to a subset of all the nodes, that is, a specific *group of nodes*. A key design issue when it comes to multicasting is to minimize the use of intermediate nodes for which the message is not intended. To make this clear, if the overlay is organized as a multi-level tree, yet only the leaf nodes are the ones who should receive a multicast message, then clearly there may be quite some nodes who need to store and subsequently forward a message that is not meant for them.

One simple way to avoid such inefficiency, is to construct an overlay network *per multicast group*. As a consequence, multicasting a message m to a

group G is the same as broadcasting m to G . The drawback of this solution is that a node belonging to several groups, will, in principle, need to maintain a separate list of its neighbors for each group of which it is a member.

If we assume that an overlay corresponds to a multicast group, and thus that we need to broadcast a message, a naive way of doing so is to apply **flooding**. In this case, each node simply forwards a message m to each of its neighbors, except to the one from which it received m . Furthermore, if a node keeps track of the messages it received and forwarded, it can simply ignore duplicates.

To understand the performance of flooding, we model an overlay network as a connected undirected graph $G = (V, E)$ with nodes V and links E . Let v_0 be the node that initiates flooding. In principle, every node other than v_0 will send out the message to its neighbors once, except to the neighbor from which it had received the message. We will thus see a total of $\delta(v_0) + \sum_{v \in V - v_0} (\delta(v) - 1)$ messages, with $\delta(v)$ the number of neighbors of the node v . For any undirected graph, $\sum_{v \in V} \delta(v) = 2 \cdot |E|$, with $|E|$ denoting the total number of edges. In other words, we will see $2|E| - |V| + 1$ messages being sent, making flooding quite inefficient. Only if G is a tree, will flooding be optimal, for in that case, $|E| = |V| - 1$. In the worst case, when G is fully connected, we have $|E| = \binom{|V|}{2}$ leading to an order of $|V|^2$ messages.

Suppose now that we have no information on the structure of the overlay network and that the best we can assume is that it can be represented as a **random graph**, which (to keep it simple) is a graph having a probability p_{edge} that two vertices are joined by an edge, also known as an **Erdős-Rényi graph** [Erdős and Rényi, 1959]. Note that we are actually considering our overlay network to be an unstructured peer-to-peer network, and that we do not have any information on how it is being constructed. With a probability p_{edge} that two nodes are joined, and a total of $\binom{|V|}{2}$ edges, it is not difficult to see that we can expect our overlay to have $|E| = \frac{1}{2} \cdot p_{edge} \cdot |V| \cdot (|V| - 1)$ edges. To give an impression of what we are dealing with, Figure 4.32 shows the relationship between the number of nodes and edges for different values of p_{edge} . As can be seen, the number of edges can easily become large, even for small values of p_{edge} .

To reduce the number of messages, we can also use **probabilistic flooding** as introduced by Banaei-Kashani and Shahab [2003] and formally analyzed by Oikonomou and Stavrakakis [2007]. The idea is simple: when a node is flooding a message m and needs to forward m to a specific neighbor, it will do so with a probability p_{flood} . The effect can be dramatic: the total number of messages sent will drop linearly in p_{flood} . However, there is also a risk: the lower p_{flood} , the higher the chance that not all nodes in the network will be reached. This risk is caused by the simple fact that all neighbors of a specific node Q may have decided *not* to forward m to Q . If Q has n neighbors, then this can happen roughly with a probability of $(1 - p_{flood})^n$. Clearly, the

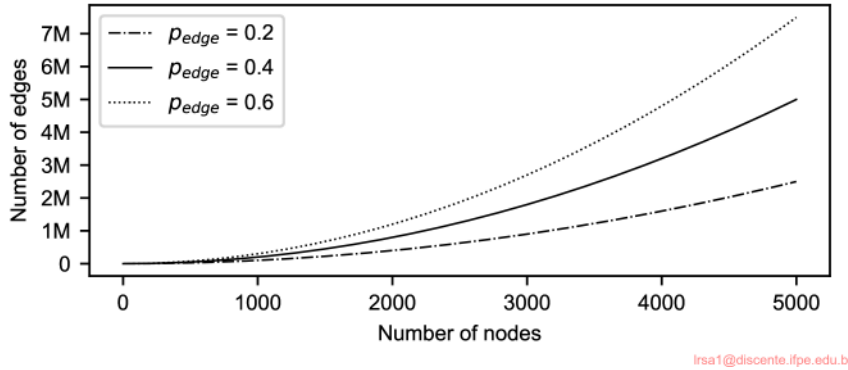


Figure 4.32: The size of a random overlay as function of the number of nodes.

number of neighbors plays an important role in deciding whether to forward a message, and, indeed, we can replace the static probability of forwarding with one that takes the degree of the neighbor into account. This has been further developed and analyzed by Sereno and Gaeta [2011]. To give an idea of the efficiency of probabilistic broadcasting: in a random network of 10,000 nodes and $p_{edge} = 0.1$, we need only set $p_{flood} = 0.01$ to establish a more than 50-fold reduction in the number of messages sent in comparison to full flooding.

When dealing with a structured overlay, that is, one having a more or less deterministic topology, designing efficient flooding schemes is simpler. As an example, consider an n -dimensional hypercube, shown in Figure 4.33 for the case $n = 4$, as also discussed in Section 2.4.1.

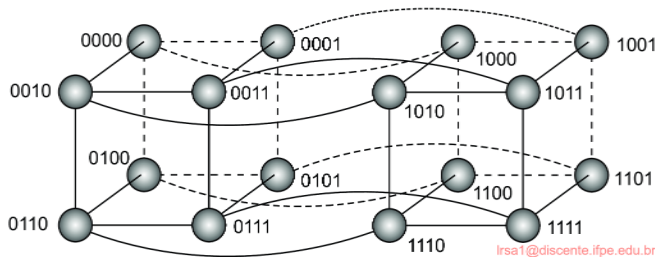


Figure 4.33: A simple peer-to-peer, four-dimensional hypercube.

A simple and efficient broadcast scheme has been designed by Schlosser et al. [2002] and relies on keeping track of neighbors *per dimension*. This is best explained by considering that every node in an n -dimensional hypercube is represented by a bit string of length n . Each edge in the overlay is labeled with its dimension. For the case $n = 4$, node 0000 will have as its neighbors the set $\{0001, 0010, 0100, 1000\}$. The edge between 0000 and 0001 is labeled

“4” corresponding to changing the 4th bit when comparing 0000 to 0001 and *vice versa*. Likewise, the edge $\langle 0000, 0100 \rangle$ is labeled “2,” and so forth. A node initially broadcasts a message m to all of its neighbors, and tags m with the label of the edge over which it sends the message. In our example, if node 1001 broadcasts a message, it will send the following:

- $(m,1)$ to 0001
- $(m,2)$ to 1101
- $(m,3)$ to 1011
- $(m,4)$ to 1000

When a node receives a broadcast message, it will forward it only along edges that have a higher dimension. In other words, in our example, node 1101 will forward m only to nodes 1111 (joined to 1101 by an edge labeled “3”) and 1100 (joined by an edge with label “4”). Using this scheme, it can be shown that every broadcast requires precisely $N - 1$ messages, where $N = 2^n$, that is the number of nodes in a n -dimensional hypercube. This broadcasting scheme is therefore optimal in terms of the number of messages sent.

Note 4.15 (Advanced: Ring-based flooding)

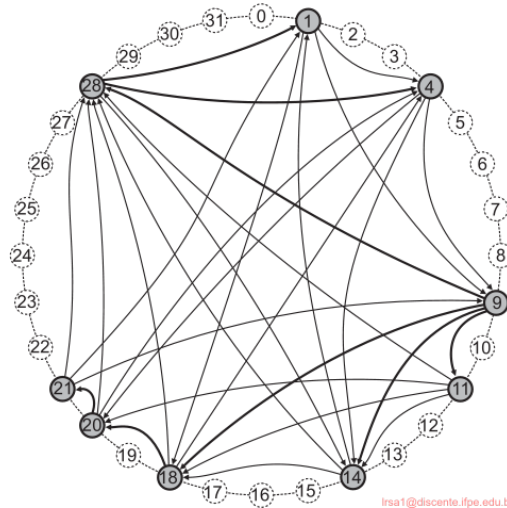


Figure 4.34: A Chord ring in which node 9 broadcasts a message.

A hypercube is a straightforward example of how we can effectively use knowledge of the structure of an overlay network to establish efficient flooding. In the case of Chord, we can follow an approach proposed by Ghodsi [2010]. Recall that in Chord each node is identified by a number p , and each resource (typically

a file), is assigned a key k from the same space as used for node identifiers. The successor $\text{succ}(k)$ of a key k is the node with the smallest identifier $p \geq k$. Consider the small Chord ring shown in Figure 4.34 and assume that node 9 wants to flood a message to all other nodes.

In our example, node 9 divides the identifier space into four segments (one for each of its neighbors). Node 28 is requested to make sure that the message reaches all nodes with identifiers $28 \leq k < 9$ (recall that we are applying modulo arithmetic); node 18 takes care of nodes with identifiers $18 \leq k < 28$; node 14 for $14 \leq k < 18$; and 11 for identifiers $11 \leq k < 14$.

Node 28 will subsequently divide the part of the identifier space it is requested to handle into two subsegments: one for its neighboring node 1 and another for 4. Likewise, node 18, responsible for segment $[18, 28)$ will “split” that segment into only one part: it has only one neighbor to delegate the flood to, and forwards the message to node 20, telling it that it should handle segment $[20, 28)$.

In the last step, only node 20 has work to do. It forwards the message to node 21, telling it to forward it to nodes known to it in the segment $[21, 28)$. As there are no such nodes anymore, the broadcast completes.

As in the case of our hypercube example, we see that flooding is done with $N - 1$ messages, with N being the number of nodes in the system.

4.4.3 Gossip-based data dissemination

An important technique for disseminating information is to rely on **epidemic behavior**, also referred to as **gossiping**. Observing how diseases spread among people, researchers have since long investigated whether simple techniques could be developed for spreading information in very large-scale distributed systems. The main goal of these **epidemic protocols** is to rapidly propagate information among a large collection of nodes using only local information. In other words, there is no central component by which information dissemination is coordinated.

To explain the general principles of these algorithms, we assume that all updates for a specific data item are initiated at a single node. In this way, we simply avoid write-write conflicts. The following presentation is based on the classical paper by Demers et al. [1987] on epidemic algorithms. An overview of epidemic information dissemination can be found in [Eugster et al., 2004].

Information dissemination models

As the name suggests, epidemic algorithms are based on the theory of epidemics, which studies the spreading of infectious diseases. In the case of large-scale distributed systems, instead of spreading diseases, they spread information. Research on epidemics for distributed systems also aims at an entirely different goal: whereas health organizations will do their best to prevent infectious diseases from spreading across large groups of people,

designers of epidemic algorithms for distributed systems will try to “infect” all nodes with new information as fast as possible.

Using the terminology from epidemics, a node that is part of a distributed system is called **infected** if it holds data that it is willing to spread to other nodes. A node that has not yet seen this data is called **susceptible**. Finally, an updated node that is not willing or able to spread its data is said to have been **removed**. Note that we assume we can distinguish old from new data, for example because it has been timestamped or versioned. In this light, nodes are also said to spread updates.

A popular propagation model is that of **anti-entropy**. In this model, a node P picks another node Q at random, and subsequently exchanges updates with Q. There are three approaches to exchanging updates:

1. P only pulls in new updates from Q
2. P only pushes its own updates to Q
3. P and Q send updates to each other (i.e., a push-pull approach)

When it comes to rapidly spreading updates, only pushing updates turns out to be a bad choice. Intuitively, this can be understood as follows. First, note that in a pure push-based approach, updates can be propagated only by infected nodes. However, if many nodes are infected, the probability of each one selecting a susceptible node is relatively small. Consequently, chances are that a particular node remains susceptible for a long period simply because an infected node does not select it.

In contrast, the pull-based approach works much better when many nodes are infected. In that case, spreading updates is essentially triggered by susceptible nodes. Chances are big that such a node will contact an infected one to subsequently pull in the updates and become infected as well.

If only a single node is infected, updates will rapidly spread across all nodes using either form of anti-entropy, although push-pull remains the best strategy [Jelasity et al., 2007]. Define a **round** as spanning a period in which every node will have taken the initiative once to exchange updates with a randomly chosen other node. It can then be shown that the number of rounds to propagate a single update to all nodes is of the order $\mathcal{O}(\log(N))$, where N is the number of nodes in the system. This indicates indeed that propagating updates is fast, but above all scalable.

Note 4.16 (Advanced: An analysis of anti-entropy)

A simple and straightforward analysis will give some idea on how well anti-entropy works. Consider a system with N nodes. One of these nodes initiates the spreading of a message m to all other nodes. Let p_i denote the probability that a node P has not yet received m after the i^{th} round. We distinguish the following three cases:

- With a pure pull-based approach, $p_{i+1} = (p_i)^2$: not only had P not yet been updated in the previous round, also P contacts a node that had not yet received m.
- With a pure push-based approach, $p_{i+1} = p_i \cdot (1 - \frac{1}{N-1})^{N(1-p_i)}$: again, P should not have been updated in the previous round, but also none of the updated nodes should contact P. The probability that not a single node contacts P is $1 - \frac{1}{N-1}$; we can expect that there are $N(1 - p_i)$ updated nodes in round i .
- In a push-pull approach, we can simply combine the two: P should not contact an updated node, and should not be contacted by one.

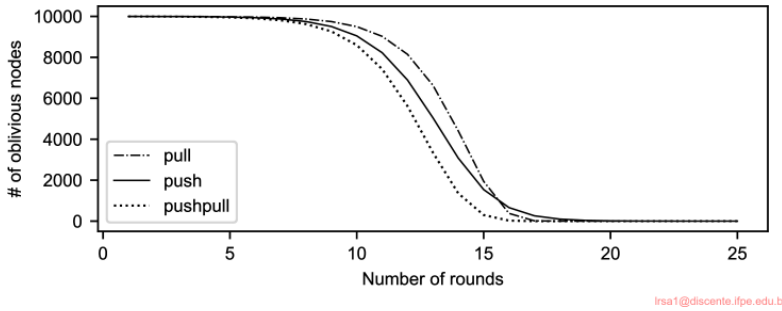


Figure 4.35: (a) The number of nodes that have not been updated as a function of the number of dissemination rounds.

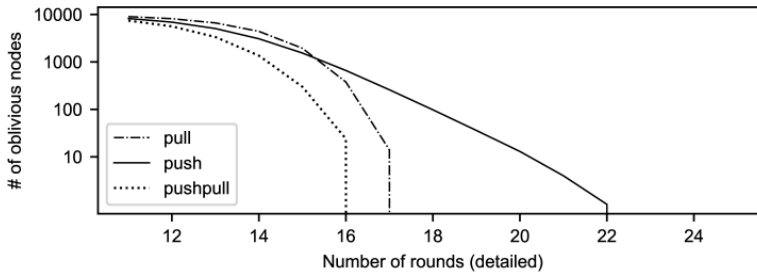


Figure 4.35: (b) Zooming in the differences between the three anti-entropy approaches when almost all nodes have been updated.

Figure 4.35(a) shows how quickly the probability of not yet being updated drops as a function of the number of rounds in a network of 10,000 nodes. After a slow start, the information is rapidly disseminated. In Figure 4.35(b) we have zoomed into what happens in the last number of rounds while also using a log scale for the y-axis. Indeed, assuming that nodes are up-and-running all the time,

it turns out that an anti-entropy push-pull strategy can be an extremely effective dissemination protocol.

One specific variant of epidemic protocols is called **rumor spreading**. It works as follows. If node P has just been updated for a data item x , it contacts an arbitrary other node Q and tries to push the update to Q. However, it is possible that Q was already updated by another node. In that case, P may lose interest in spreading the update any further, say with probability p_{stop} . In other words, it then becomes removed.

Rumor spreading is gossiping as we mostly experience it in real life. When Bob has some hot news to spread around, he may phone his friend Alice, telling her all about it. Alice, like Bob, will be really excited to spread the rumor to her friends as well. However, she will become disappointed when phoning a friend, say Chuck, only to hear that the news has already reached him. Chances are that she will stop phoning other friends, for what good is it if they already know?

Rumor spreading turns out to be an excellent way of rapidly spreading news. However, it cannot guarantee that all nodes will actually be updated [Demers et al., 1987]. In fact, when there are many nodes that participate in the epidemics, the fraction s of nodes that will remain ignorant of an update, that is, remain susceptible, satisfies the equation:

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

To get an idea of what this means, take a look at Figure 4.36, which shows s as a function of p_{stop} . Even for high values of p_{stop} we see that the fraction of nodes that remains ignorant is relatively low, and always less than approximately 0.2. For $p_{stop} = 0.20$ it can be shown that $s = 0.0025$. However, in those cases when p_{stop} is relatively high, additional measures will need to be taken to ensure that *all* nodes are updated.

Note 4.17 (Advanced: Analysis of rumor spreading)

To formally analyze the situation for rumor spreading, we let s denote the fraction of nodes that have not yet been updated, i.e., the fraction of susceptible nodes. Likewise, let i denote the fraction of infected nodes: the ones that have been updated and are still contacting other nodes to spread information. Finally, r is the fraction of nodes that have been updated, but have given up, i.e., they are passive and no longer play a role in disseminating information. Obviously, $s + i + r = 1$. Using theory from epidemics, it is not difficult to see the following:

$$\begin{aligned} (1) \quad ds/dt &= -s \cdot i \\ (2) \quad di/dt &= s \cdot i - p_{stop} \cdot (1-s) \cdot i \\ \Rightarrow \quad di/ds &= -(1 + p_{stop}) + \frac{p_{stop}}{s} \\ \Rightarrow \quad i(s) &= -(1 + p_{stop}) \cdot s + p_{stop} \cdot \ln(s) + C \end{aligned}$$

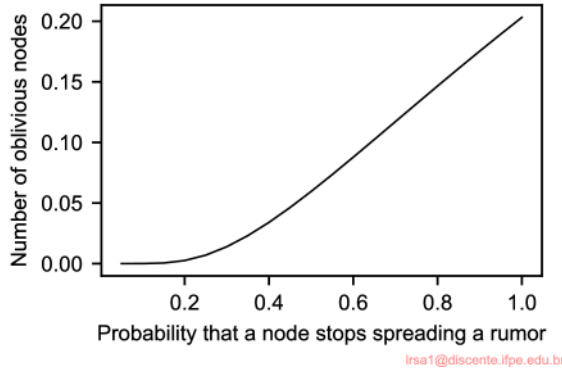


Figure 4.36: The relation between the fraction s of update-ignorant nodes and the probability p_{stop} that a node will stop gossiping once it contacts a node that has already been updated.

where we use the notation $i(s)$ to express i as a function of s . When $s = 1$, no nodes have yet been infected, meaning that $i(1) = 0$. This allows us to derive that $C = 1 + p_{stop}$, and thus

$$i(s) = (1 + p_{stop}) \cdot (1 - s) + p_{stop} \cdot \ln(s)$$

We are looking for the situation that there is no more rumor spreading, i.e., when $i(s) = 0$. Having a closed expression for $i(s)$ then leads to

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

One of the main advantages of epidemic algorithms is their scalability since the number of synchronizations between processes is relatively small compared to other propagation methods. For wide-area systems, Lin and Marzullo [1999] have shown that it makes sense to take the actual network topology into account to achieve better results. In that case, nodes that are connected to only a few other nodes are contacted with a relatively high probability. The underlying assumption is that such nodes form a bridge to other remote parts of the network; therefore, they should be contacted as soon as possible. This approach is referred to as **directional gossiping** and comes in different variants.

This problem touches upon an important assumption that most epidemic solutions make, namely that a node can randomly select any other node to gossip with. This implies that, in principle, the complete set of nodes should be known to each member. In a large system, this assumption can never hold, and special measures will need to be taken to mimic such properties. We return to this issue in Section 5.5.2 when we discuss a **peer-sampling service**.

Removing data

Epidemic algorithms are fantastic for spreading updates. However, they have a rather strange side effect: spreading the *deletion* of a data item is hard. The essence of the problem lies in the fact that deletion of a data item destroys all information on that item. Consequently, when a data item is simply removed from a node, that node will eventually receive old copies of the data item and interpret those as updates on something it did not have before.

The trick is to record the deletion of a data item as just another update, and keep a record of that deletion. In this way, old copies will not be interpreted as something new, but merely treated as versions that have been updated by a delete operation. The recording of a deletion is done by spreading **death certificates**.

Of course, the problem with death certificates is that they should eventually be cleaned up, or otherwise each node will gradually build a huge local database of historical information on deleted data items that is otherwise not used. Demers et al. [1987] propose to use what are called **dormant death certificates**. Each death certificate is timestamped when it is created. If it can be assumed that updates propagate to all nodes within a known finite time, then death certificates can be removed after this maximum propagation time has elapsed.

However, to provide hard guarantees that deletions are indeed spread to all nodes, only a very few nodes maintain dormant death certificates that are never thrown away. Assume node P has such a certificate for a data item x. If by any chance an obsolete update for x reaches P, P will react by simply spreading the death certificate for x again.

4.5 Summary

Having powerful and flexible facilities for communication between processes is essential for any distributed system. In traditional network applications, communication is often based on the low-level message-passing primitives offered by the transport layer. An important issue in middleware systems is to offer a higher level of abstraction that will make it easier to express communication between processes than the support offered by the interface to the transport layer.

One of the most widely used abstractions is the Remote Procedure Call (RPC). The essence of an RPC is that a service is implemented through a procedure, of which the body is executed at a server. The client is offered only the signature of the procedure, that is, the procedure's name along with its parameters. When the client calls the procedure, the client-side implementation, called a stub, takes care of wrapping the parameter values into a message and sending that to the server. The latter calls the actual procedure and returns the results, again in a message. The client's stub

extracts the result values from the return message and passes it back to the calling client application.

RPCs offer synchronous communication facilities, by which a client is blocked until the server has sent a reply. Although variations of either mechanism exist by which this strict synchronous model is relaxed, it turns out that general-purpose, high-level message-oriented models are often more convenient.

In message-oriented models, the issues are whether communication is persistent, and whether communication is synchronous. The essence of persistent communication is that a message that is submitted for transmission, is stored by the communication system as long as it takes to deliver it. In other words, neither the sender nor the receiver needs to be up and running for message transmission to take place. In transient communication, no storage facilities are offered so that the receiver must be prepared to accept the message when it is sent.

In asynchronous communication, the sender is allowed to continue immediately after the message has been submitted for transmission, possibly before it has even been sent. In synchronous communication, the sender is blocked at least until a message has been received. Alternatively, the sender may be blocked until message delivery has taken place or even until the receiver has responded, as with RPCs.

Message-oriented middleware models generally offer persistent asynchronous communication, and are used where RPCs are not appropriate. They are often used to assist the integration of (widely dispersed) collections of databases into large-scale information systems.

Finally, an important class of communication protocols in distributed systems is multicasting. The basic idea is to disseminate information from one sender to multiple receivers. We have discussed two different approaches. First, multicasting can be achieved by setting up a tree from the sender to the receivers. Considering that it is now well understood how nodes can self-organize into peer-to-peer system, solutions have also appeared to dynamically set up trees in a decentralized fashion. Second, flooding messages across the network is extremely robust, yet requires special attention if we want to avoid severe waste of resources as nodes may see messages multiple times. Probabilistic flooding, by which a node forwards a message with a certain probability, often proves to combine simplicity and efficiency, while being highly effective.

Another important class of dissemination solutions deploys epidemic protocols. These protocols have proven to be simple and extremely robust.